

The Limits of Tractability: Vertex Cover

Lecturer: Seth Gilbert

August 22, 2024

Abstract

Today we are talking about the problem of *vertex cover*. Vertex cover is a classic NP-hard problem, and to solve it, we need to compromise. We look at three approaches. First, we consider restricting our attention to a special case: a tree. Then, we look at an exponential algorithm parameterized by the size of the vertex cover. Finally, we looked at approximation algorithms (both a simple randomized approach and a deterministic approach).

1 Overview

Today we will study the problem of **vertex cover**, a classical NP-hard problem. Vertex cover is a great model problem to think about at the beginning of the semester because, on the one hand, it has a relatively simple structure and the algorithms we will see today are quite simple; on the other hand, it illustrates several of the basic techniques we will use this semester, and many problems that show up in the real world are quite similar to vertex cover.

We begin by defining the problem.

Definition 1 A **vertex cover** for a graph $G = (V, E)$ is a set $S \subseteq V$ such that for every edge $e = (u, v) \in E$, either $u \in S$ or $v \in S$.

That is, every edge is covered by one of the nodes in the set S .

Definition 2 The **Minimum Vertex Cover** problem is defined as follows: given a graph $G = (V, E)$, find a minimum-sized set S that is a vertex cover for G .

Imagine, for example, that you are given a map of Singapore and you want to choose where to open Starbucks coffee shops to ensure that, no matter where you are in Singapore, there is a Starbucks on a nearby corner. (Assume that you have decided to open your coffee shops only at an intersection.) If you find a vertex cover, you can be sure that no one in Singapore is too far away from a Starbucks! See Figure 1 for an example.

As we will see shortly, Vertex Cover is a hard problem—specifically it is NP-hard. We are not likely to come up with an efficient algorithm (unless $P = NP$).

When you have a hard problem like vertex cover, there are three things that you want a solution to be:

- Fast (i.e., polynomial time)
- Optimal (i.e., yielding the best solution possible)
- Universal (i.e., good for all instances/inputs)

When you have an NP-hard problem, you can only have two of these three! You can sacrifice speed, and have an algorithm that runs in exponential time. You can sacrifice optimality, and explore heuristics and approximation algorithms—try to find a solution that is *good enough*¹. Or you can sacrifice universality, and focus on solving special cases. Today, we will talk today about these three different approaches for coping with an otherwise intractable problem.

¹“Le mieux est l’ennemi du bien,” according to Voltaire. (“The best is the enemy of the good.”)

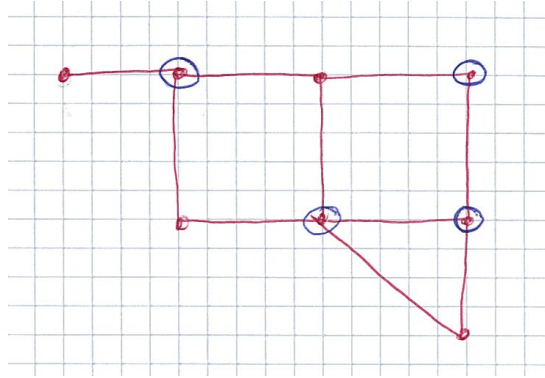


Figure 1: Example of a graph with a vertex cover of size 4. The circled nodes are in the vertex cover.

- First, we consider the case where the input to your problem is somehow special (and easier than the general case); specifically, we will look at finding a vertex cover on a tree. (Here we sacrifice universality.)
- Second, we will see a *parameterized* solution that can guarantee good performance when the vertex cover being sought is small (e.g., we cannot afford to open more than 10 Starbucks, regardless). In this case, we given an algorithm that achieves an exponential-time solution that is exponential only in the parameter k , not the size of the problem n . (Here we sacrifice speed: it is exponential-time, but hopefully reasonable.)
- Finally, we will consider approximation algorithms that do not find an *optimal* solution, but a find a solution that is not too far from optimal. We will discuss a randomized 2-approximation algorithm, and also a deterministic 2-approximation algorithm. (Here we sacrifice optimality, finding a solution that is “good enough.”)

2 NP-completeness

Before we get to algorithms, let us quickly review the proof that Vertex Cover is NP-complete.²

It is relatively easy to show that Vertex Cover (as a decision problem) is NP-complete. First, it is easy to observe that it is in NP: given a set S of size at most k , we can easily verify whether or not it is a vertex cover simply by checking whether, for every edge, one of the two endpoints can be found in S .

Next, we have to show that it is NP-hard. We can show this by reducing some other NP-hard problem to Vertex Cover, thus showing that solving Vertex Cover is at least as hard as solving this other problem. The original NP-complete problem is 3SAT, and we could reduce 3SAT to Vertex Cover. Instead we focus on a different problem, that of finding the largest clique³ in a graph.

Definition 3 The **Max-Clique** problem is defined as follows: given a graph $G = (V, E)$ and an integer k , is there a subset $K \subseteq V$ of size at least k such that K is a clique in G ?

The goal of the Max-Clique problem is to determine whether a given graph G contains a clique of size at least k . Max-Clique is known to be NP-complete. We can show that Vertex Cover is NP-hard by giving an algorithm for

²To be a bit pedantic, it is actually a mistake to refer to Vertex Cover as NP-complete. The complexity class NP refers only to decision problems, not to optimization problems. The version of Vertex Cover that is NP-complete is as follows: given a graph $G = (V, E)$ and a parameter k , does there exist a vertex cover of G of size at most k ? Clearly, if we could efficiently find a minimum sized vertex cover, then we could also efficiently answer the decision version of the question. Hence finding a minimum sized vertex cover is at least as hard as the decision version, and hence we term it NP-hard.

³A clique is a set of nodes $K \subseteq V$ such that every pair of nodes in K is connected by an edge.

solving Max-Clique by using Vertex Cover. First, we need to define the **complement** of a graph G , i.e., the graph that contains only the edges that are *not* in G :

Definition 4 Given a graph $G = (V, E)$ we define the **complement** of G to be the graph $G' = (V, E')$ where edge $e \in E'$ if and only if $e \notin E$.

We can now give an algorithm for solving the Max-Clique problem. Assume we have an algorithm VC which solves the problem of vertex cover. On an input of graph $G = (V, E)$ with n nodes, and parameter k , execute the vertex cover algorithm on the complement of G ; return **true** if and only if G' has a vertex cover of size $\leq n - k$. The correctness

```

1 Algorithm: MaxClique( $G = (V, E), k$ )
2 Procedure:
3   Let  $G'$  be the complement of  $G$ .
4   Execute VertexCover on  $G'$  with parameter  $n - k$ .
5   if  $\exists$  vertex cover of  $G'$  of size  $\leq n - k$  then
6     return true.
7   else return false.

```

of this algorithm for Max-Clique immediately shows that Vertex Cover is NP-complete and depends on the following claim:

Lemma 5 Graph $G = (V, E)$ with n nodes has a clique of size at least k if and only if its complement G' has a vertex cover of size at most $n - k$.

The proof is left as an exercise for the problem set.

3 Vertex Cover on a Tree

While finding a minimum sized vertex cover is NP-hard in general, there are many special cases that are tractable. Consider the case, for example, where the graph $G = (V, E)$ is a rooted binary tree. In this case, we can find a minimum sized vertex cover in linear time using dynamic programming. Here we outline the solution for determining the *size* of the minimum cost vertex cover. It is relatively easy to turn this idea into an algorithm for finding the minimum vertex cover.

For each node v in the tree, we define two variables:

- $in(v)$ is the size of the minimum vertex cover for the sub-tree of G rooted at v , assuming that v is contained in the vertex cover.
- $out(v)$ is the size of the minimum vertex cover for the sub-tree of G rooted at v , assuming that v is not contained in the vertex cover.

Notice that if r is the root of the tree, the size of the minimum vertex cover is simply $\min[in(r), out(r)]$.

We can now traverse the tree starting from the leaves up to the root calculating for every node v both $in(v)$ and $out(v)$. For a leaf, we can trivially define $in(v) = 1$ and $out(v) = 0$. Now consider a node v in the tree with children w and z .

- To calculate $out(v)$: in this case, both children w and z *must* be included in the vertex cover. Thus $out(v) = in(w) + in(z)$.

- To calculate $in(v)$: in this case, we can either include or not include w or z in the tree, depending on which solution is better. In either case, we add 1 to the count, since we are including v . Thus, $in(v) = 1 + \min[in(w), out(w)] + \min[in(z), out(z)]$.

It is easy to see that this calculation can be performed in linear time, and that it results in the minimum sized vertex cover. (The proof would proceed by induction over the height of the tree.)

Note (in addition) that the same idea can be used for non-binary trees, and also for weighted variants of vertex cover (where each node has a cost for including it in the vertex cover).

4 Parameterized Complexity

Another special case is when you know, in advance, that the vertex cover is small. For example, if you are told that the vertex cover is of size $k = 2$, then there is an easy polynomial time algorithm: simply test every pair of nodes (and every individual node) and check whether that node is a valid vertex cover. Assuming that you can check the vertex cover of a graph $G = (V, E)$ with n nodes and m edges in time $O(m)$, then this algorithm runs in time $O(n^2m)$.

More generally, if you know that the vertex cover is of size at most k , you can find it in time $O(n^k m)$. Unfortunately, as n gets large, the term n^k becomes prohibitively large and the algorithm rapidly becomes unusable, even for small values of k . For example, if $k = 10$ and $n = 1000$ and you use a CPU that can perform 12 billion instructions per second (e.g., an Intel Core i7), then it would take more than one trillion years to find the vertex cover.

Instead, we would like an algorithm that runs in time $O(f(k)g(n, m))$, for some functions f and g where g is a polynomial in n and m . That is, the algorithm may be exponential in k , but the exponential function should not have any dependence on n and m . If k is small, this might yield a usable algorithm.

Here's the basic idea. For every edge $e = (u, v)$, we know that either u or v is in the vertex cover. So let's consider both cases. For every node $z \in V$, define G_{-z} to be the graph G where z and all edges adjacent to z have been deleted. If we can find a vertex cover of size $k - 1$ in either G_{-u} or G_{-v} , when we can construct a vertex cover for G . The key lemma here is as follows:

Lemma 6 *Let $G = (V, E)$ be a graph and let $e = (u, v)$ be an edge in E . The graph G has a vertex cover of size k if and only if either G_{-u} or G_{-v} has a vertex cover of size $k - 1$.*

(We leave the proof as an exercise.) This yields the following algorithm:

```

1 Algorithm: ParameterizedVertexCover( $G = (V, E), k$ )
2 Procedure:
   /* Base case of recursion:                                     */
3   if  $k = 0$  and  $E \neq \emptyset$  then return  $\perp$ 
4   if  $E = \emptyset$  then return  $\emptyset$ 
   /* Recurse on arbitrary edge  $e$ :                               */
5   Let  $e = (u, v)$  be any edge in  $G$ .
6    $S_u = \text{ParameterizedVertexCover}(G_{-u}, k - 1)$ 
7    $S_v = \text{ParameterizedVertexCover}(G_{-v}, k - 1)$ 
8   if  $S_u \neq \perp$  and  $|S_u| < |S_v|$  then
9     return  $S_u \cup \{u\}$ 
10  else if  $S_v \neq \perp$  and  $|S_v| < |S_u|$  then
11    return  $S_v \cup \{v\}$ 
12  else return  $\perp$ 

```

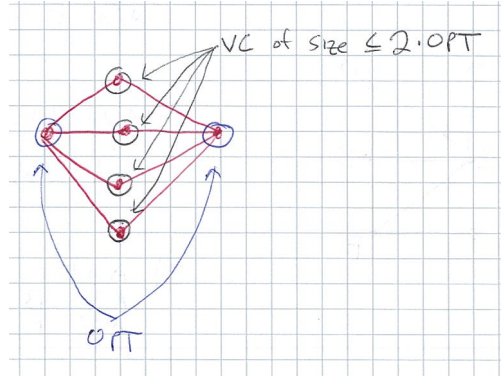


Figure 2: Example of a graph with an optimal vertex cover of size 2. The black circled nodes in the middle also form a vertex cover of size 4. This black vertex cover is not optimal, but is a 2-approximation.

The algorithm picks an arbitrary edge $e = (u, v)$, and recursively searches for a vertex cover of size $k - 1$ in G_{-u} and G_{-v} . When it has explored both options, it then return the smaller of the two vertex covers (adding either u or v as is appropriate).

Notice that this recursive algorithm has the following structure: at every step we remove one node from the graph and add it to the vertex cover, and we recurse twice. As a base case, if we have removed k nodes from the graph, and yet we have not found a vertex cover (i.e., some edges still remain in the graph), then we abort and return failure.

Assuming that the initial graph G has a vertex cover of size at most k , the running time of this algorithm is captured by the following recurrence:

$$T(k, m) \leq 2T(k - 1, m) + O(m)$$

(where I am assuming it takes approximately $O(m)$ time to delete a node and its adjacent edges from a graph, and I am ignoring the fact that the number of edges is reduced in the recursive calls). Solving this recurrence, we find that $T(n) = O(2^k m)$. For the setting above ($k = 10$, $n = 1000$), this improved algorithm would run in about 1ms!

5 Approximation Algorithms

Alas, sometimes a problem is still too hard to solve. The graph does not satisfy any of the (known) special cases. The parameterized complexity is *still* too large (i.e., the vertex cover is not sufficiently small). In this case, we cannot guarantee that you will find an optimal solution.

Instead, we relax our goals: we will not find an *optimal* solution, but we will find an *almost* optimal solution. This is the goal of an approximation algorithm. Consider, as an example, the problem of vertex cover. For a given graph G , define $OPT(G)$ to be a minimal-sized vertex cover⁴.

Definition 7 We say that algorithm A is an α -approximation algorithm for vertex cover if, for every graph G :

$$|A(G)| \leq \alpha |OPT(G)| .$$

That is, an algorithm A is a good approximation algorithm for vertex cover if it guarantees that it will find a vertex cover that is not too much larger than the best possible vertex cover.

⁴Notice that we often seem to implicitly assume there is only *one* possible optimal solution. This is not true! There may be many possible optimal vertex covers; here we fix one.

In the context of approximation algorithms, vertex cover is a particularly interesting problem to study. It is very easy to design a 2-approximation algorithm (and we will see two such algorithms today). However, the best known approximation for vertex cover is only marginally better, achieving an approximation factor of $2 - O(1/\sqrt{\log n}) = 2 - o(1)$. In fact, we know that if $P \neq NP$, then there is no polynomial time approximation algorithm that achieves an approximation factor of better than 1.3606. (This follows from the famous PCP theorem.) And if you assume that the Unique Games Conjecture⁵ is true, then you cannot achieve an approximation factor of $2 - \epsilon$ for any $\epsilon > 0$. In the latter case, the simple 2-approximation algorithms that we will see today are, effectively, the best you can do!

5.1 Randomized Approximation Algorithm

We first consider a very simple randomized approach: for every edge in the graph, simply flip a coin and randomly choose one of the two endpoints of the edge to add to the vertex cover. Repeat this process until every edge is covered.

```

1 Algorithm: RandomizedVertexCover( $G = (V, E)$ )
2 Procedure:
3    $C \leftarrow \emptyset$ 
4   /* Repeat until every edge is covered: */
5   while  $E \neq \emptyset$  do
6     Let  $e = (u, v)$  be any edge in  $G$ .
7      $b \leftarrow \text{Random}(0,1)$  // Returns  $\{0,1\}$  each with probability  $1/2$ .
8     if  $b = 0$  then  $z = u$ 
9     else if  $b = 1$  then  $z = v$ 
10     $C \leftarrow C \cup \{z\}$ 
11     $G \leftarrow G_{-z}$  // Remove  $z$  and all adjacent edges from  $G$ .
12  return  $C$ 

```

It is easy to see that, when the algorithm completes, the resulting set C is a vertex cover: each edge is removed from G only when it is covered by a node that has been added to C .

We now argue that the resulting vertex cover is, in expectation, at most twice the size of $OPT(G)$. The intuition here is that each time we add a node to C , we have a $1/2$ probability of being right: for every edge $e = (u, v)$, either $u \in OPT(G)$ or $v \in OPT(G)$ (or both of them are), and so with probability (at least) $1/2$ we choose the correct one. Thus we would expect C to be only about twice as large as OPT .

To make this intuition a little more formal, we define two sets: $A = C \cap OPT(G)$, and $B = C \setminus OPT(G)$. The set A contains all the “correct” elements that we have added to C , i.e., all the elements that are in both C and $OPT(G)$. The set B contains all the “incorrect” elements that we have added to C , i.e., all the elements that are in C but not in $OPT(G)$. For every edge e considered by the algorithm, define the indicator random variable:

$$X_e = \begin{cases} 0 & \text{if the vertex added to } C \text{ after considering } e \text{ is not in } OPT \\ 1 & \text{if the vertex added to } C \text{ after considering } e \text{ is in } OPT \end{cases}$$

Notice that $E[X_e] \geq 1/2$, since the probability of choosing the correct vertex is at least $1/2$. Let E' be the set of edges considered by the algorithm. Since $|A| = \sum_{E'} X_e$, we conclude that:

$$\begin{aligned} E[|A|] &= E\left[\sum_{E'} X_e\right] \\ &= \sum_{E'} E[X_e] \quad (\text{linearity of expectation}) \\ &\geq |E'|/2. \end{aligned}$$

⁵This is a conjecture that has widespread implications in complexity theory.

Similarly, we see that $|B| = \sum_{E'} (1 - X_e)$, and hence $E[|B|] \leq |E'|/2$. Putting these two together, we observe that:

$$E[|B|] \leq E[|A|] .$$

The final observation we need is that $|A| \leq |OPT(G)|$ (since A only contains nodes that are also in $OPT(G)$), and hence $E[|A|] \leq |OPT(G)|$. Putting together the preceding equations, we see:

$$E[|B|] \leq E[|A|] \leq |OPT(G)| .$$

Finally, we conclude that calculation:

$$\begin{aligned} E[|C|] &= E[|A|] + E[|B|] && (\text{linearity of expectation}) \\ &\leq |OPT(G)| + |OPT(G)| \\ &\leq 2|OPT(G)| \end{aligned}$$

This yields the following theorem:

Theorem 8 *The algorithm `RandomizedVertexCover` yields a 2-approximation for vertex cover (in expectation).*

5.2 Deterministic Approximation Algorithm

Next, we consider a deterministic algorithm for approximating vertex cover. There are three natural candidate algorithms (see below). Each of these algorithms greedily adds nodes to the cover, with progressing attempts at cleverness. The first algorithm simply adds arbitrary nodes until every edge is covered. The second algorithm considers the edges in arbitrary order, but adds *both* endpoints. The third algorithm is the cleverest, greedily adding the node that covers the most edges that remain uncovered. Stop for a minute and think about which algorithm you think is best.

```

/* This algorithm adds nodes greedily, one at a time, until everything is
   covered. The edges are considered in an arbitrary order, and for each
   edge, an arbitrary endpoint is added. */
1 Algorithm: ApproxVertexCover-1( $G = (V, E)$ )
2 Procedure:
3    $C \leftarrow \emptyset$ 
   /* Repeat until every edge is covered: */
4   while  $E \neq \emptyset$  do
5     Let  $e = (u, v)$  be any edge in  $G$ .
6      $C \leftarrow C \cup \{u\}$ 
7      $G \leftarrow G_{-u}$  // Remove  $u$  and all adjacent edges from  $G$ .
8   return  $C$ 

```

```

/* This algorithm adds nodes greedily, two at a time, until everything is
   covered. The edges are considered in an arbitrary order, and for each
   edge, both endpoints are added. */
1 Algorithm: ApproxVertexCover-2( $G = (V, E)$ )
2 Procedure:
3    $C \leftarrow \emptyset$ 
   /* Repeat until every edge is covered: */
4   while  $E \neq \emptyset$  do
5     Let  $e = (u, v)$  be any edge in  $G$ .
6      $C \leftarrow C \cup \{u, v\}$ 
7      $G \leftarrow G_{-\{u, v\}}$  // Remove  $u$  and  $v$  and all adjacent edges from  $G$ .
8   return  $C$ 

```

```

/* This algorithm adds nodes greedily, one at a time, until everything is
   covered. At each step, the algorithm chooses the next node that will
   cover the most uncovered edges. */
1 Algorithm: ApproxVertexCover-3( $G = (V, E)$ )
2 Procedure:
3    $C \leftarrow \emptyset$ 
   /* Repeat until every edge is covered: */
4   while  $E \neq \emptyset$  do
5     Let  $d(x)$  = number of uncovered edges adjacent to  $x$ .
6     Let  $u = \text{argmax}_{x \in V} d(x)$ 
7      $C \leftarrow C \cup \{u\}$ 
8      $G \leftarrow G_{-u}$  // Remove  $u$  and all adjacent edges from  $G$ .
9   return  $C$ 

```

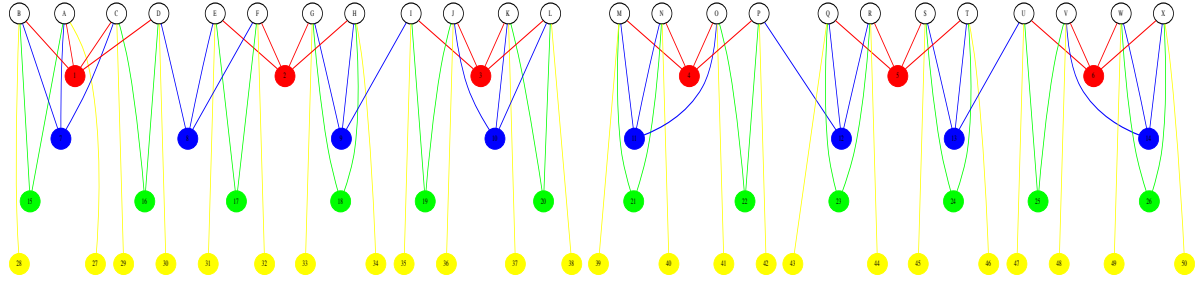


Figure 3: Counterexample for the greedy vertex cover algorithm. In this example, $n = 4$. The top row consists of $n! = 24$ white nodes, each of degree $n = 4$. The next row consists of $n!/n = 6$ red nodes, each of degree $n = 4$. The next row consists of $n!/(n - 1) = 8$ blue nodes of degree 3. The next row consists of $n!/(n - 2) = 12$ green nodes of degree 2. The last row consists of $n!/(n - 3) = 24$ yellow nodes of degree 1. Notice that there is a vertex cover of size $n!$, i.e., all the white nodes. However, the greedy algorithm might first add the red nodes (reducing the degree of every white node by one), then add the blue nodes (reducing the degree of every white node by one), then add the green nodes, and then the yellow nodes. The total number of nodes added to the vertex cover is $n! \sum_{i=1}^n (1/i) = n! \log n$, which is a factor of $\log n$ from optimal.

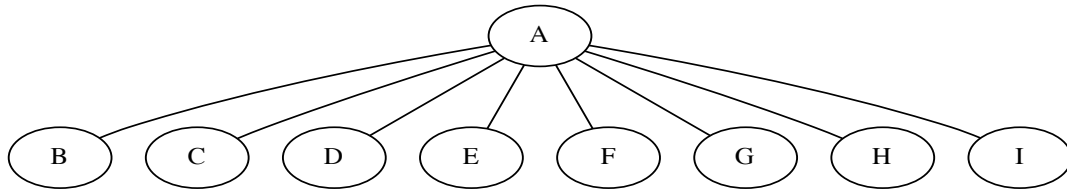


Figure 4: Counterexample for the simple vertex cover algorithm where an arbitrary node is added in each step. When each edge is considered, the leaf node is added, yielding a vertex cover of size $n - 1$. The optimal vertex cover contains only node A .

It turns out that neither Algorithm 1 nor Algorithm 3 achieve a good approximation ratio. (As an exercise, draw a graph for which each of these algorithms performs poorly.) We now proceed to analyze `ApproxVertexCover-2` and show that it is a 2-approximation algorithm. This may seem somewhat surprising: we are wastefully adding *both* endpoints of an edge e , when only one would suffice. On the other hand, thinking back to the analysis of the randomized algorithm, we know that each time at least *one* of the two nodes being added is a “correct” node, i.e., one that is added by $OPT(G)$. This guarantees that we get a vertex cover that is at most twice as big as optimal.

We now try to formalize this intuition. For pedagogic purposes, we are going to do this a little more carefully and less directly. In general, the trick to analyzing an approximation algorithm is to find some way to bound the size of OPT . If we can show that OPT cannot be too small, then it is easier to show that our algorithm is close to OPT . To this end, we are going to define another combinatorial concept:

Definition 9 Given a graph $G = (V, E)$, we say that a set of edge $M \subseteq E$ is a **matching** if no two edges in M share an endpoint, i.e., $\forall e_1, e_2 \in M : e_1 \cap e_2 = \emptyset$.

A matching is a set of edges that pairs up nodes: each node in the matching gets exactly one partner. Note that a matching can be of any size, consisting of one edge or up to $\lfloor n/2 \rfloor$ edges. If every node in the graph is adjacent to an edge in the matching, then it is a **perfect matching**.

The key observation here is that for every edge in a matching, one of the two endpoints *must* be in the vertex cover.

This yields the following lemma:

Lemma 10 *Let M be any matching of graph $G = (V, E)$. Then $|OPT(G)| \geq |M|$.*

Proof Observe that for every edge $e = (u, v)$ in the matching M , either u or v has to be in the vertex cover $OPT(G)$. Moreover, since M is a matching, neither u nor v cover any other edges in M . Thus every edge in M must have a unique node in $OPT(G)$, and hence $|OPT(G)| \geq |M|$. $\square$

This lemma gives us a nice way of showing that our vertex cover approximation algorithm is good: all we need to do is show that there exists some matching that is not too much smaller than the vertex cover it produces. Luckily, the algorithm constructs a matching as it executes:

Lemma 11 *Let E' be the set of edges considered by ApproxVertexCover-2 during its execution. Then E' is a matching.*

Proof After each edge $e = (u, v)$ in E' is considered, both the nodes u and v are removed from the graph G . Thus no edge considered later can contain either u or v . (Formally, the proof proceeds by induction, maintaining the invariant that for every edge e in E' , neither endpoint of e remains in G and hence the new edge being considered can be safely added to E' .) $\square$

Combining these two lemmas yields our final claim:

Theorem 12 *ApproxVertexCover-2 is a 2-approximation algorithm for vertex cover.*

Proof For a given graph $G = (V, E)$, let C be the set output by the algorithm and let E' be the edges considered. Notice that $|C| = 2|E'|$, since for every edge in E' we added two nodes to C . Since E' is a matching, we know that $|OPT(G)| \geq |E'|$, and hence we conclude:

$$\begin{aligned} |C| &= 2|E'| \\ &\leq 2|OPT(G)| \end{aligned}$$

$\square$

There are a couple interesting facts to take away from this proof. First, I want to emphasize the idea of trying to bound the optimal solution, and using this bound to show a good approximation. Second, notice the close relationship between finding a vertex cover and finding a matching. It turns out that vertex covers and matchings are related in a special way: they are, in a sense, *dual* problems (if considered in a fractional sense). We will come back to this idea of duality later in the semester.

LPs, Weighted Vertex Cover and Set Cover

*Lecturer: Divesh Aggarwal**August 29, 2024**Lecture Notes: Seth Gilbert***Abstract**

Today we introduce linear programming, a powerful tool for solving optimization problems. We discuss some terminology and notation, and we give some examples of how to use LPs to solve basic optimization problems. Then we talk about Integer Linear Programs (ILPs), and discuss how to relax ILPs to solve combinatorial optimization problems that require integral answer. We use weighted vertex cover as an example, giving a 2-approximation algorithm. We will also consider the problem of set cover. We begin with an example, and then discuss the classical greedy solution. Finally, we analyze the greedy algorithm and show that it is an $O(\log n)$ -approximation algorithm. In the analysis, notice how we begin by giving a lower bound on OPT, showing that OPT can only do so well. We then relate this to the performance of the Greedy Algorithm to get the approximation ratio.

1 Linear Programming

Linear programming is a general and very powerful technique for solving optimization problems where the objective (i.e., the thing being optimized) and the constraints are *linear*. Out in the real world, this is the standard approach for solving the combinatorial optimization problems that arise all the time. This technique is so common that an LP solver is now included in most common spreadsheets, e.g., Excel and OpenOffice. (Note that the term “programming” refers not to a computer program, more to a program in the sense of an “event program,” i.e., a plan for something.)

A typical linear program consists of three components:

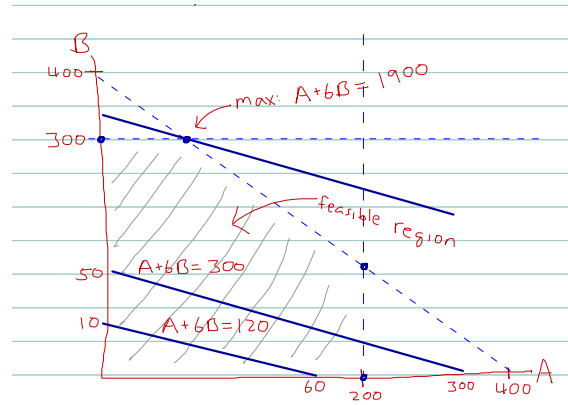
- A list of (real-valued) variables $x_1, x_2, \dots, x_n$. The goal of your optimization problem is to find good values for these variables.
- An objective function $f(x_1, x_2, \dots, x_n)$ that you are trying to maximize or minimize. The goal is to find the best values for the variables so as to optimize this function.
- A set of constraints that limits the feasible solution space. Each of these constraints is specified as an inequality.

In a linear programming problem, both the objective function and the constraints are *linear* functions of the variables.

Example 1. You are employed by Acme Corporation, a company that makes two products: widgets and bobbles. Widgets sell for SGD 1/widget, and bobbles sell for SGD 6/bobble. Bobbles clearly make more money for you, and so ideally you would like to sell as many bobbles as you can. However, after doing some market research, you have discovered that there is only demand for at most 300 bobbles and at most 200 widgets. It also turns out that your factory can only produce at most 400 units, whether they are widgets or bobbles. How many widgets and bobbles should you make, in order to maximize your total revenue?

We will represent this as a linear programming problem. This problem has two variables: A and B . The variable A represents the number of widgets and the variable B represents the number of bobbles. Your revenue is equal to $A + 6B$, i.e., 1/widget and 6/bobble. Your goal is to maximize this revenue. Your constraints are that $A \leq 200$ and $B \leq 300$: that represents the demand constraints from the market. Your other constraint is a supply constraint: $A + B \leq 400$, since you can only make 400 units total. Finally, we will include two more constraints that are obvious:

$$\begin{array}{ll} \max (A + 6B) & \text{where:} \\ A \leq 200 & \\ B \leq 300 & \\ A + B \leq 400 & \\ A \geq 0 & \\ B \geq 0 & \end{array}$$



$A \geq 0$ and $B \geq 0$. These were not included in the problem, but are important to exclude negative solutions. Put together, this yields the following linear program:

On the left, is the LP represented mathematically, specified in terms of an objective function and a set of constraints. On the right is a picture representing the LP geometrically, where the variable A is drawn as the x-axis and the variable B is drawn as the y-axis.

The dashed lines here represent the constraints: $A \leq 200$ (i.e., a vertical line), $B \leq 300$ (i.e., a horizontal line), and $A + B \leq 400$ (i.e., the diagonal line). Each constraint defines a **halfspace**, i.e., it divides the universe of possible solutions in half. In two-dimensions, each constraint is a line. In higher dimensions, a constraint is defined by a hyperplane.

Everything that is beneath the three lines represents the **feasible region**, which is defined as the values of A and B that satisfy all the constraints. In general, the feasible region is the intersection of the halfspaces defined by the hyperplanes, and from this we conclude that the feasible region is a convex polygon.

Definition 1 The **feasible region** for a linear program with variables $x_1, x_2, \dots, x_n$ is the set of points $(x_1, \dots, x_n)$ that satisfy all the constraints.

Notice that the feasible region for a linear program may be: (i) empty, (ii) a single point, or (iii) infinite.

For every point in the feasible region, we can calculate the value of the objective function: $A + 6B$. The goal is to find a point in the feasible region that maximizes this objective. For each value of c , we can draw the line for $A + 6B = c$. Our goal is to find the maximum value of c for which this line intersects the feasible region. You can see, above, we have drawn in this line for three values of c : $c = 120$, $c = 300$, and $c = 1900$. The last line, where $A + 6B = 1900$ intersects the feasible region at exactly one point: $(100, 300)$. This point, then, is the maximum value that can be achieved.

One obvious difficulty in solving LPs is that the feasible space may be infinite, and in fact, there may be an infinite number of optimal solutions. (Remember, we are considering real numbers here, so any line segment has a infinite number of points on it.) Let's think a little bit about whether we can reduce this space of possible solutions.

Imagine that I give you possible solution $(100, 300)$ and ask you to decide if it maximizes the objective function. How might you decide?

1. You draw the geometric picture and look at it. Recall that $f(100, 300) = 1900$. The line represented by the objective function $A + 6B = 1900$ cannot move up any farther. Thus, clearly 1900 is the maximum that can be achieved and hence $(100, 300)$ is a maximum.
2. Maybe we can prove algebraically that $(100, 300)$ is maximal. Recall, one of the constraints shows that $A + B \leq$

400. We also know that $B \leq 300$, and so $5B \leq 1500$. Putting these facts together, we conclude that:

$$\begin{array}{rcl} A + B & \leq & 400 \\ 5B & \leq & 1500 \\ \hline A + 6B & \leq & 1900 \end{array}$$

Since the objective function is equal to $A + 6B$, this shows that we cannot possibly find a solution better than 1900. Since we have found such a solution, we know that $(100, 300)$ is optimal.

This may seem like a special case, but in fact it turns out that you can always generate such a set of equations to prove that you have solved your LP optimally! This amazing fact is implied by the theory of **duality**, which we will come back to later.

3. One important fact about linear programs is that this maximum is always achieved at a vertex of the polygon defined by the constraints (if the feasible region is not empty). Notice that there may be other points (e.g., on an edge or a face) that also maximize the objective, but there is always a vertex that is at least as good. (We will not prove this fact today, but if you think about the geometric picture, it should make sense.) Therefore, one way to prove that your solution is optimal is to examine all the vertices of the polygon.

How many vertices can there be? In two dimensions, a vertex may occur wherever two (independent) constraints intersect. In general, if there are n dimensions (i.e., there are n variables), a vertex may occur wherever n (linearly independent) hyperplanes (i.e., constraints) intersect. Recall that if you have n linearly independent equations and n variables, there is a single solution—that solution defines a vertex. Of course, if the equations are not linearly independent, you may get many solutions—in that case, there is no vertex. (For example, if the constraints define hyperplanes that are parallel and do not intersect, there is no vertex.) Or, alternatively, if the intersection point is outside the feasible region, this too is not a vertex.

So in a system with m constraints and n variables, there are $\binom{m}{n} = O(m^n)$ vertices. (In fact, a more difficult analysis shows that the number of vertices is bounded by $O(m^{\lfloor n/2 \rfloor})$.)

We have thus discovered an exponential time $O(m^n)$ time algorithm for solving a linear program: enumerate each of the $O(m^n)$ vertices of the polytope, calculate the value of the objective function for each point, and take the maximum.

2 Solving Linear Programs

In this class, for the most part, we will ignore the problem of solving linear programs. There exist fast and efficient LP solvers, and we will rely on these as a black-box. However, here we give a few hints as to how these LP solvers work. (It is a fascinating and well-studied problem, and if you can build a faster LP solver, you can make a lot of money!)

2.1 Simplex Method

One of the earliest techniques for solving an LP—and still one of the fastest today—is the Simplex method. It was invented by Dantzig in 1947, and remains in common use today. There are many variants, but all take exponential time in the worst-case. However, in practice, for almost every LP that anyone has ever generated, it is remarkably fast.

The basic idea behind the Simplex method is remarkably simple. Recall that if an LP is feasible, its optimum is found at a vertex. (Again, remember that the optimum may be achieved at other points as well, but this does not matter to us.) Hence, the basic algorithm can be described as follows, where the function f represents the objective function.

1. Find any (feasible) vertex v .

2. Examine all the neighboring vertices of v : $v_1, v_2, \dots, v_k$.
3. Calculate $f(v)$ and $f(v_1), f(v_2), \dots, f(v_k)$. If $f(v)$ is the maximum (among its neighbors), then stop and return v .
4. Otherwise, choose one of the neighboring vertices v_j where $f(v_j) > f(v)$. Let $v = v_j$.
5. Go to step (2).

There are several things to notice about this algorithm. First, if the algorithm stops in step (3), then it really has found an optimum point. To prove this fact, we need to examine the geometry of the polytope; because the feasible region is convex and the objective function linear, we can conclude that if $f(v)$ is maximum among its neighbors, then it is maximum in the entire feasible region. This fact ensures that the Simplex Method always returns the right answer.

Second, observe that it will eventually terminate. At some point, it will have explored all the vertices of the polytope. Recall that at one of the vertices we will find an optimum, and hence we will eventually terminate. This also bounds the worst-case running time as $O(m^n)$ for an LP with n variables and m constraints.

Third, notice that Step (4) is somewhat underspecified: which neighboring vertex should we choose? Depending on how we choose the neighboring vertex, we can get very different performance. (In the case where $n = 2$, there are not many choices. But as the dimension grows, there become a much larger number of neighboring vertices to choose among.) The rule for choosing the next vertex is known as the *pivot rule*, and a large part of designing an efficient simplex implementation is choosing the pivot rule. Even so, all known pivot rules take worst-case exponential time.

Fourth, implementing this algorithm efficiently requires some care. How should one find neighboring vertices and determine where to go next (without re-calculating all the vertices in every step)? In fact, using some basic linear algebra and matrix manipulation, this can be implemented quite efficiently (e.g., using techniques like Gaussian elimination).

2.2 An example

As an example, consider running the Simplex Method on Example 1 above (i.e., the Acme corporation optimization problem). In this case, the Simplex method might begin with the feasible vertex $(0, 0)$.

First iteration. In the first iteration, it would calculate $f(0, 0) = 0$. It would also look at the two neighboring vertices, calculating that $f(0, 300) = 1800$ and $f(200, 0) = 200$. Having discovered that $(0, 0)$ is not optimal, it would choose one of the two neighbors. Assume, in this case, that the algorithm chooses next to visit neighbor $(200, 0)$.

Second iteration. In the second iteration, it would calculate $f(200, 0) = 200$. It would also look at the two neighboring vertices, calculating that $f(0, 0) = 0$ and $f(200, 200) = 1400$. In this case, there is only one neighboring vertex that is better, and it would move to $(200, 200)$.

Third iteration. In the third iteration, it would calculate $f(200, 200) = 1400$. It would also look at the two neighboring vertices, calculating that $f(200, 0) = 200$ and $f(100, 300) = 1900$. In this case, there is only one neighboring vertex that is better, and it would move to $(100, 300)$.

Fourth iteration. In the fourth iteration, it would calculate $f(100, 300) = 1900$. It would also look at the two neighboring vertices, calculating that $f(200, 200) = 1400$ and $f(0, 300) = 1800$. Discover that $(100, 300)$ is better than any of its neighbors, the algorithm would stop and return $(100, 300)$ as the optimal point.

Notice that along the way, the algorithm might calculate some points that were not vertices. For example, in the second iteration, it might find the point $(400, 0)$ —which is not feasible. Clearly, a critical part of any good implementation is quickly calculating the feasible neighboring vertices.

2.3 Other techniques for solving a linear program

While the Simplex Method runs in exponential time, we can solve linear programs in polynomial time! This was a huge breakthrough in the late 1970's and 1980's.

Let us begin with a simple idea. Imagine we had a technique for determining if there existed *any* feasible points for an LP. That is, we have a black box algorithm that returns “yes, *this LP is feasible*” or “no, *there is no feasible solution*”. Assume that we also have an upper bound M on the maximum value that the object function might achieve. Here is an algorithm for solving the LP:

- Start with value M .
- Add the constraint $f(x) = M$ to the LP.

For the remainder of today, we will ignore the problem of *how to solve* linear programs, and instead focus on how to use linear programming to solve combinatorial graph problems. Later this semester, I hope, we will say a little more about solving linear programs. For now, here's what you need to know about solving linear programs:

- If you can represent your linear program in terms of a polynomial number of variables and a polynomial number of constraints, then there exist polynomial time algorithms for solving them. (There are also other more general solutions, involving *separation oracles* that we will ignore for now.)
- The existing LP-solvers are very efficient for almost all the LPs that you would want to solve.
- You can find an LP solver in Excel or Open Office to experiment with.

3 LPs for Weighted Vertex Cover

We now look at how to express weighted vertex cover and weighted set cover as linear programs.

3.1 Vertex Cover as a Linear Program

First, let's look at vertex cover. Assume we are given a graph $G = (V, E)$ with weight function $w : V \rightarrow \mathbb{R}$, and we want to find a minimum cost vertex cover.

The first step is to define a set of variables. In this case, it is natural to define one variable for each node in the graph. Assuming there are n nodes in the graph, we define variables $x_1, x_2, \dots, x_n$. These variables should be interpreted as follows: $x_j = 1$ implies that node v_j is included in the vertex cover; $x_j = 0$ implies that node v_j is *not* included in the vertex cover.

Given these variables, we can now define the objective function. We want to minimize the sum of the weights of the nodes included in the vertex cover. That is, we want to minimize: $\sum_{j=1}^n w(v_j) \cdot x_j$.

Finally, we define the constraints. First, we need to ensure that every edge is covered: for every edge $e = (v_i, v_j)$, we need to ensure that either v_i or v_j is in the vertex cover. We need to represent this constraint as a linear function. Here is one way to do that: $x_i + x_j \geq 1$. This ensures that either x_i or x_j is included in the vertex cover.

We need one more constraint: for each variable x_j , we need to ensure that either $x_j = 1$ or $x_j = 0$. What would it mean if the linear programming solver returned that $x_j = 0.4$? This would not be useful. Unfortunately, there is no good way to represent this constraint as a linear function.

And it should not be surprising to you that we cannot represent the problem of vertex cover as a linear program! We know, already, that vertex cover is NP-hard. We also know that linear programs can be solved in polynomial time. Thus, if we found a linear programming formulation for vertex cover, that would imply that $P = NP$.

Instead, we formulate an *integer* linear program:

Definition 2 An *integer linear program* is a linear program in which all the variables are constrained to be integer values.

Thus, we can now represent weighted vertex cover as an integer linear program as follows:

$$\begin{aligned} \min \left(\sum_{j=1}^n w(v_j) \cdot x_j \right) \quad & \text{where:} \\ x_i + x_j &\geq 1 \quad \text{for all } (i, j) \in E \\ x_j &\geq 0 \quad \text{for all } j \in V \\ x_j &\leq 1 \quad \text{for all } j \in V \\ x_j &\in \mathbb{Z} \quad \text{for all } j \in V \end{aligned}$$

Notice that the objective function is a linear function of the variables x_j , where the weights are simply constants. (There is no term in the expression that looks like $x_i x_j$, i.e., multiplying variables together.) Similarly, each of the constraints is a linear function of the variables. The only constraint that cannot be expressed as a linear function is the last one, where we assert that each of the variables must be integral. (We will often abbreviate the last three lines by simply stating that $x_j \in \{0, 1\}$.)

3.2 Relaxation

Unfortunately, there is no polynomial time algorithm for solving integer linear programs (abbreviated ILP). We have already effectively shown that solving ILPs is NP-hard. If there were a polynomial time algorithm, we would have proved that $P = NP$. Instead, we will *relax* the integer linear program to a (regular) linear program. That is, we will consider the same optimization problem, dropping the constraint that the variables be integers. Here, for example, is the vertex cover relaxation:

$$\begin{aligned} \min \left(\sum_{j=1}^n w(v_j) \cdot x_j \right) \quad & \text{where:} \\ x_i + x_j &\geq 1 \quad \text{for all } (i, j) \in E \\ x_j &\geq 0 \quad \text{for all } j \in V \\ x_j &\leq 1 \quad \text{for all } j \in V \\ x_j &\in \mathbb{Z} \quad \text{for all } j \in V \end{aligned}$$

Notice that the solution to this LP is no longer guaranteed to be a solution to vertex cover! In fact, there is no obvious way to interpret the solution to this LP. What does it mean if we decide that $x_j = 0.4$?

Solving the relaxed ILP does tell us something: the solution to the linear program is *at least as good* as the optimal solution for the original ILP. In the case of weighted vertex cover, imagine we solve the relaxed ILP and the LP solver returns a set of variables $x_1, x_2, \dots, x_n$ such that $\left(\sum_{j=1}^n w(v_j) \cdot x_j \right) = c$, for some value c . Then we know that $OPT(G) \geq c$, where $OPT(G)$ is the optimal (integral) solution for vertex cover.

Why? Imagine there were a better solution $x'_1, x'_2, \dots, x'_n$ returned by $OPT(G)$. In that case, this solution would also be a feasible solution for the relaxed linear program: each of these variables x'_j is either 0 or 1, and hence would be a valid choice for the relaxed case where $0 \leq x_j \leq 1$. Hence the LP solver would have found this better solution.

The general rule is that when you expand the space being optimized over, your solution can only improve. By relaxing an ILP, we are expanding the range of possible solutions, and hence we can only find a better solution.

Lemma 3 *Let I be an integer linear program, and let $L = \text{relax}(I)$ be the relaxed integer linear program. Then $\text{OPT}(I) \geq \text{OPT}(L)$.*

3.3 Solving Weighted Vertex Cover

Returning to vertex cover, we have defined an integer linear program I for solving vertex cover. We have relaxed this ILP and generated a linear program L . The first thing we must argue is that there is a feasible solution the linear program:

Lemma 4 *The relaxed ILP for the vertex cover problem has a feasible solution.*

Proof Consider the solution where each $x_j = 1$. This solution satisfies all the constraints. $\square$

Assume we have now solved the LP L using an LP solver and discovered a solution $x_1, x_2, \dots, x_n$ to the linear program L . Our goal is to use this (non-integral) solution to find an (integral) solution to the weighted vertex cover problem.

Here is a simple observation: if (u, v) is an edge in the graph, then either $x_u \geq 1/2$ or $x_v \geq 1/2$. Why? Well, the linear program guarantees that $x_u + x_v \geq 1$. The linear program may well choose non-integral values for x_u and x_v , but it will always ensure that all the (linear) constraints are met.

Consider, then, the following procedure for **rounding** our solution to the linear program:

- For every node $u \in V$: if $x_u \geq 1/2$, then add u to the vertex cover C .

We claim that the resulting set C is a vertex cover:

Lemma 5 *The set C constructed by rounding the variables $x_1, \dots, x_n$ is a vertex cover.*

Proof Assume, for the sake of contradiction, that there is some edge (u, v) that is not covered by the set C . Since neither u nor v was added to the vertex cover, this implies that $x_u < 1/2$ and $x_v < 1/2$. In that case, $x_u + x_v < 1$, which violates the constraint for the LP, which is a contradiction. $\square$

It remains to show that the rounded solution is a good approximation, i.e., that we have not increased the cost too much.

Lemma 6 *Let C be the set constructed by rounding the variables $x_1, \dots, x_n$. Then $\text{cost}(C) \leq 2\text{cost}(\text{OPT})$.*

Proof The proof relies on two inequalities. First, we relate the cost of OPT to the cost of the linear program solution:

$$\text{cost}(\text{OPT}) \geq \sum_{j=1}^n w(v_j) \cdot x_j \quad (1)$$

This follows because the x_j were calculated as the optimal solution to a relaxation of the original vertex cover problem. Recall, by relaxing the ILP to a linear program, we can only improve the solution (i.e., in this case, we can only get a solution that is $\leq$ the integral solution).

Second, we related the cost of the LP solution to the cost of the rounded solution. To represent the rounded solution, let $y_j = 1$ if $x_j \geq 1/2$, and let $y_j = 0$ otherwise. Now, the cost of the final solution, i.e., $\text{cost}(C)$, is equal to

$\sum_{j=1}^n w(v_j) \cdot y_j$. Notice, however, that $y_j \leq 2x_j$, for all j . Therefore:

$$\begin{aligned} \sum_{j=1}^n w(v_j) \cdot y_j &\leq \sum_{j=1}^n w(v_j) \cdot (2x_j) \\ &\leq 2 \left(\sum_{j=1}^n w(v_j) \cdot x_j \right) \\ &\leq 2OPT(G) \end{aligned}$$

Thus, the rounded solution has cost at most $2OPT(G)$. □

3.4 General Approach

We have thus discovered a polynomial time 2-approximation algorithm for the weighted vertex cover:

- Define the vertex cover problem as an integer linear program.
- Relax the integer linear program to a standard LP.
- Solve the LP using an LP solver.
- Round the solution, adding vertex v to the cover if and only if $x_j \geq 1/2$.

We will see later in the class how we can translate this an algorithm that does not require an LP solver.

The general approach we have defined here for vertex cover can also apply to a wide variety of combinatorial optimization problems. The basic idea is to find an ILP formulation, relax it to an LP, solve the LP, and then round the solution until it is integral. Vertex cover yields a solution that is particularly easy to round. For other problems, however, rounding the solution may be more difficult.

One question, perhaps, is when this approach works and when this approach fails. Sometimes, the non-integral solution (returned by the LP solver) will be much better than the optimal integral solution. In such a case, it will be very difficult to round the solution to find a good approximation. This ratio is defined as the **integrality gap**. Assume I is some integer linear program:

$$\text{integrality gap} = \frac{OPT(\text{relax}(I))}{OPT(I)} \quad (2)$$

For a specific problem, if the integrality gap is at least c , then the best approximation algorithm you can achieve by rounding the solution the relaxed integer linear program is a c -approximation. This is a fundamental limit on this technique for developing approximation algorithms.

3.5 Linear Programming Summary

I want to quickly summarize the material on linear programming. In general, a linear program consists of:

- A set of variables: $x_1, x_2, \dots, x_n$.
- A linear objective to maximize (or minimize): $c_1x_1 + c_2x_2 + \dots + c_nx_n$. Thinking of c and x as vectors, this is often written more tersely as: $c^T x$ (where c^T represents the transpose of c , and multiplication here represents the dot product.)

- A set of linear constraints of the form: $a_{j,1}x_1 + a_{j,2}x_2 + \dots + a_{j,n}x_n \leq b_j$. (This version represents the j th constraint.) This is often abbreviated by the matrix equation: $Ax \leq b$.

Putting these pieces together, a linear program is often presented in the following form:

$$\begin{array}{ll} \max & cx \\ & Ax \leq b \\ & x \geq 0 \end{array} \quad \text{where}$$

For every linear program, we know that one of the following three cases holds:

- The LP is infeasible. There is no value of x that satisfies the constraints.
- The LP has an optimal solution.
- The LP is unbounded.

(Mathematically, this follows from the fact that if the LP is feasible and bounded, then it is a closed and bounded subset of $\mathbb{R}^n$ and hence has a maximum point.)

Standard terminology for linear programs:

- A point x is **feasible** if it satisfies all the constraints.
- An LP is bounded if there is some value V such that $c^T x \leq V$ for all points x .
- Given a point x and a constraint $a^T x \leq b$, we say that the constraint is **tight** if $a^T x = b$; we say that the constraint is **slack** if $a^T x < b$.
- A **halfspace** is the set of points x that satisfy one constraint. For example, the constraint $a^T x \leq b$ defines a halfspace containing all the points x for which this inequality is true. A halfspace is a convex set.
- The **polytope** of the LP is the set of points that satisfy all the constraints, i.e., the intersection of all the constraints. The polytope of an LP is convex, since it is the intersection of halfspaces (which are convex).
- A point x is a vertex for an n -dimensional LP if there are n linearly independent constraints for which it is tight.

4 Set Cover

The problem of **Set Cover** is a combinatorial optimization problem that frequently shows up in real-world scenarios, typically when you have collections of needs (e.g., tasks, responsibilities, or capabilities) and collections of resources (e.g., employees or machines) and you need to find a minimal set of resources to satisfy your needs. In fact, Set Cover generalizes Vertex Cover: in vertex cover, each vertex (i.e., set) covers the adjacent edges (i.e., elements); in set cover, each set can cover an arbitrary set of elements.

Unfortunately, Set Cover is NP-hard (i.e., NP-complete as a decision problem). As an exercise, prove that Set Cover is NP-hard by reducing another NP-complete problem to Set Cover.

In fact, we know that Set Cover is hard to approximate. There are no polynomial-time α -approximation algorithms for any constant, assuming $P \neq NP$. In fact, if there were a polynomial-time $(1 - \epsilon) \ln n$ -approximation algorithms, for any $\epsilon > 0$, then we could solve every problem in NP in $O(n^{\log \log n})$ time. Most computer scientists believe that this is unlikely!

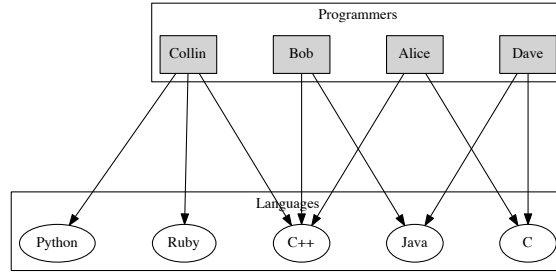


Figure 1: The problem of hiring software developers, represented as a bipartite graph. The goal is to choose a minimum sized set of programmers to ensure that every language is known by at least one of your programmers.

4.1 Problem Definition

We first define the problem, and then give some examples that show how set cover might appear in the real world.

Definition 7 Let $X = \{x_1, x_2, \dots, x_n\}$ be a set of n elements. Let $S_1, S_2, \dots, S_m$ be subsets of X , i.e., each $S_j \subseteq X$. Assume that every item in X appears in some set, i.e., $\bigcup_j S_j = X$. A **set cover** of X with S is a set $I \subseteq \{1, \dots, m\}$ such that $\bigcup_{j \in I} S_j = X$. A **minimum set cover** is a set cover I of minimum size.

That is, a minimum set cover is the smallest set of sets $\{S_{i_1}, S_{i_2}, \dots, S_{i_k}\}$ that covers X .

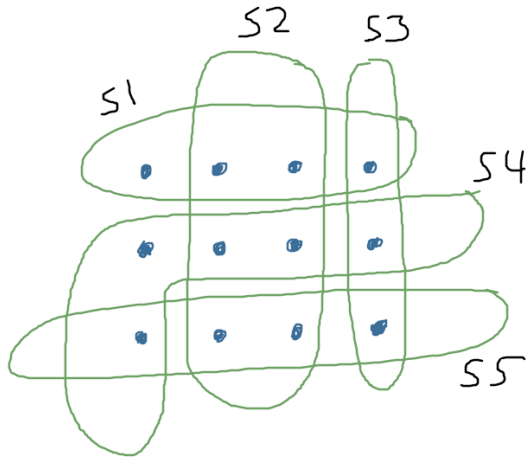
Example 1. Assume you have a set of software developers: Alice, Bob, Collin, and Dave. Each programmer knows at least one programming language. Alice knows C and C++. Bob knows C++ and Java. Collin knows C++, Ruby, and Python. Dave knows C and Java. Your job is to hire a team of programmers. You are given two requirements: (i) there has to be at least one person on the team who knows each language (i.e., C, C++, Java, Python, and Ruby), and (ii) your team should be as small as possible.

This is precisely a set cover problem. The base elements X are the 5 different programming languages. Each programmer represents a set. Your job is to find the minimum number of programmers (i.e., the minimum number of sets) such that every language is covered.

See Figure 1 for an example of this problem, depicted here as a bipartite graph. You will notice that any set cover problem can be represented as a bipartite graph, with the sets represented on one side and the base elements represented on the other side.

In this case, Alice, Bob, and Collin form a set cover of size 3. However, Collin and Dave form a set cover of size 2, which is optimal.

Example 2. Any vertex cover problem can be represented as a set cover problem. Assume you are given a graph $G = (V, E)$ and you want to find a vertex cover. In this case, you can define $X = E$, i.e., the elements to be covered are the edges. Each vertex represents a set. We define the set $S_u = \{(u, v) : (u, v) \in E\}$, i.e., S_u is the set of edges adjacent to u . The problem of vertex cover is now to find a set of sets $S_{i_1}, S_{i_2}, \dots, S_{i_k}$ such that every edge is covered.



Set	Step 1	Step 2	Step 3	Step 4	Step 5
S1	4	2	1	1	0
S2	6	0	0	0	0
S3	3	3	0	0	0
S4	5	3	2	0	0
S5	4	2	1	0	0

Figure 2: On the left is an example of set cover consisting of twelve elements and five sets. On the right is a depiction of what happens when you execute the GreedySetCover algorithm on this example. Each column represents the number of new elements covered by each set at the beginning of the step.

4.2 Greedy Algorithm

There is a simple greedy algorithm for solving set cover:

```

/* This algorithm adds sets greedily, one at a time, until everything is
   covered. At each step, the algorithm chooses the next set that will
   cover the most uncovered elements. */
1 Algorithm: GreedySetCover( $X, S_1, S_2, \dots, S_m$ )
2 Procedure:
3    $I \leftarrow \emptyset$ 
   /* Repeat until every element in  $X$  is covered: */
4   while  $X \neq \emptyset$  do
5     Let  $d(j) = |S_j \cap X|$  // This is the number of uncovered elements in  $S_j$ 
6     Let  $j = \underset{i \in \{1, \dots, m\}}{\text{argmax}} d(i)$ 
7      $I \leftarrow I \cup \{j\}$ 
8      $X \leftarrow X \setminus S_j$  // Remove elements in  $S_j$  from  $X$ .
9   return  $I$ 

```

The algorithm proceeds greedily, adding one set at a time to the set cover until every element in X is covered by at least one set. In the first step, we add the set that covers the most elements. At each ensuing step, the algorithm chooses the set that covers the most elements that remain uncovered.

Let's look at an example—see Figure 2. Here we have 12 elements (represented by the dots) and 5 sets: S_1, S_2, S_3, S_4, S_5 . In the first iteration, we notice that set S_2 covers the most elements, i.e., 6 elements, and hence it is added to the set cover. In the second iteration, set S_3 and S_4 both cover 3 new elements, and so we add set S_3 to the set cover. In the third iteration, set S_4 covers 2 new elements, and so we add it to the set cover. Finally, in the fourth step, set S_1 covers one new element and so we add it to the set cover. Thus, we end up with a set cover consisting of S_1, S_2, S_3, S_4 . Notice, though, that the optimal set cover consists of only three elements: S_1, S_4, S_5 .

4.3 Analysis

Our goal is to show that the greedy set cover algorithm is an $O(\log n)$ approximation of optimal. As is typical, in order to show that it is a good approximation of optimal, we need some way to bound the optimal solution. Throughout this section, we will let OPT refer to the optimal set cover.

Intuition. To get some intuition, let's consider Figure 2 and see what we can say about the optimal solution. Notice that there are 12 elements that need to be covered, and none of the sets cover more than 6 elements. Clearly, then, any solution to the set cover problem requires at least $12/6 = 2$ sets. Now consider the situation after the first iteration, i.e., after adding set S_3 to the set cover. At this point, there are 6 elements that remain to be covered, and none of the sets cover more than 3 elements. Any solution to the set cover problem requires at least $6/3 = 2$ sets.

In general, if at some point during the greedy algorithm, there are only k elements that remain uncovered and none of the sets covers more than t elements, then we can conclude that $OPT \geq k/t$. We will apply this intuition to show that the greedy algorithm is a good approximation of OPT .

Definitions. Assume we run the greedy set cover algorithm on elements X and sets $S_1, S_2, \dots, S_m$. When we run the algorithm, let us label the elements in the order that they are covered:

$$\underbrace{x_1, x_2, x_3, x_4}_{S_5}, \underbrace{x_5, x_6, x_7}_{S_3}, \underbrace{x_8, x_9}_{S_1}, \underbrace{x_{10}}_{S_8}$$

That is, x_1 is the first element covered, x_2 is the second element covered, etc. Under each element, I have indicated the first set that covered it. In this example, notice that the first set chosen (S_5) covers 4 new elements, the second set chosen (S_3) covers 3 new elements, etc. Each successive set covers at most the same number of elements as the previous one, because the algorithm is greedy: for example, if S_1 here had covered more new elements than S_3 , then it would have been selected before S_3 .

For each element x_j , let c_j be the number of elements covered at the same time. In the example above, this would yield:

$$c_1 = 4, c_2 = 4, c_3 = 4, c_4 = 4, c_5 = 3, c_6 = 3, c_7 = 3, c_8 = 2, c_9 = 2, c_{10} = 1$$

We define $\text{cost}(x_j) = 1/c_j$. In this way, the cost of covering all the new elements for some set is exactly 1. In this example, the cost of covering x_1, x_2, x_3, x_4 is 1, the cost of covering x_5, x_6, x_7 is 1, etc. In general, if I is the set cover constructed by the greedy algorithm, then:

$$|I| = \sum_{j=1}^n \text{cost}(x_j).$$

Notice that $c_1 \geq c_2 \geq c_3 \geq \dots$, because the algorithm is greedy.

Key step. Let's consider the situation after elements $x_1, x_2, \dots, x_{j-1}$ have been covered already, and the elements $x_j, x_{j+1}, \dots$ remain to be covered. Let OPT be the optimal solution for covering all n elements.

What is the best that OPT can do to cover the element $x_j, \dots, x_n$? How many sets does OPT need to cover these remaining elements?

Notice that there remain $n - j + 1$ uncovered elements. However, no set covers more than $c(j)$ of the remaining elements. In particular, all the sets already selected by the greedy algorithm cover *zero* of the remaining elements. Of the sets not yet chosen by the greedy algorithm, the one that covers the most remaining elements covers $c(j)$ of those elements: otherwise, the greedy algorithm would have chosen a different set.

Therefore, OPT needs at least $(n - j + 1)/c(j)$ sets to cover the remaining $(n - j + 1)$ elements. We thus conclude that:

$$OPT \geq \frac{n - j + 1}{c(j)} \geq (n - j + 1)cost(j)$$

Or to put it differently:

$$cost(j) \leq \frac{OPT}{(n - j + 1)}$$

We can now show that the greedy algorithm provides a good approximation:

$$\begin{aligned} |I| &= \sum_{j=1}^n cost(x_j) \\ &\leq \sum_{j=1}^n \frac{OPT}{(n - j + 1)} \\ &\leq OPT \sum_{i=1}^n \frac{1}{i} \\ &\leq OPT(\ln n + O(1)) \end{aligned}$$

(Notice that the third inequality is simply a change of variable where $i = (n - j + 1)$, and the fourth inequality is because the harmonic series $1 + 1/2 + 1/3 + 1/4 + \dots + 1/n$ can be bounded by $\ln n + O(1)$.)

We have therefore shown that the set cover constructed is at most $O(\log n)$ times optimal, i.e., the greedy algorithm is an $O(\log n)$ -approximation algorithm:

Theorem 8 *The algorithm GreedySetCover is a $O(\log n)$ -approximation algorithm for Set Cover.*

There are two main points to note about this proof. First, the key idea was to (repeatedly) bound OPT , so that we could relate the performance of the greedy algorithm to the performance of OPT . Second, the proof crucially depends on the fact that the algorithm is *greedy*. (Always try to understand how the structure of the algorithm, in this case the greedy nature of the algorithm, is used in the proof.) The fact that the algorithm is greedy leads directly to the bound on OPT by limiting the maximum number of new elements that any set could cover.

4.4 Questions

There are many further questions to explore related to set cover. (See the problem set for more.) Here are some natural questions.

1. What is the relationship between Vertex Cover and Set Cover? Show that Algorithm 3 (from Lecture 3) for Vertex Cover is a $O(\log n)$ -approximation algorithm.
2. We have looked here at the *unweighted* versions of set cover: every set has the same cost. Assume instead that each set S_j has a weight $w(S_j)$. The goal is now to minimize the total weight of all the selected sets. Give an approximation algorithm for this weighted version.
3. Give an integer linear program (ILP) for solving the set cover problem.
4. Show how to round the ILP in order to achieve a $O(\log n)$ -approximation of optimal. You may use a randomized algorithm and show that the approximation ratio is achieved with probability at least $1 - 1/n$. (Hint: if you have a variable y_j associated with set S_j , think about what happens when you add set S_j with probability y_j .)
5. Assume that each element x_j appears in at most f sets. Show how to round the ILP in order to achieve an f -approximation. (In this case, use a deterministic rounding scheme.)

The Traveling Salesman

*Lecturer: Divesh Aggarwal**September 5, 2024**Lecture Notes: Seth Gilbert*

Abstract

The goal of the Traveling Salesman Problem is to find a tour that connects all the nodes in a graph at a minimal cost. There are several variants, depending on whether repeated visits are allowed, and depending on whether the distances satisfy a metric. We discuss the relationship between these variants, and give a simple 2-approximation algorithm. We then develop a more involved 1.5-approximation algorithm that relates the travelling salesman problem to Eulerian tours.

1 The Travelling Salesman Problem

Today we consider the Traveling Salesman Problem, often abbreviated TSP. The “TSP” problem is perhaps one of the most famous (and most studied) problems in combinatorial optimization.

1.1 Problem Definition

Given a set of cities (i.e., points), the goal of the traveling salesman problem is to find a minimum cost circuit that visits all the points. More formally, the problem is stated as follows:

Definition 1 *Given a set V of n points and a distance function $d : V \times V \rightarrow \mathbb{R}$, find a cycle C of minimum cost that contains all the points in V . The cost of a cycle $C = (e_1, e_2, \dots, e_m)$ is defined to be $\sum_{e \in C} d(e)$, and we assume that the distance function is non-negative (i.e., $d(x, y) \geq 0$).*

See Figure 1 for an example of one instance of the traveling salesman problem.

1.2 Variants

As with the Travelling Salesman problem, there are several variants:

- *Metric vs. General:* In the *metric* version of the traveling salesman problem, the distance function d is a metric, i.e., it satisfies the triangle inequality. In the general version, d can assign any arbitrary weight to an edge.
- *Repeated visits vs. No-Repeats:* The goal of the TSP problem is to find a cycle that visits every node. Can the cycle contain repeated nodes (or does it have to be a simple cycle)? In the version with No-Repeats, the TSP cycle must visit each node *exactly* once. In the version with repeats, it is acceptable to visit each node more than once (if that results in a shorter route).

We will summarize these four variants as follows:

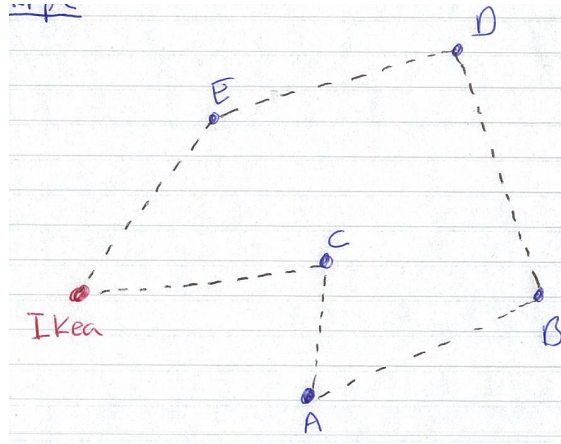


Figure 1: Example of the traveling salesman problem. Here, there are six locations. The problem is to find the shortest delivery circuit starting from Ikea and visiting each of the five houses, and then returning to Ikea.

	Repeats	No-Repeats
Metric	M-R TSP	M-NR TSP
General	G-R TSP	G-NR TSP

1.3 NP-Hard

All the variants of the traveling salesman problem are NP-hard. In fact, the problem is NP-hard even for planar graphs with maximum degree 3.

For the No-Repeats variants of the problem, this is easily seen by reduction from the Hamiltonian Cycle problem: a Hamiltonian cycle is a cycle that contains every node exactly once; the goal of the Hamiltonian Cycle problem is to determine whether a given graph has a Hamiltonian Cycle. Deciding whether a graph has a Hamiltonian Cycle is NP-hard (and this can be shown by reduction from 3SAT). Clearly, if we can solve TSP, then we can solve the Hamiltonian Cycle problem (by setting the distances properly). There are similar reductions for the other variants.

In fact, the general no-repeats version of TSP (G-NR TSP) is NP-hard even to approximate! If $P \neq NP$, there is no constant factor approximation algorithm. In fact, something even stronger is true. Let $r(n)$ be any polynomial time computable function. (For example, perhaps $r(n) = 2^{100}2^{n^2}$.) Let A be an $r(n)$ -approximation algorithm for G-NR TSP. If A is a polynomial time algorithm, then $P = NP$. Clearly, this is a problem that is very hard to approximate.

Even so, we are going to see how to (easily!) approximate the other three variants. In almost all common real-world cases (e.g., where the distance function satisfies the triangle inequality, or where repeats are acceptable), there are good approximation algorithms.

2 Equivalence

The first thing we are going to see is that the three other variants are equivalent. That is, $M - R - TSP$, $M - NR - TSP$, and $G - R - TSP$ are all equivalent: if we have a c -approximation algorithm for any one of these variants, then we also can construct a c -approximation algorithm for the other two.

We first focus on the issue of repeats: as long as the distance function is a metric, it does not matter whether or not we allow repeats. In many ways, this should be unsurprising (given our discussion of Steiner Trees last week): if we have repeated nodes on the cycle, we can always “short-cut” past them producing a cycle with no repeats at the same cost.

Lemma 2 *There exists a c -approximation algorithm for $M - NR - TSP$ if and only if there exists a c -approximation algorithm for $M - R - TSP$.*

Proof The proof has three claims. First, we show that both the version with repeats and no-repeats have the same optimal cost. We then show how to transform the solution from the NR variant into the R variant (which is trivial) and from the R variant into the NR variant (which involves skipping repeats).

Claim 1. Let (X, d) be an input for Metric TSP. Let $OPT(R)$ be the minimum cost TSP-cycle overall (i.e., with repeats), and let $OPT(NR)$ be the minimum cost TSP-cycle with no repeats. Then $OPT(R) = OPT(NR)$.

To show this, first we observe that any cycle with no repetitions is also a legal TSP-cycle overall (i.e., for the version that allows repeats). And so it follows immediately that $OPT(R) \leq OPT(NR)$. (Remember, when we minimize over a larger space of possible solutions, we always get a solution that is at least as good as the best solution of the smaller subspace.)

Next, let C be a cycle with repeats, for example:

$$C = A, B, C, D, C, E$$

We can now create cycle C' by skipping the repeated nodes. For example:

$$C' = A, B, C, D, E$$

By the triangle inequality, we know that $d(C') \leq d(C)$. For example, in constructing C' from C , we replace (D, C, E) with (D, E) ; we know that $d(D, C) + d(C, E) \leq d(D, E)$, by the triangle inequality. We can repeat this inductively for every skipped node.

If we assume that cycle $C = OPT(R)$, then we conclude that $OPT(NR) \leq d(C') \leq d(C) = OPT(R)$. Putting together the two inequalities, we have shown that $OPT(NR) \leq OPT(R)$ and $OPT(R) \leq OPT(NR)$, which yields our desired conclusion that $OPT(R) = OPT(NR)$.

Claim 2. If A is a c -approximation algorithm for $M - NR - TSP$, then A is a c -approximation algorithm for $M - R - TSP$.

Assume algorithm A outputs cycle C . By assumption, C has no repeats, and $d(C) \leq c \cdot OPT(NR) = OPT(R)$. Thus we see that C is also a valid solution for $M - R - TSP$ and provides a c -approximation.

Claim 3. If A is a c -approximation algorithm for $M - R - TSP$, then we can construct an algorithm A' which is a c -approximation for $M - NR - TSP$.

Specifically, construct algorithm A' as follows: Run algorithm A to get cycle C ; then construct C' from C by skipping any repeated node. There are two things we have to show. First, the cost of cycle C is a good

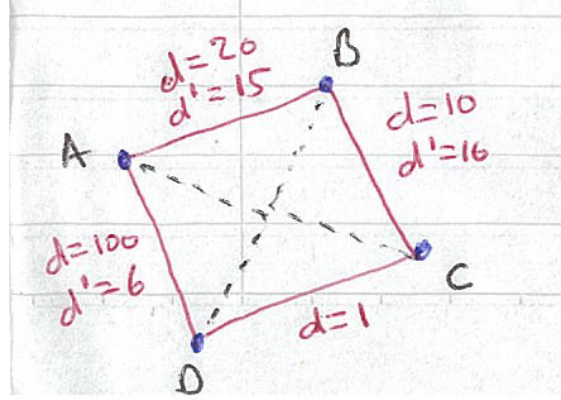


Figure 2: Example of constructing A' , reducing $G - R - TSP$ to $M - R - TSP$.

approximation of the optimal for $M - NR - OPT$:

$$\begin{aligned} d(C) &\leq c \cdot OPT(R) \\ &\leq c \cdot OPT(NR) \end{aligned}$$

Second, the cycle we have constructed in A' has cost bounded by the cost of C :

$$d(C') \leq d(C)$$

This follows from the triangle inequality. Putting these two facts together, we see that the algorithm A' is a c -approximation algorithm for $M - NR - TSP$. $\square$

We can show exactly the same type of equivalence for the general and metric versions, as long as we allow repeats. In this case, we cannot create “shortcuts” because the triangle inequality does not hold; however, we do not need to create shortcuts because repeats are allowed.

Lemma 3 *There exists a c -approximation algorithm for $M - R - TSP$ if and only if there exists a c -approximation algorithm for $G - R - TSP$.*

Proof In this case, the proof contains two claims: one transforming G into M (which is trivial), and the other M into G .

Claim 1. If A is a c -approximation algorithm for $G - R - TSP$, then A is a c -approximation algorithm for $M - R - TSP$.

This claim is immediately and obviously true. The two problems are identical, except for the restriction that the input to $M - R - TSP$ must be a metric. However, if a given input *is* a metric, then it clearly satisfies the requirements of $G - R - TSP$ and we can execute algorithm A on that instance. Moreover, the optimal solution will be the same for $G - R - TSP$ and $M - R - TSP$ (since they are optimizing over the exact same set of possible cycle).

Claim 2. If A is a c -approximation algorithm for $M - R - TSP$, then we can construct algorithm A' is a c -approximation algorithm for $G - R - TSP$.

Let (X, d_g) be the input to algorithm A' , i.e., an input for $G - R - TSP$. We construct algorithm A' as follows.

First, we need to construct a distance metric d_m . For every pair of nodes (u, v) , define $d_m(u, v)$ to be the shortest path from u to v according to the distances d_g . (Recall that d_g is not a metric, i.e., does not necessarily satisfy the triangle inequality.) We can find d_m by construct the complete graph on X , assign the weight of edge (u, v) to be $d_g(u, v)$, and executing an all-pairs-shortest-path algorithm. You will notice that d_m is a distance metric, i.e., it satisfies the triangle inequality. Recall that d_m is called the metric completion of the graph.

Second, execute algorithm A on (X, d_m) , producing cycle C . We then need to construct a new cycle C' out of C . For each edge (u, v) , we add the shortest path from u to v according to d_g to our output cycle C' . Notice this produces a cycle C' that may contain may repeated nodes.

We now argue that the result is a good approximation of the $OPT(X, d_g)$. First, observe that $d(C') = d(C)$. Each edge in C got replaced by a path in C' with the exact same cost, and hence we end up with the same cost cycle.

Second, we know by assumption that $d(C) \leq c \cdot OPT(X, d_m)$, by the assumption that A is a c -approximation algorithm.

Finally, we argue that $OPT(X, d_m) \leq OPT(X, d_g)$. In particular, if R is the optimal cycle for (X, d_g) , then R is also a cycle in (X, d_m) . Moreover, for every pair (u, v) , we know that $d_m(u, v) \leq d_g(u, v)$ (because d_m is defined as the shortest path). Thus $d_m(R) \leq d_g(R)$. Since the optimal cycle with metric d_m has to be at least as good as $d_m(R)$, we conclude that $OPT(X, d_m) \leq d_m(R) \leq d_g(R) = OPT(X, d_g)$.

Putting the pieces together, we conclude that $d(C') = d(C) \leq c \cdot OPT(X, d_m) \leq c \cdot OPT(X, d_g)$, and hence algorithm A' is a c -approximation algorithm for $G - R - TSP$. $\square$

3 2-Approximation Algorithm

We now present a 2-approximation algorithm for $G - R - TSP$. We already know that this yields a 2-approximation algorithm for the other two variants. Assume that the input is (X, d) where X is a set of points and d is a distance function (but not necessarily a metric). Consider the following algorithm:

1. Construct the complete graph $G = (X, E)$ with weights w where E contains every pair $(u, v) \in X \times X$ and $w(u, v) = d(u, v)$.
2. Let T be the minimum spanning tree of G .
3. Let C be the cycle constructed by performing a depth-first search of T .

To analyze this algorithm, as usual, we start with the optimal solution and work backwards. Let C^* be the optimal (minimum cost) TSP cycle for input (X, d) and let E^* be the edges in C^* . Notice that the graph $G^* = (X, E^*)$ is connected, because C^* is a cycle that includes every point. Let T^* be the minimum spanning tree of $G = (X, E^*)$. At this point, we know that:

$$d(T^*) \leq d(C^*) = OPT$$

Since the tree T is a minimum spanning tree of $G = (X, E)$ and $E^* \subseteq E$, we know that:

$$d(T) \leq d(T^*)$$

Finally, since C is constructed by a depth-first-search traversal of T , we know that C includes each edge in T exactly twice, and so:

$$d(C) = 2d(T)$$

Putting these inequalities together, we conclude that:

$$d(C) = 2d(T) \leq 2d(T^*) \leq 2d(C^*) \leq 2OPT$$

That is, the cost of C is at most twice the cost of OPT , and hence the algorithm is a 2-approximation algorithm.

4 Eulerian Cycles

We now want to develop a better approximation algorithm, one that will have a better approximation ratio than 2. In order to do that, we are going to need a few additional tools. Let's begin with a famous problem, known as the *Bridges of Königsberg*.

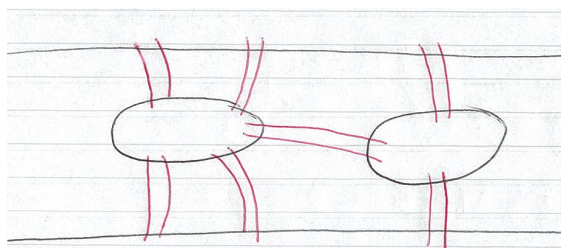


Figure 3: Illustration of the Bridges of Königsberg.

In Königsberg, there is a river with two islands. Historically, these two islands were connected by seven bridges. The bridges were beautiful and famous, and so the question that everyone wanted to answer was whether it was possible to cross each bridge exactly once—and end up back where you started. With a little bit of thought, it becomes obvious that you cannot. But how would you prove it? Euler solved this problem, and his solution is often seen as marking the beginning of graph theory as a mathematical field. Notably, he realized that the problem could be represented as a graph: Once the problem was represented abstractly as

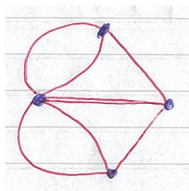


Figure 4: Illustration of the Bridges of Königsberg as a graph.

a graph, we can apply more powerful techniques to solve the problem.

For the purpose of this section, we will be talking about *multigraphs* rather than simple graphs. A *multigraph* is a graph $G = (V, E)$ where E is a multiset, rather than a set, of edges. This means that each edge can appear in the graph more than once. For example, a given edge (u, v) might appear in graph G four times.

Definition 4 A *Eulerian Cycle* in a multigraph G is a cycle that crosses each edge exactly once.

Notice that, unlike a TSP-cycle, an Eulerian Cycle focuses on edges, rather than nodes. The goal is to cross each edge exactly once—though that may mean visiting a given node many times. Clearly the problem of the Bridges of Königsberg is simply asking whether there is an Eulerian Cycle in the associated graph.

The key claim is to characterize when a graph has an Eulerian Cycle. Amazingly, there is a very simple characterization:

Lemma 5 *A multigraph $G = (V, E)$ has a Eulerian Cycle if and only if:*

- *It is connected (except for nodes of degree 0).*
- *Every node has even degree.*

Proof Notice that one direction of this claim is easy. Assume graph G has an Eulerian Cycle. In that case, since the cycle crosses every edge, clearly the cycle visits every node (except those nodes with degree zero) and hence graph is connected—except for nodes with degree zero. In addition, the cycle enters and exits each node the same number of times. For example, if the cycle enters a node v some k times, then it also exits node v the same k times. From this we conclude that every node has even degree. (In our example, node v has degree $2k$, which is even.)

The surprising result is the opposite direction. Assume graph G is connected (except for nodes of degree 0) and that every node has even degree. We need to construct a Eulerian Cycle. We will proceed by induction on the number of edges, proving a stronger invariant as we go: If every node in G has even degree, then each connected component with > 1 nodes has an Eulerian Cycle.

We will begin (in the base case) with an empty graph containing only the nodes, and add one edge at a time back to the graph. At every step, we will construct an Eulerian Cycle for each connected component.

Base case: In this case, $m = 0$, i.e., there are no edges, and hence trivially the claim holds.

Inductive step: Assume we have m edges. Let $V_1, V_2, \dots, V_k$ be the connected components of G that contain > 1 nodes. We consider two cases:

Case 1: $k \geq 2$: There are at least two connected components. Each connected component has $< m$ edges. Hence, by induction, each component has an Eulerian Cycle.

Case 2: $k = 1$: V_1 is the only connected component with > 1 nodes. Recall that all the nodes in V_1 have even degree, and every node not in V_1 has degree zero.

First, we claim that there must be a cycle in V_1 . Let $n_1 = |V_1|$. If there are no cycles in V_1 , then V_1 is a tree and contains exactly $n_1 - 1$ edges. However, the degree of every node in V_1 is at least 2 (since the degree is even and > 0). Thus there are at least $2n_1$ endpoints of edges in V_1 , and hence at least n_1 edges in V_1 . (Notice in general that if $d(v)$ is the degree of node v , then a graph has exactly $\sum_{u \in V} d(u)/2$ edges.) This contradicts the claim that V_1 is a tree.

Let C be any cycle in V_1 . Let G' be the graph G where the edges in C are removed. Let $V'_1, V'_2, \dots, V'_{k'}$ be the new connected components of G' . (By removing the cycle C , we may have split V_1 into several distinct connected components; or V_1 may remain connected.) Each of these components now has $< m$ edges. Hence, by induction, each of these components has an Eulerian Cycle. Let $C_1, C_2, \dots, C_{k'}$ be the Eulerian Cycles for connected components $V'_1, V'_2, \dots, V'_{k'}$.

It remains to stitch these Eulerian Cycles back together. Notice that these connected components are all connected by the initial cycle C . That is, if we add back the cycle C , we will have a single connected component. Here's how we do that:

Let $C = (v_1, v_2, \dots, v_\ell)$. Begin at node v_1 . We know that node v_1 is part of one of the Eulerian Cycles V_j . Follow the Eulerian Cycle V_j until it returns to node v_1 . Then proceed to node v_2 . If v_2 is part of a new

Eulerian Cycle (i.e., not V_j), then follow the new Eulerian Cycle until it returns to v_2 . Otherwise move on to v_3 . And so on. At each step, we move to the next node v_i in the cycle C . If that node v_i is part of a new Eulerian Cycle V_i that has not yet been traversed, then we traverse the cycle until it returns to v_i . Then we continue to v_{i+1} . This continues until we return to node v_1 .

Notice that the new cycle we have constructed traverses every edge in the graph G , as it traverses all the edges in the connected components $V'_1, \dots, V'_{k'}$ as well as all the edges in C . Hence we have constructed an Eulerian Cycle for G . $\square$

The other interesting fact is that this Eulerian Cycle can be easily constructed in polynomial time, exactly by following the steps of the algorithm:

1. Find a cycle in G (e.g., via DFS). If there is none, then skip the next step.
2. Remove the cycle.
3. Find the connected components in the residual graph.
4. Recurse: find an Eulerian Cycle in each connected component.
5. If a cycle was found in Step 1, paste the cycles back together.

This can readily be implemented in $O(m^2)$ time. Can you optimize it further?

5 A Better TSP Approximation

We now consider the Metric Traveling Salesman Problem, with repeats. (As before, if we can approximate this variant, then we can also approximate the other two tractable variants.) Assume (X, d) is the input to our problem.

5.1 Basic Idea

Imagine that we built a multigraph $G = (X, E)$ that contained an Eulerian Cycle. Then that cycle would give us a feasible solution to the TSP problem. Conversely, if had a solution to the TSP problem, we could build a multigraph consisting of only the edges in the TSP-cycle, which would result in a graph with a Eulerian Cycle. Hence solving TSP is equivalent to finding a minimum cost set of edges E such that $G = (X, E)$ has an Eulerian Cycle.

For example, consider the following proposed algorithm:

1. Find the MST T of the complete graph on X with weights defined by d .
2. Add every edge in T *twice* to the set E . This ensures that each node in X has even degree.
3. Since every node in multigraph $G = (V, E)$ has even degree, we can find a Eulerian Cycle C for G .
4. Return C .

This is a correct algorithm for solving the TSP problem. Unfortunately, it will not yield any improvement over the earlier 2-approximation algorithm because we are still adding each edge in the MST twice, leading to a cost that is at most twice optimal.

Notice that it is quite inefficient to add each edge twice! Our only goal is to ensure that each node has even degree—if a node already has even degree, why are we doubling all of its outgoing edges, and thus increasing our cost? Consider the example in Figure 5. Thus the question we must answer is: how do we add the

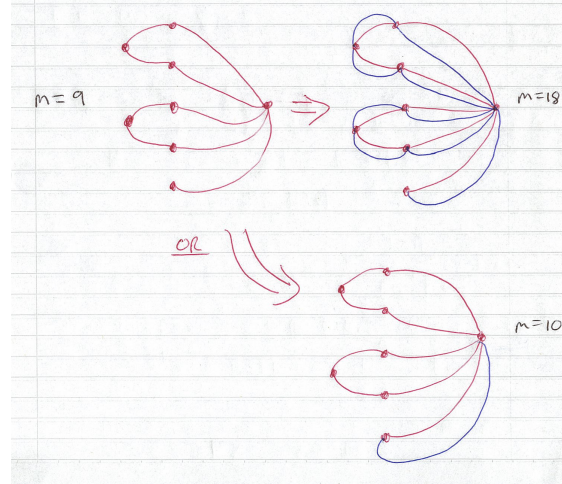


Figure 5: Example of finding a TSP by creating nodes with even degree. Initially, we have a graph with nine edges. If we double every edges, we have a graph with 18 edges. However, if we want to ensure that each node has even degree, it suffices to add just one edge to the graph, yielding a graph with only 10 edges.

minimal number of edges to a multigraph to ensure that each node has even degree? Thus, in summary, the resulting algorithm should look like:

1. Find the MST T of the complete graph on X with weights defined by d .
2. Add every edge in T to the set E .
3. Add the minimum cost set of edges E' to E such that in $E \cup E'$, every node has even degree.
4. Since every node in multigraph $G = (V, E \cup E')$ has even degree, we can find a Eulerian Cycle C for G .
5. Return C .

We already know that $\text{cost}(T) \leq \text{cost}(OPT)$, since the optimal TSP-cycle is also a spanning tree (once you have removed a single edge). The key is to prove that $\text{cost}(E') \leq OPT/2$.

5.2 Matchings

The last tool we need is a perfect matching:

Definition 6 We say that (V, M) is a **matching** if no two edges in M share an endpoint, i.e., all nodes in V have degree ≤ 1 . We say that (V, M) is a **perfect matching** if every node in V has exactly degree 1, i.e., every node is matched.

Notice that a perfect matching only exists if $|V|$ is even: if there are an odd number of nodes, it is clearly impossible to match them all.

If each edge has a weight, then we may want to find the minimum cost perfect matching. One of the most beautiful results in combinatorial optimization yields exactly that:

Theorem 7 *There is a polynomial time algorithm for finding the minimum cost perfect matching.*

This finds the optimal solution, and it does it efficiently! Unfortunately, this algorithm is beyond the scope of today's class. (See Edmond's algorithm. The solution involves his famous *Blossom Algorithm* for finding matchings, along with the primal-dual method and linear programming.) For now, we are just going to assume that we can find a minimum cost perfect matching.

5.3 Evens and Odds

We want to use the idea of perfect matchings to pair up the nodes in the multigraph that have odd degree. If we can assign each of them a partner, then they will have even degree. But there is one potential problem: what if there are an odd number of nodes that need a partner? Luckily, that is impossible due to a nice counting argument:

Lemma 8 *In every graph $G = (V, E)$, there are an even number of nodes with odd degree.*

Proof Let O be the set of nodes with odd degree, and $V - O$ the set of nodes with even degree. Let $\deg(u)$ be the degree of u .

Recall that we can count the number of edges in the graph by summing up the total degree and dividing by 2, i.e.:

$$\sum_{u \in V} \deg(u) = 2|E|$$

That is, the sum of the degrees of all the nodes is even.

Now, consider just the nodes in $V - O$: since each node has even degree, the sum of their degrees must also be even:

$$\sum_{u \in V-O} \deg(u) = 2k$$

Summing everything, we conclude that:

$$\begin{aligned} \sum_{u \in O} \deg(u) + \sum_{u \in V-O} \deg(u) &= \sum_{u \in V} \deg(u) \\ \sum_{u \in O} \deg(u) + 2k &= 2|E| \end{aligned}$$

Since the right-hand-side of the expression is even, the left-hand side of the expression must also be even. That is:

$$\sum_{u \in O} \deg(u) = 2\ell$$

But each node in O has odd degree. Hence the only way that the sum of their degrees can be even is if there are an even number of odd nodes, i.e., $|O|$ is even. $\square$

5.4 Christofides Algorithm

Putting the pieces together, we now have the following algorithm:

1. Find the MST T of the complete graph on X with weights defined by d .
2. Add every edge in T to the set E .
3. Let O be the nodes in T with odd degree. Notice that $|O|$ is even.
4. Let M be the minimum cost perfect matching for O .
5. Construct the multigraph $G = (X, E \cup M)$.
6. Since every node in multigraph $G = (V, E \cup M)$ has even degree, we can find a Eulerian Cycle E for G .
7. Return E .

To analyze this, we notice that the cost consists of the following:

$$\text{cost}(E) = \sum_{e \in E} d(e) + \sum_{e \in M} d(e)$$

The first part of the expression comes from the MST, and hence we already know that:

$$\sum_{e \in E} d(e) \leq \text{OPT}$$

Since OPT forms a cycle, we can remove any edge from the cycle to get a spanning tree; the MST T must have cost no greater than this cycle.

We now need to argue that $2 \sum_{e \in M} d(e) \leq \text{OPT}$. Let C be the cycle for OPT . Let C' be the same cycle as C where we skip all the nodes in $X - O$, and we skip repeats. That is, C' is a cycle on the odd nodes with no repeats. Notice that $\text{cost}(C') \leq \text{cost}(C)$, since we have only skipped nodes (and the triangle inequality holds).

Also, notice that cycle C' has an even number of nodes (because there are an even number of odd nodes) and an even number of edges. Assume that $C' = (v_1, v_2, \dots, v_{2k})$.

We can now construct two different perfect matchings M_1 and M_2 . We define them as follows:

$$\begin{aligned} M_1 &= (v_1, v_2), (v_3, v_4), (v_5, v_6), \dots, (v_{2k-1}, v_{2k}) \\ M_2 &= (v_2, v_3), (v_4, v_5), (v_6, v_7), \dots, (v_{2k}, v_1) \end{aligned}$$

Notice that each of these perfect matchings has $k = |O|/2$ edges, and both are valid perfect matchings for the set O . Since M is the minimum cost perfect matching, we conclude that:

$$\begin{aligned} \text{cost}(M) &\leq \text{cost}(M_1) \\ \text{cost}(M) &\leq \text{cost}(M_2) \end{aligned}$$

Notice, though, that $\text{cost}(M_1) + \text{cost}(M_2) = \text{cost}(C') \leq \text{cost}(C)$. So, if we sum the matchings, we get:

$$2\text{cost}(M) \leq \text{cost}(M_1) + \text{cost}(M_2) \leq \text{cost}(C) = \text{OPT}$$

Thus we have shown that $\text{cost}(M) \leq \text{OPT}/2$.

Putting the pieces together, we see that the cost of the cycle output by the algorithm is:

$$\begin{aligned} \text{cost}(E) = \sum_{e \in E} d(e) + \sum_{e \in M} d(e) &\leq \text{cost}(T) + \text{cost}(M) \\ &\leq OPT + OPT/2 \\ &\leq 1.5 \cdot OPT \end{aligned}$$

Thus, we have discovered a 1.5-approximation algorithm for the traveling salesman problem.

Conjunctive Normal Form Satisfaction (CNF-SAT) Problem

Lecturer: Divesh Aggarwal

September 12, 2024

Scribe: Eom Ikhoon, Chen Lyuting, Ng Yu Soon, Nguyen Doan Phuong Anh

Abstract

Today we introduce approximation algorithms for solving *Conjunctive Normal Form Satisfaction* (CNF-SAT) problem and its variants. We will first consider the special case of the problem where the weighted CNF-SAT problem has exactly k number of literals for every clause (constraints) and provide simple randomized and deterministic $(1 - \frac{1}{2^k})$ -approximation algorithm. Subsequently, we will work on devising randomized and deterministic approximation algorithms for general weighted CNF-SAT problems by relaxing constraints sequentially from several special cases. Finally, the integer programming method will be applied to approximate the solutions to the weighted CNF-SAT problem.

1 CNF-SAT Problem Statement

In this lecture, we will study CNF-SAT related problems and how strategies we have learned so far in the lecture, such as integer programming and randomization ideas, can be applied to devise clever approximation algorithms for CNF-SAT problems.

Definition 1. (*Constraint Satisfaction Problem*) A **constraint satisfaction problem** is defined to be a set of triple (X, D, C) where $X := (x_1, x_2, \dots, x_n)$ represents a set of literals, $D := (D_1, D_2, \dots, D_n)$ represents a set of domain where each variable belongs to, and $C := (C_1, C_2, \dots, C_m)$ represents a set of constraints.

One simple example of a constraint satisfaction problem is a fill-in-the-box, or sudoku-like problem, described below:

Example 2. In the board described in figure 1, we might set each blank space as respective variables $x_1, x_2, \dots, x_{10}$, and impose some domain and constraints on each variable. For instance, we could set the domain of values that variable x_1 takes as $D_1 := \{1, 2, 3, 4\}$, and impose constraints such as $C_1 : x_1 \neq x_4$, $C_2 : x_1 \neq x_3$, and $C_3 : x_1 \neq 4$. The problem is to find values for each variable x_i (we call this to be a solution to the constraint satisfaction problem) such that the variables take value in their respective domains, and the solution satisfies all constraints imposed.

1.1 CNF-SAT Satisfaction Problem Statement

Now, we want to introduce the definition of the CNF-SAT problem as a case of constraint satisfaction problem. Instead, variables and constraints are over boolean values rather than integers, unlike the above sample problem. Before we state the problem, we need to define some key terms for the sake of clarity.

Definition 3 (Literal). For a given variable x where x takes boolean values (i.e. x either takes value of True or False), **literal** l associated to x is a boolean valued variable x or its negation $(\neg x)$

Definition 4 (Clause). **Clause** is defined to be a disjunction of literals.

x_1	4	x_5	x_6
x_2	x_3	x_4	4
x_7	x_8	2	3
2	3	x_9	x_{10}

Figure 1: Sample fill-in-the-box problem

A simple example of a clause is $C := (x \vee (\neg y) \vee z)$ where x, y, z are boolean-valued variables.

Now, we are ready to state the CNF-SAT problem:

Definition 5 (CNF-SAT Problem). *The **CNF-SAT** problem is a constraint satisfaction problem where the set of variables $X = (x_1, x_2, \dots, x_n)$ take value over the domain of boolean values $D_1 = D_2 = \dots = D_n = \{\text{True}, \text{False}\}$. Each constrain $C_i \in C$, where C is a set of constraints, is given by a conjunction of literals $C_i = l_{i_1} \vee l_{i_2} \vee \dots \vee l_{i_k}$ where $i_1, i_2, \dots, i_k$ are natural numbers smaller or equal to $n = |X|$.*

Given the set of variables, domains, and constraints, the goal of the CNF-SAT problem is to find a boolean value assignment for all $x_i \in X$ such that the assignment satisfies all constraints in the set C .

Notice that the problem above is not simple to come up with an algorithm that runs in polynomial time. Indeed, the CNF-SAT satisfaction problem is an NP-complete (in particular, NP-hard) problem!

1.2 Weighted CNF-SAT Satisfiability Problem Definition

To generalize the above NP-hard problem, we want to tackle a variant of the problem where each clause (or constraint) has an associated weight value and find a boolean value assignment that maximizes the sum of weights.

Definition 6 (Weighted CNF-SAT Problem). *Given a set of boolean-valued variables $X = (x_1, x_2, \dots, x_n)$ and a set of tuples $C = ((C_1, w_1), (C_2, w_2), \dots, (C_m, w_m))$ where each C_i is clause and w_m is a non-negative real valued weight associated, **weighted CNF-SAT** problem is to find boolean value assignment to each $x_i \in X$ such that the sum $\sum_{C_j(x_1, x_2, \dots, x_n) = \text{True}} w_j$ is maximized.*

This problem still seems to be difficult to solve. It is not difficult to see that this problem is also an NP-hard problem as maximizing the sum of weights is equivalent to the original version of the problem with

the weight set to be 1. Therefore, rather than coming up with an algorithm that deterministically solves this problem with this generalized version of the CNF-SAT problem, we aim to provide an algorithm that approximates the optimal solution to a reasonable extent.

2 Weighted k-CNF-SAT problem

As a special case of this problem, we would want to consider a simpler version of the above problem called **k-CNF-SAT problem** where each clause in the set of constraints has exactly $k \in \mathbb{N}$ fixed number of literals.

2.1 Randomized algorithm for weighted k-CNF-SAT

With this special case, we first want to provide a simple randomized algorithm to approximate the optimal solution.

Keep the notation from the previous section's problem statement of weighted CNF-SAT. As one might expect, the most naive possible randomized algorithm to come up with is really to assign boolean values randomly to True (1) or False (0) for each variable $x_i \in X$ with some fixed probability, and study the expected value of the weighted sum of satisfied clauses. As we do not have special information on literals in the given clauses, we will set the probability of each $x_i \in X$ being True and False to be $\frac{1}{2}$, respectively.

Algorithm 1 Randomized algorithm for weighted k-CNF-SAT

- 1: **for** $i = 1$ **to** $|X|$ **do**:
 - 2: Set $x_i = 1$ with probability $\frac{1}{2}$ and $x_i = 0$ with probability $\frac{1}{2}$
 - 3: **end for**
 - 4: Calculate and return $\sum_{C_j(x_1, x_2, \dots, x_n) = \text{True}} w_j$
-

Surprisingly, this extremely simple-minded algorithm will provide a good approximation to the optimal solution of the weight CNF-SAT problem for this special case, especially when k is large.

Theorem 7. *The above-randomized algorithm has the expected value of the sum of weight that has $(1 - \frac{1}{2^k})$ -approximation for weighted CNF-SAT problem with exactly k literals for each clause*

Proof. Let Y_i be an indicator random variable where $Y_i = 1$ if the clause C_i is satisfied and $Y_i = 0$ otherwise. Moreover, denote w^* to be a random variable of the sum of the weight $\sum_{j=1}^m w_j Y_j$ (i.e. w^* is a linear combination of a random variable Y_i and its associated weight). Clearly, for any Y_i associated with clause C_i , $\mathbb{E}[Y_i] = \mathbb{P}(Y_i = 1) = (1 - \frac{1}{2^k})$ as the probability that all literals in the clause are False is $\frac{1}{2^k}$.

By linearity of expectation,

$$\mathbb{E}[w^*] = \sum_{j=1}^m w_j \mathbb{E}[Y_j] = (1 - \frac{1}{2^k}) \sum_{j=1}^m w_j \geq (1 - \frac{1}{2^k}) \text{OPT}$$

where OPT represents the optimal solution to the weighted CNF-SAT problem. Clearly, $\text{OPT} \leq \sum_{j=1}^m w_j$, so the result follows. $\square$

2.2 Deterministic algorithm for weighted k-CNF-SAT

The above simple algorithm gives a very good approximation to the optimal solution especially when k is large. Using the idea from the randomized algorithm above and conditional expectation, we will devise a

deterministic algorithm for this special case of the CNF-SAT problem.

The key idea to come up with a deterministic algorithm is to observe that every variable is a (binary) boolean variable taking values in either strictly True or False, and sequentially reducing the number of un-determined variables via conditioning.

Note that from the basic probability theory course, we have the law of total expectation which states that for any countable partition of probability space, the total expectation of a random variable can be split into a linear combination of conditional expectation.

Theorem 8 (Law of total expectation). *Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space and X be a random variable over $(\Omega, \mathcal{F}, \mathbb{P})$ that maps to $(\mathbb{R}, \mathcal{B})$ with $\mathcal{B}$ being a Borel σ -algebra over the real. If $(A_i)_{i=1}^{\infty}$ is a countable partition of the sample space Ω (i.e. for all $i, j \in \mathbb{N}$ where $i \neq j$, $A_i \cap A_j = \emptyset$, and $\bigcup_{i=1}^{\infty} (A_i) = \Omega$), then*

$$\mathbb{E}[X] = \sum_{j=1}^{\infty} \mathbb{E}[X | A_i] \mathbb{P}(A_i)$$

Using the above theorem and the proof of approximation ratio for the randomized algorithm, we observe that by conditioning on the boolean variable x_1 ,

$$\begin{aligned} \mathbb{E}[w^*] &= \mathbb{E}[w^* | x_1 = 1] \mathbb{P}(x_1 = 1) + \mathbb{E}[w^* | x_1 = 0] \mathbb{P}(x_1 = 0) \\ &= \frac{1}{2} (\mathbb{E}[w^* | x_1 = 1] + \mathbb{E}[w^* | x_1 = 0]) \\ &\geq (1 - \frac{1}{2^k}) \text{OPT} \end{aligned}$$

By choosing $x_1 = \text{argmax}_{\theta \in \{0,1\}} \mathbb{E}[w^* | x_1 = \theta]$, we have

$$(1 - \frac{1}{2^k}) \text{OPT} \leq \mathbb{E}[w^*] \leq \mathbb{E}[w^* | x_1]$$

which guarantees that conditional expectation is lower bounded by $(1 - \frac{1}{2^k}) \text{OPT}$ by assigning the boolean value for x_1 that maximizes the conditional expectation.

In order to compute conditional expectation $\mathbb{E}[w^* | x_1 = 1]$ and $\mathbb{E}[w^* | x_1 = 0]$, we first substitute x_1 and $(\neg x_1)$ with the value of x_1 we set for each case ($x_1 = 1$ and $x_1 = 0$) to obtain the set of reduced clause $C' = (C'_1, C'_2, \dots, C'_m)$. For instance, if we want to compute $\mathbb{E}[w^* | x_1 = 1]$, clause $C_1 = (\neg x_1) \vee x_3 \vee x_4$ will be reduced to $C'_1 = \text{False} \vee x_3 \vee x_4 \equiv x_3 \vee x_4$. After reducing each clause by substituting respective truth values for x_1 and $(\neg x_1)$, we can iteratively apply the same strategy from the randomized case to compute the respective conditional expectation.

Inductively applying this argument, we could show that the conditional expectation computed at each step is monotonically increasing. Repeating this boolean variable assignment for the entire set of $x_i \in X$ will lead to a deterministic algorithm for this problem.

Putting the above idea into action, we obtain the below deterministic algorithm for this problem:

Theorem 9 (Approximation Ratio of Deterministic Algorithm). *The above algorithm is deterministic, and it is $(1 - \frac{1}{2^k})$ -approximation algorithm.*

Algorithm 2 Deterministic algorithm for weighted k-CNF-SAT

- 1: **Input:** k-CNF-SAT instance $\phi = (X, \{0, 1\}, C)$ where $\{0, 1\}$ indicates the domain for all variables are boolean values
 - 2: Initialize dictionary D to store the previously chosen boolean values for $x_i \in X$
 - 3: **for** $i = 1$ to $|X|$ **do**
 - 4: Find $\mathbb{E}[w^* | x_1 = \theta_1, x_2 = \theta_2, \dots, x_{i-1} = \theta_{i-1}, x_i = 1]$ where each $\theta_i \in \{0, 1\}$ is previously chosen boolean values.
 - 5: Find $\mathbb{E}[w^* | x_1 = \theta_1, x_2 = \theta_2, \dots, x_i = 0]$ where each $\theta_i \in \{0, 1\}$ is previously chosen boolean values.
 - 6: If $\mathbb{E}[w^* | x_1 = \theta_1, x_2 = \theta_2, \dots, x_{i-1} = \theta_{i-1}, x_i = 1] \geq \mathbb{E}[w^* | x_1 = \theta_1, x_2 = \theta_2, \dots, x_i = 0]$, choose $x_i = 1$, store it into D , and update the constraint C by substituting $x_i = 1$. Otherwise, choose $x_i = 0$, store it into D , and update the constraint C by substituting $x_i = 0$.
 - 7: **end for**
 - 8: Substitute boolean values obtained from iteration above into the set of clauses C and return $\sum_{C_j(x_1, x_2, \dots, x_n) = \text{True}} w_j$
-

Proof. Clearly, the algorithm is deterministic as there is no source of randomness in the algorithm when we assign boolean values to each variable. (Note that the expected value is computed deterministically and there is no randomness involved). To check the algorithm is $(1 - \frac{1}{2^k})$ -approximation, we claim the loop invariant is that the conditional expectation at every step is lower bounded by $(1 - \frac{1}{2^k})\text{OPT}$.

For the base case where $i = 1$,

$$(1 - \frac{1}{2^k})\text{OPT} \leq \mathbb{E}[w^*] \leq \max_{\theta \in \{0, 1\}} \{\mathbb{E}[w^* | x_1 = \theta]\}$$

By choosing $x_1 = \arg\max_{\theta \in \{0, 1\}} \{\mathbb{E}[w^* | x_1 = \theta]\}$, we show that the lower bound for conditional expectation is $(1 - \frac{1}{2^k})\text{OPT}$.

Now, assume that the above loop invariant holds up to step $(i - 1)$. Observe that at step i in the loop where $\theta_1, \dots, \theta_{i-1}$ are already chosen by the algorithm,

$$\begin{aligned} & \mathbb{E}[w^* | x_1 = \theta_1, x_2 = \theta_2, \dots, x_{i-1} = \theta_{i-1}] \\ &= \frac{1}{2} (\mathbb{E}[w^* | x_1 = \theta_1, x_2 = \theta_2, \dots, x_{i-1} = \theta_{i-1}, x_i = 1] + \mathbb{E}[w^* | x_1 = \theta_1, x_2 = \theta_2, \dots, x_{i-1} = \theta_{i-1}, x_i = 0]) \\ &\leq \max_{\theta \in \{0, 1\}} \mathbb{E}[w^* | x_1 = \theta_1, x_2 = \theta_2, \dots, x_{i-1} = \theta_{i-1}, x_i = \theta] \end{aligned}$$

Again, choosing $x_i = \arg\max_{\theta \in \{0, 1\}} \mathbb{E}[w^* | x_1 = \theta_1, x_2 = \theta_2, \dots, x_{i-1} = \theta_{i-1}, x_i = \theta]$, it is clear that

$$\begin{aligned} \mathbb{E}[w^* | x_1 = \theta_1, x_2 = \theta_2, \dots, x_{i-1} = \theta_{i-1}, x_i] &\geq \mathbb{E}[w^* | x_1 = \theta_1, x_2 = \theta_2, \dots, x_{i-1} = \theta_{i-1}] \\ &\geq (1 - \frac{1}{2^k})\text{OPT} \end{aligned}$$

Therefore, the claim that updated conditional expectation at step i is monotonically increasing and lower bounded by $(1 - \frac{1}{2^k})\text{OPT}$ is invariant over the entire iteration. As every variable is fully determined at the end of the loop with the sum of weight to be $\sum_{C_j(x_1, x_2, \dots, x_n) = \text{True}} w_j = \mathbb{E}[w^* | x_1 = \theta_1, x_2 = \theta_2, \dots, x_n = \theta_n] \geq (1 - \frac{1}{2^k})\text{OPT}$, we are done. $\square$

3 General Weighted CNF-SAT Problem

Although the above randomized and deterministic algorithm is a good approximation to the case when we have a fixed number of literals for every clause, in general, that's not the case for the CNF-SAT problem. The

algorithm for the above special case only guarantees $\frac{1}{2}$ -approximation for the general weighted CNF-SAT problem since there can be clauses with 1 literal. Given this $\frac{1}{2}$ -approximation algorithm, can we come up with a better algorithm that has a better approximation ratio? Namely, we want an c -approximation algorithm where $c > \frac{1}{2}$.

3.1 Approximation algorithm for weighted CNF-SAT with positive unit clauses

Before we move on to the approximation algorithm for the general weighted CNF-SAT problem, we again consider a special case where every unit clause is positive. In other words, if a clause contains 1 literal, then the clause must be in the form of $C_i = x_i$ where $x_i \in X$, not being its negated form.

Recall where we obtained $(1 - \frac{1}{2^k})$ expression in the randomized algorithm for weighted k-CNF-SAT problem; the term $\frac{1}{2^k}$ is from calculating a probability that the clause with length k is not be satisfied **if** we assign 1 and 0 to each variable with equal probability.

As every unit clause is guaranteed to be positive, the idea to improve the previous randomized algorithm is via biasing toward the probability of assigning 1 to each boolean variable.

Algorithm 3 Randomized approximation algorithm for positive CNF-SAT

- 1: **for** $i = 1$ **to** $|X|$ **do**:
 - 2: Set $x_i = 1$ with probability $p > \frac{1}{2}$ and $x_i = 0$ with probability $1 - p$
 - 3: **end for**
 - 4: Calculate and return $\sum_{C_j(x_1, x_2, \dots, x_n) = \text{True}} w_j$
-

Given the algorithm above, the natural question to ask is 1) whether setting $x_i = p > \frac{1}{2}$ guarantees a better approximation ratio and 2) if so, how we should choose probability p that maximizes the approximation ratio. The below theorem will show that the modified algorithm will have a better approximation ratio than $\frac{1}{2}$, and it is maximized when we choose a specific p value.

Theorem 10. *The above modified randomized CNF-SAT algorithm for positive unit clause attains $\frac{\sqrt{5}-1}{2}$ -approximation if we choose $p = \frac{\sqrt{5}-1}{2}$, and the above choice of p value maximizes the approximation ratio.*

Proof. Denote $C = (C_1, C_2, \dots, C_m)$ to be a set of clauses, and $p > \frac{1}{2}$ to be a probability that $x_i = 1$ for any $x_i \in X$. For any $C_i \in C$, if C_i is a unit clause, the probability that C_i is satisfied is p as every unit clause is positive. If C_i is a disjunction of 2 literals,

$$\mathbb{P}(C_i = 1) = 1 - \mathbb{P}(C_i = 0) \geq \min\{1 - p^2, 1 - (1 - p)^2, 1 - p(1 - p)\} \geq 1 - p^2$$

To see why the above inequality holds, there are 4 possible forms that C_i can take; $C_i = x_j \vee x_k$, $C_i = (\neg x_j) \vee x_k$, $C_i = x_j \vee (\neg x_k)$, and $C_i = (\neg x_j) \vee (\neg x_k)$.

For each possible form of C_i , the probability that clause is satisfied is $1 - (1 - p)^2$, $1 - (1 - p)p$, $1 - p(1 - p)$, and $1 - p^2$, respectively. Thus,

$$\mathbb{P}(C_i = 1) = 1 - \mathbb{P}(C_i = 0) \geq \min\{1 - p^2, 1 - (1 - p)^2, 1 - p(1 - p)\}$$

Moreover, as we choose p such that $1 - p < \frac{1}{2} < p$, $\min\{1 - p^2, 1 - (1 - p)^2, 1 - p(1 - p)\} = 1 - p^2$, and the inequality in the first step follows.

If the number of literals composing $C_i = l_{i_1} \vee l_{i_2} \vee \dots \vee l_{i_k}$ is more than 2 (i.e. $k > 2$), then observe that

$$\mathbb{P}(C_i = 1) = 1 - \mathbb{P}(C_i = 0) \geq 1 - \mathbb{P}(l_{i_1} \vee l_{i_2} = 0) \geq 1 - p^2$$

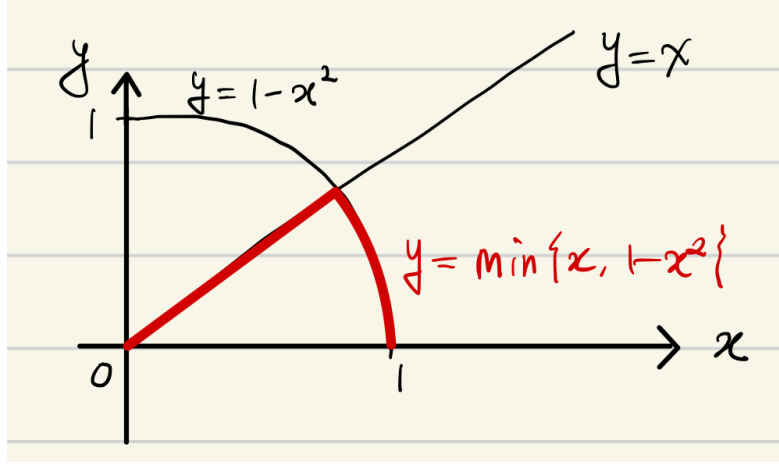


Figure 2: Graph of $y = \min\{x, 1 - x^2\}$

This is because the event where $l_{i_1} \vee l_{i_2}$ is not satisfied contains an event where $l_{i_1} \vee l_{i_2} \vee \dots \vee l_{i_k}$ is not satisfied over the sample space Ω containing all possible combination of boolean values for each literal as a sequence (i.e. $(l_{i_1}, l_{i_2}, \dots, l_{i_k}) = (0, 1, 1, \dots, 0) \in \Omega$).

To conclude our computation, we deduce that

$$\mathbb{P}(C_j = 1) = \begin{cases} p & \text{if } C_j \text{ is of 1 literal} \\ 1 - p^2 & \text{if } C_j \text{ is of more than 1 literal} \end{cases}$$

Note that $\mathbb{P}(C_j = 1) \geq \min\{p, 1 - p^2\}$ holds. Lastly, to choose the value $p > \frac{1}{2}$ that maximizes the value $\min\{p, 1 - p^2\}$ (which maximizes $\mathbb{P}(C_j = 1)$), the red graph in figure 2 above visualizes the graph $y = \min\{x, 1 - x^2\}$ where $x \in [0, 1]$.

Therefore, the maximum of $\min\{p, 1 - p^2\}$ is obtained when $p = 1 - p^2$, and by solving a quadratic equation and obtaining a positive solution to the value of p , we get $p = \frac{\sqrt{5}-1}{2} \approx 0.618$. Thus,

$$\mathbb{E}[w^*] \geq \frac{\sqrt{5}-1}{2} \sum_{j=1}^m w_j \geq \frac{\sqrt{5}-1}{2} \text{OPT}$$

and we are done. $\square$

Using the identical de-randomization strategy from the previous chapter for devising a deterministic approximation algorithm for the k-CNF-SAT problem, we can also obtain $\frac{\sqrt{5}-1}{2}$ -approximate deterministic algorithm for positive CNF-SAT problem. The only difference in this case is that $\mathbb{P}(x_j = 1) = p \neq (1 - p) = \mathbb{P}(x_j = 0)$ where we choose $p = \frac{\sqrt{5}-1}{2}$. However, we can apply the same algorithm from the previous section (**algorithm 2**) for biased probability cases as well since the sum of probability p and $(1 - p)$ is 1. Analysis and proof that the deterministic algorithm is $\frac{\sqrt{5}-1}{2}$ -approximation follows immediately (with minor modification) from the proof in the next section (refer to **theorem 12** below) where we extend the result to the general weighted CNF-SAT problem.

3.2 Approximation algorithm for general weighted CNF-SAT

Finally, we are ready to provide an algorithm that has a better approximation ratio than $\frac{1}{2}$ for the general weighted CNF-SAT problem. Relaxing the assumption that every unit clause is positive, the key idea is to bias toward unit clauses with higher weight, but by the without loss of generality assumption below, the algorithm for the general case becomes identical to the special case where every unit clause is positive.

Without loss of generality, assume that for every unit clause $C_i = x_i \in C$, $\text{weight}(x_i) \geq \text{weight}(\neg x_i)$. If $(\neg x_i)$ is not in C , then we can simply set $\text{weight}(\neg x_i) = 0$ and the above assumption holds. If $\text{weight}(x_i) < \text{weight}(\neg x_i)$, we can simply consider $y_i = (\neg x_i)$ and reverse the inequality.

For notation purpose, denote $v_i := \text{weight}(\neg x_i)$ if $(\neg x_i) \in C$ and 0 otherwise.

Lemma 1. Denote $X = (x_1, x_2, \dots, x_n)$, and $C = (C_1, C_2, \dots, C_m)$. $\text{OPT} \leq \sum_{j=1}^m w_j - \sum_{j=1}^m v_j$

Proof. The key observation is that at most one is satisfied between x_i and $(\neg x_i)$ if both unit clauses are in the set of clauses.

First of all, clearly $\text{OPT} \leq \sum_{j=1}^m w_j$. For any $i \in \mathbb{N}_{\leq n}$, if $v_i = 0$, then clearly $\text{OPT} \leq \sum_{j=1}^m w_j - \sum_{C_j \text{ is unit negative clause with weight } 0} v_j$. Thus, it suffices to consider the case when $v_i \neq 0$.

Denote $C' \subseteq C$ such that C' contains every unit negative clause with positive weight (i.e. clause with length 1 such that the literal is a negation of variable in X with positive weight). It is clear by the definition and assumption we made that C' is a collection of all unit negative literals, and if $(\neg x_i) \in C'$, then the corresponding positive unit clause x_i is in the set C . Thus, it suffices to prove that $\text{OPT} \leq \sum_{j=1}^m w_j - \sum_{C_j \in C'} v_j$.

We now prove the above inequality by induction on the cardinality of the set C' . If $|C'| = 1$, $C' = \{C_i\}$ for some $i \in \mathbb{N}_{\leq n}$ where $C_i = (\neg x_i)$. As $v_i = \text{weight}(\neg x_i) > 0$, unit clause x_i exists in the set of clause C and $\text{weight}(x_i) \geq \text{weight}(\neg x_i) > 0$ by the assumption. As the optimal solution must set x_i to be 0 or 1, either strictly x_i is satisfied or $(\neg x_i)$ is satisfied. Thus,

$$\text{OPT} \leq \left(\sum_{j=1}^m w_j \right) - \min\{\text{weight}(x_i), v_i\} = \left(\sum_{j=1}^m w_j \right) - v_i$$

This proves the base case. To prove the inductive case, suppose that $|C'| = k \geq 1$ and the above inequality holds. We want to show that the above inequality holds if $|C'| = k + 1$. We first split $C' = \{C_1, C_2, \dots, C_{k+1}\}$ into the disjoint union $C' = \{C_1, C_2, \dots, C_k\} \cup \{C_{k+1}\}$ where $C_{k+1} = (\neg x_{k+1})$. As $|\{C_1, C_2, \dots, C_k\}| = k$, from inductive hypothesis,

$$\text{OPT} \leq \sum_{j=1}^m w_j - \sum_{C_j \in C' \setminus \{C_{k+1}\}} v_j$$

The rest of the proof is identical to the proof of the base case; as OPT must set either strictly $x_{k+1} = 1$ or $x_{k+1} = 0$,

$$\begin{aligned} \text{OPT} &\leq \sum_{j=1}^m w_j - \left(\sum_{C_j \in C' \setminus \{C_{k+1}\}} v_j \right) - \min\{\text{weight}(x_{k+1}), v_{k+1}\} \\ &= \sum_{j=1}^m w_j - \left(\sum_{C_j \in C' \setminus \{C_{k+1}\}} v_j \right) - v_{k+1} \\ &= \sum_{j=1}^m w_j - \sum_{C_j \in C} v_j \end{aligned}$$

By the principle of mathematical induction, we prove that for any cardinality of C' ,

$$\text{OPT} \leq \sum_{j=1}^m w_j - \sum_{C_j \in C'} v_j$$

holds, and we conclude the proof of this lemma. $\square$

Now, we are ready to prove the main result for this section.

Theorem 11. *Running randomized **algorithm 3** for the general weighted CNF-SAT problem above with $\mathbb{P}(x_i = 1) = p = \frac{\sqrt{5}-1}{2}$ is also $\frac{\sqrt{5}-1}{2}$ -approximation algorithm.*

Proof. Again, denote $C' \subseteq C$ such that C' contains every unit negative clause with positive weight.

$$\begin{aligned} \mathbb{E}[w^*] &= \sum_{j=1}^m (w_j \mathbb{P}(C_j = 1)) = \sum_{C_j \in C'} (w_j \mathbb{P}(C_j = 1)) + \sum_{C_j \notin C'} (w_j \mathbb{P}(C_j = 1)) \\ &\geq (1-p) \sum_{C_j \in C'} w_j + \sum_{C_j \notin C'} (w_j \min\{p, 1-p^2\}) \\ &= \left(\frac{\sqrt{5}-1}{2}\right) \sum_{C_j \notin C'} w_j + \left(1 - \frac{\sqrt{5}-1}{2}\right) \sum_{C_j \in C'} w_j \\ &= \left(\frac{\sqrt{5}-1}{2}\right) \left(\sum_{j=1}^m w_j\right) + (1 - (\sqrt{5}-1)) \sum_{C_j \in C'} w_j \\ &= \left(\frac{\sqrt{5}-1}{2}\right) \left(\sum_{j=1}^m w_j\right) - (\sqrt{5}-2) \sum_{C_j \in C'} w_j \\ &\geq \left(\frac{\sqrt{5}-1}{2}\right) \left(\sum_{j=1}^m w_j - \sum_{C_j \in C'} w_j\right) \\ &= \left(\frac{\sqrt{5}-1}{2}\right) \text{OPT} \end{aligned}$$

The second last inequality is from the fact that $\sqrt{5}-2 < 0.5 < \frac{\sqrt{5}-1}{2}$, and the last equality is from the previous lemma. $\square$

The next theorem will provide a combined proof that the deterministic algorithm also attains $\frac{\sqrt{5}-1}{2}$ -approximation.

Theorem 12. *Running de-randomized deterministic algorithm (**algorithm 2**) also gives $\frac{\sqrt{5}-1}{2}$ -approximation solution.*

Proof. The proof is essentially identical to the proof of **theorem 9**. We again claim that the loop invariant is that the conditional expectation (i.e. the resultant conditional expectation computed at the end of the loop in **algorithm 2**) at every step is lower bounded by $\frac{\sqrt{5}-1}{2} \text{OPT}$.

Let $p = \frac{\sqrt{5}-1}{2}$. For the base case where $i = 1$, from **theorem 11** and the law of total expectations,

$$\begin{aligned} \frac{\sqrt{5}-1}{2} \text{OPT} &\leq \mathbb{E}[w^*] = \mathbb{P}(x_i = 1) \mathbb{E}[w^* | x_i = 1] + \mathbb{P}(x_i = 0) \mathbb{E}[w^* | x_i = 0] \\ &= p \cdot \mathbb{E}[w^* | x_i = 1] + (1-p) \cdot \mathbb{E}[w^* | x_i = 0] \\ &\leq (p + (1-p)) \max_{\theta \in \{0,1\}} \{\mathbb{E}[w^* | x_1 = \theta]\} \end{aligned}$$

$$= \max_{\theta \in \{0,1\}} \{\mathbb{E}[w^* | x_1 = \theta]\}$$

Again, identical to **theorem 9**, by choosing $x_1 = \operatorname{argmax}_{\theta \in \{0,1\}} \max\{\mathbb{E}[w^* | x_1 = \theta]\}$, we show that the lower bound for conditional expectation is $\frac{\sqrt{5}-1}{2} \text{OPT}$.

The inductive case is completely identical to the proof of **theorem 9** once we show that the base case holds.

Therefore, the claim that updated conditional expectation at each step i is lower bounded by $\frac{\sqrt{5}-1}{2} \text{OPT}$ is invariant over the entire loop, and by identical argument from **theorem 9**, we are done with the proof. (You can check that the details are really identical to **theorem 9** once you prove **theorem 11** and the base case!) $\square$

3.3 Linear programming formulation of general weighted CNF-SAT

The last part of this lecture is devoted to formulating general weighted CNF-SAT problems into integer and linear programming by encoding the variables and constraints as a linear constraint and conditions. We will first formula the problem to integer programming, and relax the conditions to linear programming to find the optimal solution.

The first step is to set variables. For the weighted CNF-SAT problem, it is natural to reuse the set of boolean variables as a variable for linear programming. If $x_i = 1$, it means that x_i is set to be True, and if $x_i = 0$, x_i is set to be False. In addition, we want to set another auxiliary variable $(y_k)_{k=1}^m$ (where m is the total number of clauses) to indicate whether the clause $C_i \in C$ is satisfied or not.

Given these variables, our aim is to maximize the total sum of the weights of the satisfied clauses. In other words, we want to maximize $\sum_{j=1}^m w_j \cdot y_j$ where $y_j = 1$ means that clause C_j is satisfied and $y_j = 0$ means that the clause is not satisfied.

Finally, we translate the constraints of the CNF-SAT problem into integer programming form. For any clause $C_i \in C$, C_i is satisfied if any one of the literal composing C_i is set to be true. Denote $C_i = l_{i_1} \vee l_{i_2} \vee \dots \vee l_{i_k}$. Then each literal composing C_i , say l_{i_j} , is either strictly x_{i_j} or $(\neg x_{i_j})$. If $l_{i_j} = x_{i_j}$, we represent l_{i_j} as a variable x_{i_j} we defined above. If $l_{i_j} = (\neg x_{i_j})$, l_{i_j} is satisfied if x_{i_j} is assigned to False. Thus, we represent l_{i_j} as a variable (or value of) $(1 - x_{i_j})$.

Using the above formulation, if we represent $C_i = l_{i_1} \vee l_{i_2} \vee \dots \vee l_{i_k}$ where k is the number of literals composing C_i , then integer programming formulation of the constraint is to impose the following constraint:

$$\sum_{i=1}^k l_{i_j} = \sum_{l_j \text{ is positive}} x_j + \sum_{l_j \text{ is negative}} (1 - x_j) \geq y_i$$

where positive and negative here have the same meaning as in previous sections. This is because, if the clause is not satisfied, the sum can be any value larger than or equal to 0. If the clause is satisfied, then the sum of (values associated with) literals should be greater than or equal to $1 = y_i$.

To conclude, we have the following integer programming formulation of the weighted CNF-SAT problem:

$$\text{Maximize } \sum_{j=1}^m w_j \cdot y_j$$

subject to:

$$0 \leq x_i \leq 1 \quad \text{and } x_i \in \mathbb{Z} \text{ for all } i \text{ smaller than the number of variables } n$$

$$0 \leq y_j \leq 1 \quad \text{and } y_j \in \mathbb{Z} \text{ for all } j \text{ smaller than the number of clauses } m$$

For each clause $C_i = l_{i_1} \vee l_{i_2} \vee \dots \vee l_{i_k}$, where k is the number of literals composing C_i , the corresponding constraint is:

$$\sum_{i=1}^k l_{i_j} = \sum_{l_j \text{ is positive}} x_j + \sum_{l_j \text{ is negative}} (1 - x_j) \geq y_i$$

Relaxing the condition that for each x_i and y_j are integers, the above becomes a linear programming formulation of a general weighted CNF-SAT problem.

After obtaining the LP solution to the above problem, where we denote $x_1^*, x_2^*, \dots, x_n^*, y_1^*, \dots, y_m^*$ to be an optimal solution to the LP problem above, we simply choose $\mathbb{P}(x_i = 1) = x_i^*$ and run the **algorithm 3** with the chosen probability above for each variable x_i to obtain the weighted sum of satisfied clauses. We can obtain a deterministic solution (algorithm) as well via the exact same de-randomization procedures we used multiple times in the previous section.

Now we have obtained a linear programming formulation of the general weighted CNF-SAT problem, analysis of the approximation ratio and performance of this algorithm will be further continued to be analyzed in the next lecture, lecture 5.

Lovász Local Lemma

Lecturer: Divesh Aggarwal

September 19, 2024

Scribe: Tai Tze Kin, Li Yuting

Abstract

In today's session, we will continue our exploration of solving **Conjunctive Normal Form Satisfaction** (CNF-SAT) problems using linear programming, following up on our discussion from last week. We will analyze the algorithm to evaluate its approximation ratio. Subsequently, we will introduce the **Lovász Local Lemma** and examine its utility in determining when a **Constraint Satisfaction Problem** (CSP) is fully satisfiable. Furthermore, we will discuss an algorithm that leverages this lemma and demonstrate how it can be applied to prove satisfiability for specific CSPs that meet certain conditions.

1 Linear Programming based Algorithm

Last week, we ended off by explaining how to formulate the general weighted CNF-SAT problem as an integer linear programming (ILP) problem. To recap, the formulation is as follows:

$$\text{Maximize } \sum_{j=1}^m w_j \cdot z_j,$$

subject to:

$$\sum_{i \in P_j} y_i + \sum_{i \in N_j} (1 - y_i) \geq z_j,$$

$0 \leq y_i \leq 1$ and $y_i \in \mathbb{Z}$ for all $i \leq$ the numbers of variables n ,

$0 \leq z_j \leq 1$ and $z_j \in \mathbb{Z}$ for all $j \leq$ the numbers of clauses m ,

where P_j denotes the set of positive literals (x_i) in clause C_j , and N_j denotes the set of negative literals ($\neg x_i$) in clause C_j .

Directly solving the ILP is challenging. Consequently, we relax the integer constraints on y and z from $\mathbb{Z}$ to $\mathbb{R}$, thereby transforming the problem into a linear programming (LP) problem. By solving this LP, we can obtain solutions $y_1^*, \dots, y_n^*$ and $z_1^*, \dots, z_m^*$, with a total weight given by $W^* = \sum_{j=1}^m w_j z_j^*$. We can then sample $x_1, \dots, x_n$ independently from $\{0, 1\}$ with probabilities $\mathbb{P}(x_i = 1) = y_i^*$ to derive an approximate solution to the original ILP problem.

1.1 Approximation Ratio

We now start to analyze the approximation ratio of the above algorithm.

Theorem 1. *The above algorithm is a $(1 - \frac{1}{e})$ -approximation algorithm for CNF-SAT.*

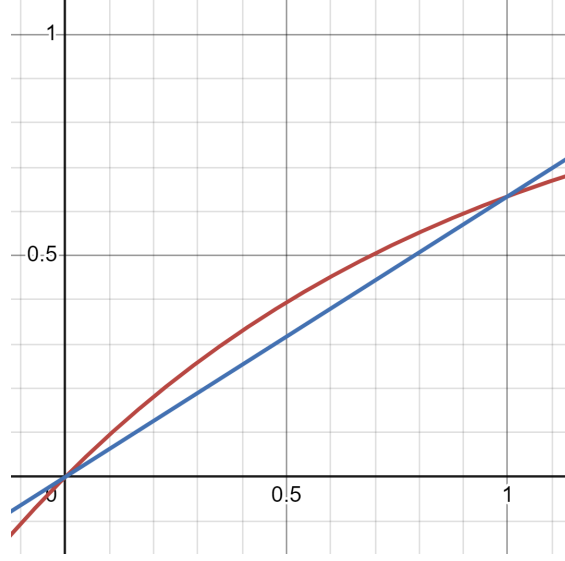


Figure 1: $f(z)$ in red and $g(z) := f(1)z$ in blue. As f is concave downwards, $f(z) \geq f(1)z$ for $z \in [0, 1]$.

Proof. We want to find the expected number of clauses that will be satisfied. By linearity of expectation, we have

$$\mathbb{E}[\text{\# of clauses satisfied}] = \sum_{j=1}^m \mathbb{P}[C_j \text{ is satisfied}]$$

To determine the probability that a single clause is satisfied, it is more convenient to first consider the probability that the clause is not satisfied. Recall that each clause is of the form $C_j = x_{i_1} \vee x_{i_2} \vee \dots \vee x_{i_{l_j}}$, where l_j represents the number of literals in clause j . For a disjunction to evaluate to false, all literals must evaluate to false.

$$\begin{aligned} \mathbb{P}[C_j \text{ is NOT satisfied}] &= \prod_{i \in P_j} (1 - y_i^*) \prod_{i \in N_j} y_i^* \\ &\leq \left[\frac{1}{l_j} \left(\sum_{i \in P_j} (1 - y_i^*) + \sum_{i \in N_j} y_i^* \right) \right]^{l_j} \quad (\text{by A.M.-G.M. inequality}) \\ &= \left[1 - \frac{1}{l_j} \left(\sum_{i \in P_j} y_i^* + \sum_{i \in N_j} (1 - y_i^*) \right) \right]^{l_j} \quad (\because |P_j| + |N_j| = l_j) \\ &\leq \left(1 - \frac{z_j^*}{l_j} \right)^{l_j} \end{aligned}$$

And so we now have a lower bound on the probability of clause C_j being satisfied as a function of z_j^* :

$$\mathbb{P}[C_j \text{ is satisfied}] \geq 1 - \left(1 - \frac{z_j^*}{l_j} \right)^{l_j} = f(z_j^*)$$

Next, we will analyze $f(z_j^*)$ and $f(1)z_j^*$. Notably, the function f is concave downwards. The term $f(1)z_j^*$

represents a straight line connecting the points $(0,0)$ and $(1, f(1))$. Consequently, $f(1)z_j^*$ serves as a strategic choice for a lower bound of $f(z_j^*)$ within the interval $z_j^* \in [0, 1]$, as illustrated in Figure 1.

$$f(z_j^*) \geq f(1)z_j^* = \left[1 - \left(1 - \frac{1}{l_j}\right)^{l_j}\right] z_j^*$$

To remove l_j from the bound, we can utilize the fact that $\forall x \in [0, 1], (1-x)^{\frac{1}{x}} \leq \frac{1}{e}$.

$$\left[1 - \left(1 - \frac{1}{l_j}\right)^{l_j}\right] z_j^* \geq \left(1 - \frac{1}{e}\right) z_j^* \approx 0.63 z_j^*$$

To wrap up this analysis, we finally can bound the expected number of satisfied clauses:

$$\begin{aligned} \mathbb{E}[\text{\# of classes satisfied}] &= \sum_{j=1}^m \mathbb{P}[C_j \text{ is satisfied}] \\ &\geq \sum_{j=1}^m f(z_j^*) \\ &\geq \left(1 - \frac{1}{e}\right) \sum_{j=1}^m z_j^* \\ &\geq \left(1 - \frac{1}{e}\right) \text{OPT} \end{aligned}$$

The last step relies on the fact that the solution to the more relaxed LP $(\sum_{j=1}^m z_j^*)$ will always be at least as large as the actual optimal solution for the ILP. This concludes the proof of the approximation ratio achieved by our LP-based algorithm. $\square$

1.2 Combined Solution

We can actually achieve even better results by simply combining the LP-based algorithm with the randomized algorithm introduced in the last lecture. Recall the simple randomized algorithm randomly assigns True or False to each variable, which leads to an approximation ratio of $\left(1 - \frac{1}{2^{l_j}}\right)$. However, its performance diminishes when the clauses contain fewer literals, resulting in a worst-case approximation ratio of $\frac{1}{2}$.

To improve upon this, we can combine the randomized algorithm with the LP-based algorithm, which has an approximation ratio of $\left(1 - \left(1 - \frac{1}{l_j}\right)^{l_j}\right)$. By running both algorithms on the problem and retaining a better solution, we can enhance our overall results.

The approximation ratio of our combined algorithm will be the maximum of the approximation ratios from the two individual algorithms. Upon graphing these two functions, we observe that they intersect at a value of 0.75 when $l_j = 2$. This indicates that our combined algorithm achieves an improved approximation ratio of 0.75.

Furthermore, we can apply the same derandomization technique introduced last week to develop a deterministic algorithm.

2 CSP satisfiability

Let us now return to the more general Constraint Satisfaction Problem (CSP), where each variable x_i can take on any value from a domain D_i , rather than being restricted to *True* or *False*. Each constraint takes the form $C : x_{i_1} \neq \alpha_1 \vee x_{i_2} \neq \alpha_2 \vee \dots \vee x_{i_k} \neq \alpha_k$. In other words, a constraint is false if and only if every variable x assumes a specific undesirable value α from its domain. Furthermore, instead of seeking an assignment of variables that maximizes the total number of satisfied constraints, we aim to determine whether it is possible to satisfy all constraints simultaneously.

Is there a straightforward method to ascertain the satisfiability of certain CSPs? Intuitively, one might observe that the following two properties could facilitate the satisfiability of the CSP,

- The number of variables is relatively larger than the number of constraints.
- Constraints are largely independent, *i.e.*, they do not share variables with most other constraints.

Indeed, there exists a lemma that encapsulates this intuition and can be employed to prove the existence of a solution under specific conditions.

2.1 Lovász Local Lemma

Let ϕ be an instance of a CSP and $C \in \phi$ be a constraint, and $\text{vbl}(C)$ be the set of variables that occur in constraint C . Then we can define $w(C)$ of a constraint as follows:

$$w(C) := \prod_{x_i \in \text{vbl}(C)} \frac{1}{|D_i|}$$

This represents the probability that the constraint will not be satisfied if we randomly assign values to each of the variables.

We further define the *neighborhood* of a constraint C in a CSP as the set of other constraints that share at least one variable with C .

$$\Gamma_\phi(C) := \{C' \in \phi : \text{vbl}(C) \cap \text{vbl}(C') \neq \emptyset, C' \neq C\}$$

Now we are prepared to introduce the Lovász Local Lemma as it applies to the Constraint Satisfaction Problem (CSP).

Lemma 1 (Lovász Local Lemma). *Let ϕ be a CSP. If $\exists \mu_C \in (0, 1)$ for all $C \in \phi$ such that $\forall C \in \phi, w(C) \leq \mu_C \prod_{C' \in \Gamma_\phi(C)} (1 - \mu_{C'})$, then ϕ is satisfiable.*

The value of μ_C intuitively represents the difficulty of satisfying a given constraint. If constraint C is more challenging to satisfy, μ_C is chosen to be larger; conversely, if C is easier to satisfy, μ_C can be smaller. When a constraint has a larger neighborhood $\Gamma_\phi(C)$, it necessitates a larger μ_C to maintain the inequality. However, this increased value makes it more difficult to satisfy the inequalities for other constraints C' that share variables with C .

Proof. To prove the Lovász Local Lemma, we employ a probabilistic method. We want to show that if we randomly assign values for all the variables, the probability that every constraint in our CSP is satisfied is

greater than 0. This would be sufficient to show that a solution must exist. First, we express this probability as a product of conditional probabilities as follows:

$$\mathbb{P}[C_1 \wedge C_2 \wedge \dots \wedge C_m] = \mathbb{P}[C_1] \cdot \mathbb{P}[C_2|C_1] \cdot \mathbb{P}[C_3|C_1, C_2] \cdot \dots \cdot \mathbb{P}[C_m|C_1, C_2, \dots, C_{m-1}]$$

If we can show that each of these individual conditional probabilities is greater than 0, the whole product will be positive and we would have successfully proved our lemma. To achieve this, we will prove an even stronger bound on the probabilities. Note that we've shortened μ_i to mean μ_{C_i} for brevity.

Lemma 2. $\forall i, \mathbb{P}[C_{i+1}|C_1, C_2, \dots, C_i] \geq 1 - \mu_{i+1}$

We will prove the lemma using strong induction on i . First, consider the base case $i = 0$. Clearly, $\mathbb{P}[C_1] = 1 - w(C_1)$. By the condition set forth by the Lovász Local Lemma, we know that $w(C_1) \leq \mu_1 \prod_{C' \in \Gamma_\phi(C_1)} (1 - \mu_{C'})$.

Since each $\mu_{C'} \in (0, 1)$, we observe that $\prod_{C' \in \Gamma_\phi(C_1)} (1 - \mu_{C'}) \leq 1$. Thus, it follows that:

$$w(C_1) \leq \mu_1,$$

and therefore,

$$\mathbb{P}[C_1] = 1 - w(C_1) \geq 1 - \mu_1,$$

which proves the base case.

For the inductive step, suppose the inductive hypothesis holds for all $i = 0, 1, \dots, j-1$. Since the lemma does not make any assumptions regarding the order of the constraints, we are free to reorder them as needed. Let us reorder the constraints such that C_1 to C_l are the constraints that share variables with C_{j+1} , while C_{l+1} to C_j are the constraints that do not share any variables with C_{j+1} .

Here it is easier to consider the complement - the conditional probability that constraint C_{j+1} is not satisfied.

$$\begin{aligned} & \mathbb{P}[\overline{C_{j+1}}|C_1, C_2, \dots, C_j] \\ &= \frac{\mathbb{P}[\overline{C_{j+1}} \wedge C_1 \wedge C_2 \wedge \dots \wedge C_l|C_{l+1}, \dots, C_j]}{\mathbb{P}[C_1 \wedge C_2 \wedge \dots \wedge C_l|C_{l+1}, \dots, C_j]} \\ &\leq \frac{\mathbb{P}[\overline{C_{j+1}}|C_{l+1}, \dots, C_j]}{\mathbb{P}[C_1 \wedge C_2 \wedge \dots \wedge C_l|C_{l+1}, \dots, C_j]} \\ &= \frac{\mathbb{P}[\overline{C_{j+1}}]}{\mathbb{P}[C_1 \wedge C_2 \wedge \dots \wedge C_l|C_{l+1}, \dots, C_j]} \\ &= \frac{w(C_{j+1})}{\mathbb{P}[C_1 \wedge C_2 \wedge \dots \wedge C_l|C_{l+1}, \dots, C_j]} \\ &= \frac{w(C_{j+1})}{\mathbb{P}[C_1|C_2, \dots, C_j] \cdot \mathbb{P}[C_2|C_3, \dots, C_j] \cdot \dots \cdot \mathbb{P}[C_l|C_{l+1}, \dots, C_j]} \\ &\leq \frac{w(C_{j+1})}{(1 - \mu_1)(1 - \mu_2) \dots (1 - \mu_l)} \end{aligned}$$

The first inequality holds because removing additional terms in a conjunction can only increase the probability that the entire expression is satisfied. The second inequality follows by applying the induction hypothesis to the denominator. Again, since the induction hypothesis does not assume a specific ordering of the constraints, we can renumber the constraints as needed to apply the result effectively.

By the condition required for the Lovász Local Lemma, we know that

$$w(C_{j+1}) \leq \mu_{j+1} \prod_{C' \in \Gamma_\phi(C_{j+1})} (1 - \mu_{C'}) \implies \frac{w(C_{j+1})}{\prod_{C' \in \Gamma_\phi(C_{j+1})} (1 - \mu_{C'})} \leq \mu_{j+1}.$$

The fraction on the right matches exactly the expression we derived earlier. Therefore, we have shown that $\mathbb{P}[\overline{C_{j+1}} \mid C_1, C_2, \dots, C_j] \leq \mu_{j+1}$, which implies $\mathbb{P}[C_{j+1} \mid C_1, C_2, \dots, C_j] \geq 1 - \mu_{j+1}$, thus completing the proof. $\square$

2.2 Moser's algorithm

In 2008, Robin Moser published an algorithm based on the Lovász Local Lemma to actually find a solution to the CSP. Provided that the condition for the lemma is satisfied, the following algorithm can be used to find a solution:

Algorithm 1 Moser's Algorithm

Randomly assign values to all variables
while $\exists$ an unsatisfied constraint C_j **do**
 Randomly assign values for all variables in C_j
end while

Despite its seeming simplicity, we can prove that it is expected to find a solution fairly quickly. The expected number of times the loop will run is given by:

$$E[N] = \frac{\mu_1}{1 - \mu_1} + \frac{\mu_2}{1 - \mu_2} + \dots + \frac{\mu_m}{1 - \mu_m}$$

The proof for this fact is too complicated and so it is omitted.

Incidentally, the best algorithm for k-SAT, Schoning's algorithm, also works in a similar local search-like fashion, randomly assigning values and then modifying them iteratively until the system is satisfied. It can be shown that the algorithm expects to find a solution in $(\frac{2(k-1)}{k})^n$ iterations.

2.3 (d, k)-CSP satisfiability

Here we provide an example of using the Lovász Local Lemma to prove a particular kind of CSP is satisfiable under an appropriate condition.

Theorem 2. *Let ϕ be a (d, k) -CSP, meaning each constraint has exactly k literals and each variable has a domain of size d . Suppose that $\forall C \in \phi, |\Gamma_\phi(C)| \leq \frac{d^k}{e} - 1$. Then ϕ is satisfiable.*

Proof. In order to apply Lovász Local Lemma, we want to find appropriate values for μ such that $w(C) \leq \mu_C \prod_{C' \in \Gamma_\phi(C)} (1 - \mu_{C'})$. It turns out that choosing $\mu = \frac{e}{d^k}$ for every constraint suffices. The right hand side now becomes:

$$\mu(1 - \mu)^{|\Gamma_\phi(C)|}$$

$$\begin{aligned}
&\geq \mu(1-\mu)^{\frac{d^k}{e}-1} \\
&= \frac{e}{d^k} \left(1 - \frac{e}{d^k}\right)^{\frac{d^k}{e}-1} \\
&= \frac{e}{d^k} \left(1 - \frac{1}{\frac{d^k}{e}}\right)^{\frac{d^k}{e}-1} \\
&\geq \frac{e}{d^k} \left(\frac{1}{e}\right) \\
&= \frac{1}{d^k}
\end{aligned}$$

For the last inequality, we use the fact that $\forall x \geq 1, (1 - \frac{1}{x})^{x-1} \geq \frac{1}{e}$ to simplify the expression. Clearly $\frac{1}{d^k} = w(C) \leq \frac{1}{d^k} \leq \mu(1-\mu)^{|\Gamma_\phi(C)|}$, so we have successfully proven that the CSP is satisfiable using Lovász Local Lemma. $\square$

In essence, what we have demonstrated is that, as long as each constraint does not share variables with too many other constraints, the CSP is guaranteed to be satisfiable. In such cases, we can apply algorithms like *Moser's algorithm* to efficiently find a solution. However, if the system is highly dense, with many constraints interdependent on one another, it may be more appropriate to consider alternative algorithms we have studied to find an approximate or good-enough solution.

Network Flow Problem

Lecturer: Divesh Aggarwal

October 3, 2024

Scribe: Jeffrey Lee, Shaun Quek, Yeo Jun Jie, Low ZhengTing Malcolm, Tan Juay Hee

Abstract

In this lecture, we introduce the *Network Flow* problem, and describe *flows* and *cuts*. We observe the *flow-cut duality* bound, and strengthen it into the *Max-flow Min-cut Theorem*. To prove the Max-flow Min-cut Theorem, we introduce the concepts of *augmenting paths* and *residual flows & graphs*; this culminates in a presentation of the *Ford-Fulkerson algorithm* as an augmenting path algorithm.

1 Flow Networks

Suppose we have a directed graph $G = (V, E)$, with vertices V and directed edges E .

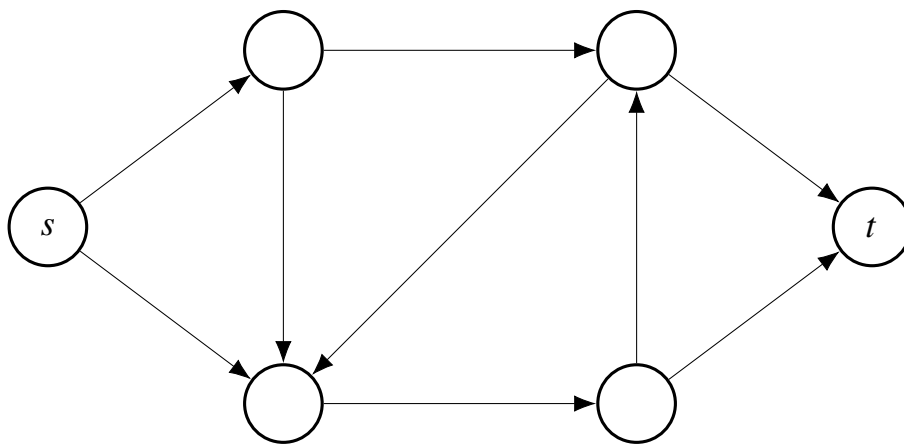


Figure 1: A directed graph.

We can specialize this graph into a **flow network** by giving each edge $u \rightarrow v$ some nonnegative real (and most often, integer) capacity $c(u \rightarrow v)$. That is,

Definition 1 (Flow Network). A **flow network** is a directed graph $G = (V, E)$ equipped with a capacity function c , where

$$c : E \rightarrow \mathbb{R}_{\geq 0}$$

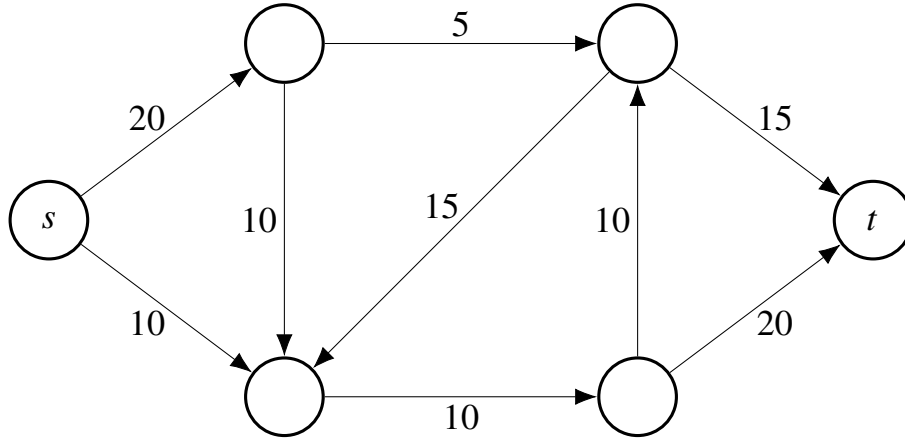


Figure 2: A flow network, with edge capacities marked.

Two of the most common use-cases for flow networks are for analysis in terms of **flows** (especially maximum flows), and **cuts** (especially minimum cuts). In short, a maximum flow describes the maximum amount of material we can transfer between two vertices $s, t \in V$, while a minimum cut describes the cheapest way to disconnect $s, t \in V$ within the network.

Let us introduce both, beginning with **flows**.

2 Network Flows

A **flow** is any one way to assign each edge e of a flow network some **flow value** $f(e)$, such that $f(e)$ is within the edge's capacity $c(e)$ and flow values are conserved at vertices. Formally,

Definition 2 ((s, t) -flow). Let $G = (V, E)$ with capacity c be a flow network, and let $s, t \in V$ be any two fixed vertices of G . An (s, t) -**flow on G** is a function f ,

$$f : E \rightarrow \mathbb{R}_{\geq 0}$$

where f satisfies the conservation constraint:

Conservation Constraint. For any $v \in V \setminus \{s, t\}$, we must have¹

$$\sum_{u \in V} f(u \rightarrow v) = \sum_{w \in V} f(v \rightarrow w)$$

In other words, the **inflow** into each vertex v (LHS above) must be equal to the **outflow** from v (RHS above), except at vertices s and t . This can be characterized by describing the **net flow** on each vertex,

$$\Delta f(v) = \sum_{u \in V} f(u \rightarrow v) - \sum_{w \in V} f(v \rightarrow w)$$

¹ $f(u \rightarrow v) := 0$ if $(u \rightarrow v) \notin E$

where again all $\Delta f(v) = 0$ except $\Delta f(s)$ and $\Delta f(t)$.

Where s and t are fixed & obvious, we may refer to an (s,t) -flow simply as a flow.

Definition 3 (Flow value). The **value** $|f|$ of a flow f is the net flow out of s ,

$$|f| := \Delta f(s)$$

Recall that $\Delta f(s)$ and $\Delta f(t)$ are unconstrained, whereas all other $\Delta f(v) = 0$. However, we find that we must also have

$$\begin{aligned} |f| = \Delta f(s) &= -\Delta f(t) \\ \sum_{v \in V} f(v) &= 0 \end{aligned}$$

Proof.

$$\begin{aligned} \sum_{v \in V \setminus \{s,t\}} \Delta f(v) &= \sum_{v \in V \setminus \{s,t\}} \left(\sum_{u \in V} f(u \rightarrow v) - \sum_{w \in V} f(v \rightarrow w) \right) = 0 \\ &\Rightarrow \sum_{v \in V \setminus \{s,t\}, u \in V} f(u \rightarrow v) - \sum_{v \in V \setminus \{s,t\}, w \in V} f(v \rightarrow w) = 0 \\ &\Rightarrow \sum_{v \in V \setminus \{s,t\}, u \in \{s,t\}} f(u \rightarrow v) - \sum_{v \in V \setminus \{s,t\}, w \in \{s,t\}} f(v \rightarrow w) = 0 \\ &\Rightarrow \sum_{v \in V \setminus \{s,t\}} f(s \rightarrow v) + \sum_{v \in V \setminus \{s,t\}} f(t \rightarrow v) - \sum_{v \in V \setminus \{s,t\}} f(v \rightarrow s) - \sum_{v \in V \setminus \{s,t\}} f(v \rightarrow t) = 0 \\ &\Rightarrow \Delta f(s) + \Delta f(t) = 0 \end{aligned}$$

□

Lastly, we call a flow **feasible** if the flow value is within capacity on every edge.

Definition 4 (Feasible flow). A flow is a **feasible flow** if for all $u \rightarrow v \in E$,

$$0 \leq f(u \rightarrow v) \leq c(u, v)$$

Where obvious, we may also discuss only feasible flows, and simply call them flows.

We call a feasible flow a **maximum flow** if it has maximum flow value among all feasible flows on the same graph.

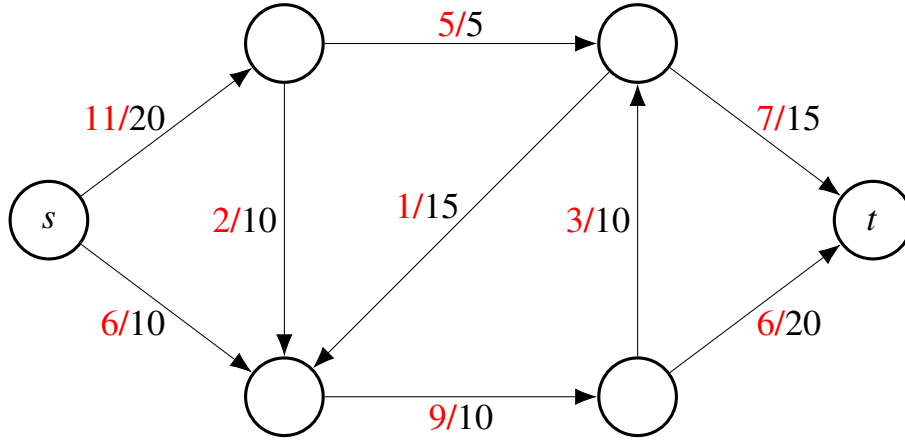


Figure 3: A feasible flow on a flow network, with flow value 13. Edges are marked $f(u \rightarrow v)/c(u \rightarrow v)$.

3 Network Cuts

Definition 5 ((s,t) -cut). Let $G = (V, E)$ with capacity c be a flow network, and let $s, t \in V$ be any two fixed vertices of G . An (s,t) -cut on G is a pair of sets (S, T) that form a partition of V ,

$$S \cup T = V$$

$$S \cap T = \emptyset$$

and $s \in S, t \in T$.

Where s and t are fixed & obvious, we may refer to an (s,t) -cut simply as a cut.

Definition 6 (Cut capacity). The **cut capacity** $\|S, T\|$ of a cut (S, T) is the sum capacity of edges deleted by the partitioning,

$$\|S, T\| := \sum_{v \in S} \sum_{w \in T} c(v \rightarrow w)$$

We call a cut a **minimum cut** if it has minimum cut capacity among all cuts on the same graph.

Lemma 1. Let $G = (V, E)$ with capacity c be a flow network, and fix any $s, t \in V$. Suppose f is any feasible (s,t) -flow on G , and suppose (S, T) is any (s,t) -cut of G . Then, the value of f is less than or equal the capacity of (S, T) ,

$$|f| \leq \|S, T\|$$

Moreover, $|f| = ||S, T||$ if and only if f saturates every edge from S to T and avoids every edge from T to S .

Proof.

$$\begin{aligned}
|f| &= \Delta f(s) = \sum_{v \in S} \Delta f(v) \\
&= \sum_{v \in S} \sum_{w \in V} f(v \rightarrow w) - \sum_{v \in S} \sum_{u \in V} f(u \rightarrow v) \\
&= \sum_{v \in S} \sum_{w \in S} f(v \rightarrow w) + \sum_{v \in S} \sum_{w \in T} f(v \rightarrow w) - \sum_{v \in S} \sum_{u \in S} f(u \rightarrow v) - \sum_{v \in S} \sum_{u \in T} f(u \rightarrow v) \\
&= \sum_{v \in S} \sum_{w \in T} f(v \rightarrow w) - \sum_{v \in S} \sum_{u \in T} f(u \rightarrow v) \\
&\leq \sum_{v \in S} \sum_{w \in T} c(v \rightarrow w) - \sum_{v \in S} \sum_{u \in T} 0 \\
&= \sum_{v \in S} \sum_{w \in T} c(v \rightarrow w) = ||S, T||
\end{aligned}$$

with the above inequality since all such $f(v \rightarrow w) \leq c(v \rightarrow w)$, and all such $f(u \rightarrow v) \geq 0$. Moreover, equality holds if and only if all such $f(v \rightarrow w) = c(v \rightarrow w)$ and all such $f(u \rightarrow v) = 0$.

□

4 MFMC Theorem & Augmenting Paths

Theorem 7 (Max-flow Min-cut Theorem). *Let $G = (V, E)$ with capacity c be a flow network, and fix any $s, t \in V$. There exists a feasible flow f and a cut (S, T) on G , such that f saturates every edge from S to T and avoids every edge from T to S , with*

$$|f| = ||S, T||$$

Note that this f must have the maximum flow value $|f|$ out of all flows, since all flow values are bounded above by $||S, T||$.

Similarly, this (S, T) must have the minimum cut capacity out of all cuts, since all cut capacities are bounded below by $|f|$.

Thus, in other words, all flow networks have Max-flow = Min-cut,

$$\max_f |f| = \min_{(S, T)} ||S, T||$$

Proof. In this proof, we assume that for any $u, v \in V$, at most one of $c(u \rightarrow v)$ and $c(v \rightarrow u)$ are non-zero. However, this proof still applies to arbitrary flow networks, since an arbitrary flow network can be converted into one satisfying the condition via:

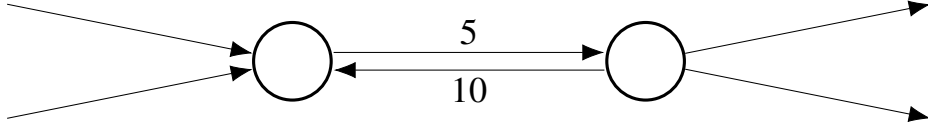


Figure 4: A bidirectional edge in a flow network ...

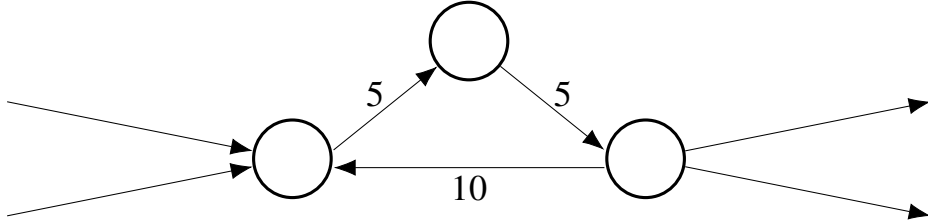


Figure 5: ... converted into a cycle of single directed edges.

after which any flow, cut, or path on the converted network can easily be translated into one on the original network.

In this proof, we will also consider only feasible flows and refer to them as flows.

For a flow f , we can describe a **residual capacity** of f , based on where flow value can still be added after f is applied to the network. The non-zero edges of a flow's residual capacity can also be taken to form a **residual graph**.

Definition 8 (Residual capacity). *Let f be a (feasible) flow on the flow network G . The **residual capacity** c_f of f is the function $c_f : E \rightarrow \mathbb{R}_{\geq 0}$,*

$$c_f(u \rightarrow v) = \begin{cases} c(u \rightarrow v) - f(u \rightarrow v) & \text{if } u \rightarrow v \in E \\ f(v \rightarrow u) & \text{if } v \rightarrow u \in E \\ 0 & \text{otherwise} \end{cases}$$

Definition 9 (Residual graph). *Let f be a flow on the flow network $G = (V, E)$. The **residual graph** G_f of G under f is the directed graph $G_f = (V, E_f)$,*

$$E_f = \{u \rightarrow v : \forall u, v \in V \text{ such that } c_f(u \rightarrow v) > 0\}$$

It is helpful to think of G_f and c_f together as a residual flow network.

Now, let's say we have some flow f on our flow network G . Observe that if there exists some unbroken path² from s to t through the residual graph G_f , then we can still add some flow value to f along each edge on the path. In this way we can **augment** f into some new flow with a higher flow value - these paths are therefore called **augmenting paths**.

Let us spend some time to verify that this style of *augmentation* works.

Definition 10 (Augmenting path). *Let f be a flow on the flow network G , and recall the residuals G_f, c_f as previously defined. An **augmenting path** is a path in G_f starting from vertex s and ending at vertex t .*

Lemma 2. *Let f be a flow on the flow network G . If there exists an augmenting path P on G_f , then f is not a maximum flow on G , and in particular, we can explicitly construct a new flow f' , based on f and P , such that $|f'| > |f|$.*

Suppose indeed that a flow f leaves an augmenting path P . Let F be the smallest residual capacity on P ,

$$F = \min_{u \rightarrow v \in P} c_f(u \rightarrow v)$$

We first write down the function $f' : E \rightarrow \mathbb{R}$,

$$f'(u \rightarrow v) = \begin{cases} f(u \rightarrow v) + F & \text{if } u \rightarrow v \in P \\ f(u \rightarrow v) - F & \text{if } v \rightarrow u \in P \\ f(u \rightarrow v) & \text{otherwise} \end{cases}$$

Note that this f' is only defined on the original edges $u \rightarrow v \in E$.

At every vertex $v \in V \setminus \{s, t\}$, either v is disjoint from P , or v occurs somewhere in the middle of P and so is incident on exactly two edges $u \rightarrow v$ and $v \rightarrow w$ in P . In the latter case, $u \rightarrow v$ always increases the net flow at v by F (regardless of whether $u \rightarrow v \in E$, or $v \rightarrow u \in E$), while $v \rightarrow w$ always decreases the net flow at v by F .

Either way,

$$\begin{aligned} \Delta f'(v) &= \sum_{u \in V} f'(u \rightarrow v) - \sum_{w \in V} f'(v \rightarrow w) \\ &= \sum_{u \in V} f(u \rightarrow v) - \sum_{w \in V} f(v \rightarrow w) = \Delta f(v) = 0 \end{aligned}$$

so f' satisfies the *conservation constraint*.

It remains to prove f is feasible. At every edge $u \rightarrow v \in E$, there are three cases:

1. ($u \rightarrow v \in P$). Then we have

$$f'(u \rightarrow v) = f(u \rightarrow v) + F$$

²Recall the definition that a **path** has no repeated vertices, and consequently no repeated edges.

$$\begin{aligned}
&\leq f(u \rightarrow v) + c_f(u \rightarrow v) \\
&= f(u \rightarrow v) + c(u \rightarrow v) - f(u \rightarrow v) = c(u \rightarrow v)
\end{aligned}$$

It is also clear that $f'(u \rightarrow v) \geq 0$.

2. ($v \rightarrow u \in P$). Then we have

$$\begin{aligned}
f'(u \rightarrow v) &= f(u \rightarrow v) - F \\
&\geq f(u \rightarrow v) - c_f(u \rightarrow v) \\
&= f(u \rightarrow v) - f(u \rightarrow v) = 0
\end{aligned}$$

It is also clear that $f'(u \rightarrow v) \leq f(u \rightarrow v) \leq c(u \rightarrow v)$.

3. (Neither). Then we continue to have

$$0 \leq f'(u \rightarrow v) = f(u \rightarrow v) \leq c(u \rightarrow v)$$

In all three, we have

$$0 \leq f'(u \rightarrow v) \leq c(u \rightarrow v)$$

and therefore f' is indeed a (feasible) flow, as we have constructed.

So, we have proven that an augmenting path can be used to augment a flow into one with higher flow value. We complete the proof by showing that any non-maximum flow can always be augmented this way, or in other words any flow that cannot be augmented must already be maximum.

Lemma 3. *Let f be a flow on the flow network G . If there exists **no** augmenting path on G_f , then f is a maximum flow on G , and furthermore we can explicitly construct a cut (S, T) such that $|f| = ||S, T||$, and f saturates every edge from S to T and avoids every edge from T to S .*

Suppose indeed that a flow f has no augmenting path in G_f . We construct a cut on G - let S be the set of vertices reachable from s by traversing on G_f , and let $T = V \setminus S$ be the remaining vertices. Clearly $s \in S$, and on the other hand $t \in T$ since there was no augmenting path from s to t , meaning t was unreachable from s . Thus (S, T) is a valid cut.

However, think back additionally to the proof of the flow-cut inequality. We had

$$\begin{aligned}
|f| &= \sum_{v \in S} \sum_{w \in T} f(v \rightarrow w) - \sum_{v \in S} \sum_{u \in T} f(u \rightarrow v) \\
&\leq \sum_{v \in S} \sum_{w \in T} c(v \rightarrow w) - \sum_{v \in S} \sum_{u \in T} 0 = ||S, T||
\end{aligned}$$

with equality if and only if all such $f(v \rightarrow w) = c(v \rightarrow w)$, and all such $f(u \rightarrow v) = 0$. This is exactly the case for our given flow f and cut (S, T) :

For all $u \in S, v \in T$, we must have $u \rightarrow v \notin G_f$, since s can reach u but not v . This implies $c_f(u \rightarrow v) = 0$, after which we have three cases,

1. $(u \rightarrow v \in E)$. Then from the definition of c_f ,

$$\begin{aligned} c_f(u \rightarrow v) &= c(u \rightarrow v) - f(u \rightarrow v) = 0 \\ \Rightarrow f(u \rightarrow v) &= c(u \rightarrow v) \end{aligned}$$

as well as $f(v \rightarrow u) = c(v \rightarrow u) = 0$, since $v \rightarrow u \notin E$.

2. $(v \rightarrow u \in E)$. Then from the definition of c_f ,

$$c_f(u \rightarrow v) = f(v \rightarrow u) = 0$$

as well as $f(u \rightarrow v) = c(u \rightarrow v) = 0$, since $u \rightarrow v \notin E$.

3. (Neither). Then $f(u \rightarrow v) = c(u \rightarrow v) = 0$ since $u \rightarrow v \notin E$, and $f(v \rightarrow u) = c(v \rightarrow u) = 0$ since $v \rightarrow u \notin E$.

In all three, every $f(u \rightarrow v) = c(u \rightarrow v)$ is saturated, while every $f(v \rightarrow u)$ is avoided. Therefore $|f| = ||S, T||$ by the flow-cut inequality lemma, so f is a maximum flow.

We can henceforth wrap up the proof. Let $G = (V, E)$ with capacity c be a flow network, and fix any $s, t \in V$. There exists at least one flow in G - namely, the empty flow $f = 0$ always exists. Since there exists at least one flow in G , there exists a maximum flow. Denote this maximum flow as f .

As f is a maximum flow, by the above lemma there must exist no augmenting path on its residual graph G_f . Thus there furthermore exists a cut (S, T) of G such that $|f| = ||S, T||$, and f saturates every edge from S to T and avoids every edge from T to S .

□

5 Ford-Fulkerson Algorithm

The Max-flow Min-cut Theorem grants us the existence of a maximum flow and dual minimum cut, but more importantly its proof gives us the tools to explicitly compute a maximum flow & minimum cut on any flow network of choice. Let us briefly restate the two Augmenting Path lemmas.

- If a flow f has an augmenting path P , it is not a maximum flow but can be augmented using P to compute a new flow f' with $|f'| > |f|$.
- If a flow f has no augmenting paths, it is a maximum flow.

This immediately lends itself to an algorithm outline:

1. Start with an arbitrary flow f (e.g. the zero flow),
2. If f has an augmenting path P , augment it with P and set f to be the augmented flow,

3. If f still has an augmenting path, return to the previous step and repeat,
4. Otherwise f no longer has an augmenting path and thus is a maximum flow.

The Ford-Fulkerson algorithm is a fairly direct implementation of this outline.

Definition 11 (Ford-Fulkerson algorithm). *The inputs of the Ford-Fulkerson algorithm are a flow network $G = (V, E)$ with capacity c , and source vertex s & sink vertex t . The algorithm outputs a maximum flow f for the flow network.*

1. Let $f(u \rightarrow v) \leftarrow 0, \forall u \rightarrow v \in E$.
2. While s can reach t in G_f (after computing the new G_f):
 - (a) Use BFS or DFS to find an augmenting path P , from s to t in G_f .
 - (b) $F \leftarrow \min_{u \rightarrow v \in P} c_f(u \rightarrow v)$
 - (c) For all $u \rightarrow v \in P$, update its flow value:
 - i. $f(u \rightarrow v) \leftarrow f(u \rightarrow v) + F$,
 - ii. $f(v \rightarrow u) \leftarrow f(v \rightarrow u) - F$
3. Output f .

This algorithm runs in $O(E|f|)$ time based on the definition above - at most $|f|$ augmenting paths will occur, each incrementing the flow value by $+1$, while it takes $O(E)$ time to compute G_f, P, F , and update f for each augmenting path.

To illustrate that the $O(E|f|)$ bound is tight, consider the following example:

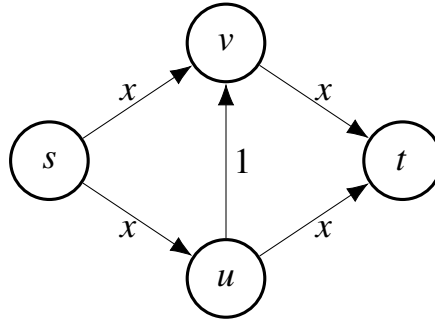


Figure 6: The original flow network, G .

Here we first find an augmenting path $s \rightarrow u \rightarrow v \rightarrow t$, with width $F = 1$. Pushing $F = 1$ unit of flow across the augmenting path, we get the residual graph

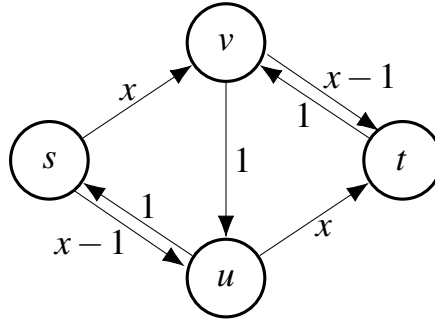


Figure 7: The residual graph G_f , after the first augmenting path push.

The next augmenting path found may then be $s \rightarrow v \rightarrow u \rightarrow t$, with width $F = 1$. Pushing $F = 1$ unit of flow across this augmenting path, we get

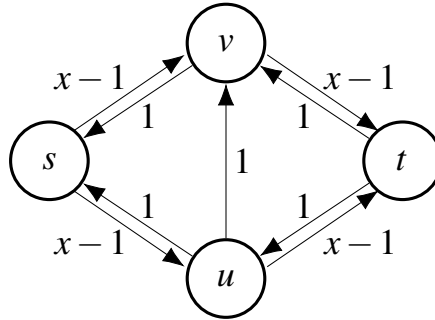


Figure 8: The residual graph G_f , after the second augmenting path push.

At this point, the next augmenting path found may be $s \rightarrow u \rightarrow v \rightarrow t$. It is now apparent that this process could repeat up to x times, before finally producing a maximum flow of flow value $2x$.

This forms a sketch of how the Ford-Fulkerson algorithm could go through $O(|f|)$ augmenting paths before terminating - in a larger graph, each such augmenting path would invoke $O(E)$ of overhead in recomputing G_f , performing BFS/DFS etc - for a total of $O(E|f|)$ time.

Network Flow Problem (II)

Lecturer: Divesh Aggarwal

October 10, 2024

Scribe: Tan Yi Long, Russell Tan Kai Min, Timothy Wan Kai Yang, Albin Liljefors, Chieu Le Heng

Abstract

We have seen *Ford-Fulkerson Algorithm 1* last week to obtain the Max-Flow from a given input flow network, with a time complexity of $O(|E| \cdot f^*)$, where f^* is the max flow. In this week's lecture, we introduce the *Flow Decomposition Theorem 2*. Next, we discuss how different heuristics can be used to obtain an algorithm with a time complexity faster with respect to f^* , specifically *Edmonds-Karp Fat Pipes Algorithm 2* and *Dinitz's Algorithm 3*.

1 Recap

Given a directed graph, $G = (V, E)$, where each edge is assigned a capacity, $c : E \rightarrow \mathbb{R}_{\geq 0}$, a (s, t) -flow $f : E \rightarrow \mathbb{R}$ is a flow that satisfies the *conservation constraint*. For all vertices except the source, s , and terminal, t , the sum of the flow going into the vertex is equal to the sum of the flow going out of the vertex.

$$\forall v \in V \setminus \{s, t\} \quad \sum_{u \in V} f(u \rightarrow v) = \sum_{w \in V} f(v \rightarrow w)$$

A flow is *feasible* if for every edge, the amount of flow sent through the edge is non-negative and at most the edge's capacity.

$$\forall u \rightarrow v \quad 0 \leq f(u \rightarrow v) \leq c(u \rightarrow v)$$

We were introduced *Ford-Fulkerson Algorithm 1* which has a running time of $O(|E| \cdot f^*)$ where f^* is the value of the max-flow. However, f^* can be large. In the worst case, as shown in Figure 1, Ford-Fulkerson Algorithm continuously selects an augmenting path that is not optimal in each iteration.

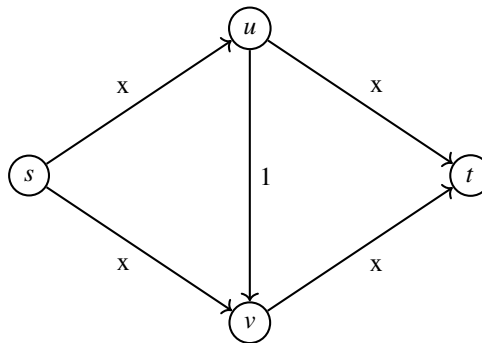


Figure 1: An adversarial example that can cause the Ford-Fulkerson algorithm to work poorly.

Here s, t are the source and sink nodes respectively, whereas the other nodes are just intermediate nodes. We can see that the Ford-Fulkerson algorithm actually takes $O(x)$ time here in the worst case. The worst case here refers to the poor choice of augmenting paths in the residual graph. One can see this by considering the 2 augmenting paths:

Algorithm 1 Ford-Fulkerson Algorithm

Inputs: $G = (V, E)$, a flow network where $\forall e \in E \ c_e \in \mathbb{R}_{\geq 0}$, s , a source vertex, t , a terminal vertex

Output: f , the max-flow of the input graph

```
1: Let  $G_f \leftarrow G$  and  $\forall u \rightarrow v \ f(u \rightarrow v) \leftarrow 0$ 
2: while  $\exists$  a path from  $s$  to  $t$  in  $G_f$  do
3:   Use BFS or DFS to obtain an augmenting path,  $P$ , from  $s$  to  $t$  in  $G_f$ 
4:    $F \leftarrow \min_{u \rightarrow v \in P} c_{u \rightarrow v}$ 
5:   for  $u \rightarrow v \in P$  do
6:      $f(u \rightarrow v) \leftarrow f(u \rightarrow v) + F$ 
7:      $f(v \rightarrow u) \leftarrow f(v \rightarrow u) - F$ 
8:   end for
9: end while
10: return  $f$ 
```

$$P_1 = S \rightarrow u \rightarrow v \rightarrow T,$$

$$P_2 = S \rightarrow v \rightarrow u \rightarrow T,$$

and then alternate the choice of paths (i.e. $P_1, P_2, P_1, P_2, \dots$). Notice that after each path, we only manage to send 1 additional flow across from S to T . The maximum flow for this channel is in fact, $2x$ units, which then implies that given a poor choice of augmenting paths, the Ford-Fulkerson algorithm would take $O(x)$ time.

By changing how we select an augmenting path, we can obtain the max-flow more efficiently, namely by choosing an augmenting path that either maximizes the minimum capacity along the path, or minimizes the number of edges along the path.

2 Flow Decomposition Theorem

Before discussing how to use the two heuristics, we will introduce the idea of *flow decomposition* and the *Flow Decomposition Theorem*.

2.1 Flow Decomposition

Addition of Flows. Fix any two (s, t) -flows f, g . Fix $\alpha, \beta \in \mathbb{R}$. Define a new flow,

$$h(u \rightarrow v) = \alpha \cdot f(u \rightarrow v) + \beta \cdot g(u \rightarrow v)$$

Proof. Applying the definition of a flow to each term, we see that h also satisfies the conservation constraint.

$$\forall v \in V \setminus \{s, t\} \ (\alpha + \beta) \sum_{u \in V} f(u \rightarrow v) = (\alpha + \beta) \sum_{w \in V} f(v \rightarrow w)$$

□

Constructing a Flow from a Path. Suppose P is a path from s to t , $P : s \rightarrow v_0 \rightarrow \dots \rightarrow t$, construct a new

flow f_p by assigning flow values to the edges along the path as follows,

$$\forall u \rightarrow v \in E \quad f_p(u \rightarrow v) = \begin{cases} 1 & u \rightarrow v \in P \\ -1 & v \rightarrow u \in P \\ 0 & \text{otherwise} \end{cases}$$

Proof. Consider a path from s to t and a vertex v along this path. By the definition of a path, there are no cycles, therefore, there is only one edge coming into the path and one edge leaving the path. The value of the flows entering and leaving each vertex is set to $|1|$. Therefore, the conservation constraint is satisfied. Furthermore, the value of the flow f_p is 1. $\square$

Constructing a Flow from a Cycle. Suppose C is a cycle, $C : s \rightarrow v_0 \rightarrow t \rightarrow s$, similarly assign flow values to the edges along the cycle.

$$f_C(u \rightarrow v) = \begin{cases} 1 & u \rightarrow v \in C \\ -1 & v \rightarrow u \in C \\ 0 & \text{otherwise} \end{cases}$$

f_C is a flow, for the same reason as f_p . However, since we have built the flow from a cycle, at the source and terminal vertices, the inflow and outflow through them are equal. Therefore, the value of the flow f_C is 0.

Definition 1 (Circulation). A flow, f , with 0 value, where the inflow and outflow of every vertex, including s and t , are equal, is a circulation.

f_C is also a circulation 1.

2.2 Flow Decomposition Theorem

Theorem 2 (Flow Decomposition Theorem). Every non-negative (s,t) -flow can be written as a positive linear combination (the coefficients should not be negative) of directed (s,t) path flows and directed cycle flows. An edge appears in the flow if and only if the amount of flow through the edge is positive. The sum of the number of paths and the number of cycles is at most the number of edges.

The Flow Decomposition Theorem can be described with a few mathematical relations.

$$\begin{aligned} h(u \rightarrow v) &= \alpha \cdot f(u \rightarrow v) + \beta \cdot g(u \rightarrow v) \quad \alpha, \beta > 0 \\ u \rightarrow v \in h(E) &\Leftrightarrow h(u \rightarrow v) > 0 \\ \# \text{ of Paths} + \# \text{ of Cycles} &\leq |E| \end{aligned}$$

First, we shall prove the statement about the sum of the number of paths and number of cycles is at most the number of edges for the special case when the flow is a circulation 1.

Proof. Suppose we have a circulation, C . We can decompose C into $\#f - 1$ cycles, where $\#f$ is the number of edges with positive flow in F . We can show this by induction on $\#f$.

Base case. $\#f = 2$. We can only have a single cycle between 2 vertices. Decompose the flow into a single cycle via f_C . We have obtained $\#f - 1 = 2 - 1 = 1$ cycle.

Induction Step. Suppose that $\#f \leq k - 1$ holds, we now want to show that $\#f = k$ holds. Since there are finitely many vertices and C is a circulation, while travelling from s in C , we will encounter v_j that leads to v_i , which was previously seen along the traversal. Identify the cycle from v_i to v_j as $v_i \rightarrow v_j$.

Define the *bottleneck value* on this cycle as the minimum value of flow on this cycle. Let F be the bottleneck value on this cycle. Define a new flow $f'(u \rightarrow v) = f(u \rightarrow v) - F \cdot f_C(u \rightarrow v)$. Since we have removed F units of flow from the cycle, and F corresponds to the bottleneck value, there must be at least one edge that corresponds to F . Therefore, $\#f' \leq \#f - 1 = k - 1$, we have at least removed the edge that corresponds to the bottleneck value F .

By the induction hypothesis, f' holds and can therefore be decomposed into $\#f' - 1 \leq \#f - 2$ cycles. Combining with the cycle found, we have found at most $\#f - 1$ cycles.

To prove the result for the general case, we can convert the general case to the circulation flow by simply send a flow from t to s of value $|f|$. Since we have shown that the number of cycles is at most the number of edges for the circulation, we can first obtain the cycles in the circulation. Next, simply remove the added edge from t to s in all the cycles in which the edge appears, which converts the cycles to the paths. Since we have not added any new paths or cycles, and only transformed some cycles to paths, the result holds for the general case. $\square$

3 Edmonds-Karp Algorithm: Fat Pipes

To avoid choosing inefficient augmenting paths, Edmonds-Karp Fat Pipes Algorithm 2 chooses the augmenting path in G_f with the largest bottleneck value. We can obtain this augmenting path by using modified Dijkstra's algorithm that has a time complexity of $O(|E|\log|V|)$ per execution.

Algorithm 2 Edmonds-Karp Fat Pipes Algorithm

Inputs: $G = (V, E)$, a flow network where $\forall e \in E \ c_e \in \mathbb{R}_{\geq 0}$, s , a source vertex, t , a terminal vertex

Output: f , the max-flow of the input graph

```

1: Let  $G_f \leftarrow G$  and  $\forall u \rightarrow v \ f(u \rightarrow v) \leftarrow 0$ 
2: while  $\exists$  a path from  $s$  to  $t$  in  $G_f$  do
3:   Use Dijkstra's Algorithm to obtain an augmenting path,  $P$ , from  $s$  to  $t$  in  $G_f$ , with the largest bottle-
     neck value
4:    $F \leftarrow$  bottleneck value along  $P$ 
5:   for  $u \rightarrow v \in P$  do
6:      $f(u \rightarrow v) \leftarrow f(u \rightarrow v) + F$ 
7:      $f(v \rightarrow u) \leftarrow f(v \rightarrow u) - F$ 
8:   end for
9: end while
10: return  $f$ 

```

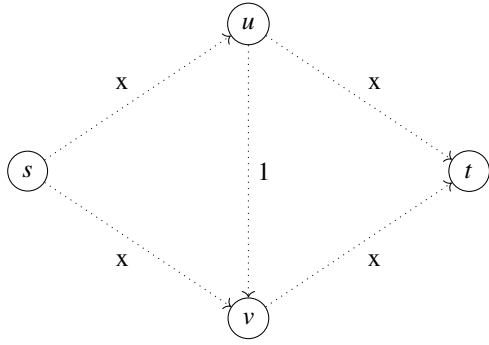
We will make an assumption that all capacities are integers, $c : E \rightarrow \mathbb{Z}$. We can perform a relaxation from rationals to integers by multiplying all the capacities by the lowest common multiple of the denominators of the rational capacities.

Let the max flow be $|f^*|$. The value of f' for the path chosen by the algorithm in the current residual graph G_f . Form a cut using the value of f' , putting all the edges with capacity greater than f' in S and all other edges in T . The capacity of this (S, T) -cut is at most $|f'| |E|$. From the Max Cut-Min Flow theorem presented last week, $|f^*| \leq ||S, T||$, where $||S, T||$ is the capacity of the (S, T) cut. Putting the equations together, $|f^*| \leq |f'| |E|$, which implies $|f'| \geq \frac{|f^*|}{|E|}$.

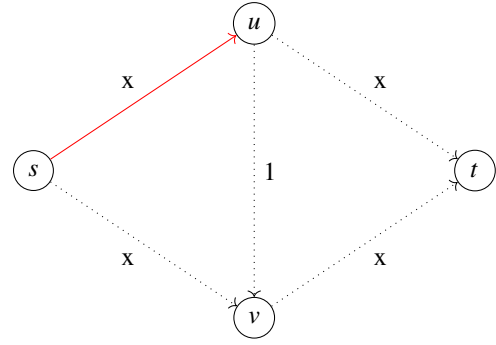
After performing one augmentation, the amount of flow remaining is $\leq |f^*| - \frac{|f^*|}{|E|} = |f^*| \cdot (1 - \frac{1}{|E|}) \leq |f^*| \cdot (1 - \frac{1}{2|E|})$. After the second iteration, the flow remaining is $\leq |f^*| \cdot (1 - \frac{1}{2|E|}) \cdot (1 - \frac{1}{2|E|})$. After k iterations, the flow remaining is $\leq |f^*| \cdot (1 - \frac{1}{2|E|})^k$. Suppose $k = 2|E| \cdot \ln |f^*|$, then using the inequality $1 + x \leq e^x$,

$$\begin{aligned} |f^*| \cdot (1 - \frac{1}{2|E|})^k &= |f^*| \cdot (1 - \frac{1}{2|E|})^{2|E| \cdot \ln |f^*|} \\ &< |f^*| \exp\{-\ln |f^*|\} \\ &= 1 \end{aligned}$$

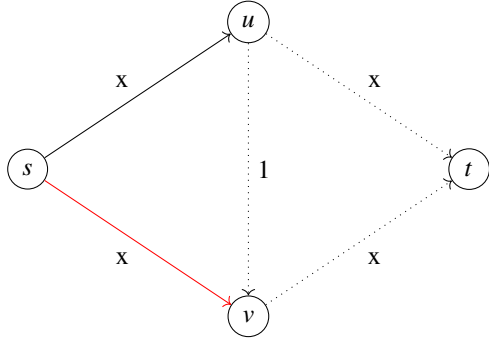
Therefore, the complexity is $O((|E| \log |V|) \cdot (2|E| \ln |f^*|)) = O(|E|^2 \log |V| \log |f^*|)$.



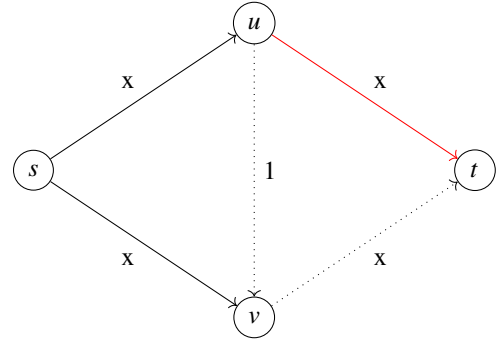
(a) Initial Graph: We start selecting from the neighbors of s



(b) Vertex u is selected to be extended to s



(c) We pick v to be the next vertex to be extended to s



(d) We now set focus on vertex u , and we proceed to pick the edge with the largest capacity. Now, all the vertices are added, hence we terminate

Figure 2: **Edmonds-Karp Algorithm for the example in Figure 1:** Execution of the tree building process. Dotted edges signify that the edge is not selected during that stage, whereas past and current selected edges will be represented by a solid black and red line respectively.

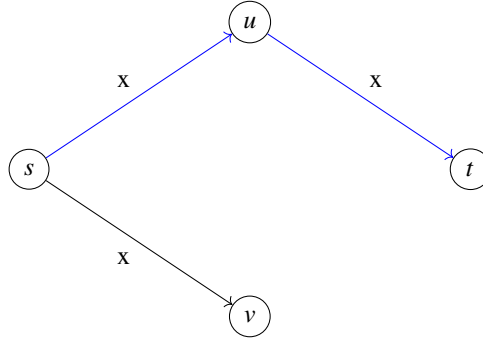


Figure 3: **Edmonds-Karp Algorithm for the example in Figure 1:** The path from s to t (in blue) will be the selected augmenting path from the constructed tree

4 Dinitz Algorithm

To choose a more efficient augmenting path, Dinitz Algorithm 3 chooses the augmenting path in G_f with the smallest number of edges. Edmonds and Karp also proposed a similar idea independently, which we differentiate from Algorithm 2 as Edmonds-Karp Short Pipes Algorithm.

Algorithm 3 Dinitz Algorithm

Inputs: $G = (V, E)$, a flow network where $\forall e \in E \ c_e \in \mathbb{R}_{\geq 0}$, s , a source vertex, t , a terminal vertex

Output: f , the max-flow of the input graph

- 1: Let $G_f \leftarrow G$ and $\forall u \rightarrow v \ f(u \rightarrow v) \leftarrow 0$
 - 2: **while** $\exists$ a path from s to t in G_f **do**
 - 3: Use BFS to obtain an augmenting path, P , from s to t in G_f , with the least number of vertices along the path
 - 4: $F \leftarrow \min_{u \rightarrow v \in P} c_{u \rightarrow v}$
 - 5: **for** $u \rightarrow v \in P$ **do**
 - 6: $f(u \rightarrow v) \leftarrow f(u \rightarrow v) + F$
 - 7: $f(v \rightarrow u) \leftarrow f(v \rightarrow u) - F$
 - 8: **end for**
 - 9: **end while**
 - 10: **return** f
-

Definition 3 (Level of a Vertex). Let $level_i(v)$ be the unweighted, shortest distance from s to v in G_i , where $G_i \equiv G_{f_i}$, where f_i is the flow after i iterations.

Lemma 1 (Levels cannot decrease after each Iteration). $\forall i, v \ level_i(v) \geq level_{i-1}(v)$

Proof. We shall prove the lemma by induction on $level_i(v)$.

Base Case. $level_i(v) = 0$ means the v is s . It remains unchanged regardless of how many iterations i occur.

Induction Case. Suppose that $level_i(v) \leq k - 1$ holds true, we now want to show that $level_i(v) = k$ holds true. In G_i , the shortest path from s to v where $v \neq s$ is

$$s \rightarrow v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{k-1} = u \rightarrow v_k = v$$

Because we have found the shortest path, $level_i(u) = k - 1$. If $level_i(u) < k - 1$, we would have taken a different path to reach v from s .

By Induction hypothesis, $\text{level}_{i-1}(u) \leq k-1$. We now need to consider two cases, one where $u \rightarrow v \in G_{i-1}$ and the other where $u \rightarrow v \notin G_{i-1}$.

Case 1. $u \rightarrow v$ is present in G_{i-1} . If so, then we can reach v by using the shortest path to u and then using this $u \rightarrow v$ edge which implies that $\text{level}_{i-1}(v) \leq \text{level}_{i-1}(u) + 1 \leq k-1+1 = k = \text{level}_i(v)$. Therefore, $\text{level}_i(v) \geq \text{level}_{i-1}(v)$

Case 2. $u \rightarrow v \notin G_{i-1}$. This implies that $v \rightarrow u$ is present in G_{i-1} and is on the shortest path from s to t hence for any vertex on this path the distance from s on this path is the level of that vertex in $G_{i-1} \implies \text{level}_{i-1}(v) \leq \text{level}_{i-1}(u) - 1 \leq k-2$. It is clear that $\text{level}_{i-1}(u) - 1 \leq \text{level}_{i-1}(u) + 1$, which we can then follow the same steps described in *Case 1* to derive that $\text{level}_{i-1}(v) \leq \text{level}_i(v)$. $\square$

Lemma 2 (Maximum Number of Edge Disappearances). *During the execution of Dinitz Algorithm, any edge disappears $\leq \frac{|V|}{2}$ times.*

Proof. Let $u \rightarrow v \in G_i$, $u \rightarrow v \notin G_{i+1}$, $u \rightarrow v \notin G_j$, $u \rightarrow v \in G_{j+1}$. In other words, $u \rightarrow v$ is present in G_i , but disappears in G_{i+1} . It then reappears in G_{j+1} , where $i < j$.

In G_i , there was an augmenting path that included $u \rightarrow v$, $\text{level}_i(u) = \text{level}_i(v) - 1$. It was selected and removed in G_{i+1} . In G_j , there is an augmenting path that goes from $s \rightarrow \dots \rightarrow v \rightarrow u$, meaning $v \rightarrow u$ was inserted. $\text{level}_j(v) = \text{level}_j(u) - 1$

By Lemma 1,

$$\text{level}_j(u) = \text{level}_j(v) + 1 \geq \text{level}_i(v) + 1 = \text{level}_i(u) + 2$$

This means that since the disappearance of $u \rightarrow v$, the distance from s to u increased by at least 2. The maximum distance is $|V|$, so the number of times an edge disappears $\leq \frac{|V|}{2}$ times. $\square$

Each edge can disappear at most $\frac{|V|}{2}$ times, but at each iteration we are deleting one edge. Therefore, the number of iterations is bounded by $\frac{|V|}{2}$. In each iteration, we are running BFS to obtain the shortest path in terms of number of edges from s to t , which takes $O(E)$ time. Therefore, the time complexity of Dinitz Algorithm is $O(|E| \cdot \frac{|V|}{2} \cdot |E|) = O(|E|^2 \cdot |V|)$.

More recently, there have been advances in obtaining faster max-flow algorithms. Orlin and Gong, 2019 proposed an algorithm with a time complexity of $O(|V| \cdot |E|)$. Chen et al., 2022 proposed an algorithm with time complexity $O(|E|^{1+O(1)} \cdot \log |f^*|)$.

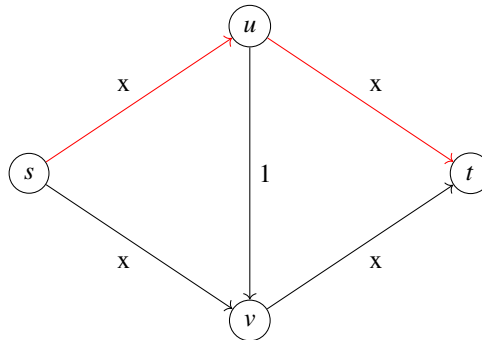


Figure 4: **Dinitz Algorithm for the example in Figure 1:** The path from s to t (in red) will be a selected path using BFS.

5 Applications

In next week's lecture, we will look at the application of Max Flow / Min Cut Algorithms.

Edge Disjoint Paths. Given a graph, $G = (V, E)$, find the maximum number of edge disjoint paths from a source vertex, s , to a terminal vertex, t .

Vertex Disjoint Paths. Given a graph, $G = (V, E)$, from a source vertex, s , to a terminal vertex, t , find the maximum number of paths that do not share any vertices, excluding s and t .

References

- [Che+22] L. Chen et al. "Maximum flow and minimum-cost flow in almost-linear time". In: *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE. 2022, pp. 612–623.
- [OG19] J. B. Orlin and X.-Y. Gong. *A Fast Max Flow Algorithm*. 2019. arXiv: 1910.04848 [cs.DS]. URL: <https://arxiv.org/abs/1910.04848>.

Network Flow Problem (III)

*Lecturer: Divesh Aggarwal**October 17, 2024**Scribe: Ong Zheng Long, Kenny Chon Jin Yong, Nicholas Patrick, Wang Minhong, Koh Shang Hui,
Matthew Tay, Jonathan Nicolas Frohne***Abstract**

In this lecture, we discuss several problems that can be reduced to an instance of a Max-Flow problem, which can be efficiently solved in $O(|F||E|)$ time using the algorithm by Ford and Fulkerson, 1956 or in $O(|V||E|)$ time using the algorithm by Orlin, 2013. These problems include counting the number of edge- or vertex-disjoint paths between a fixed pair of vertices, finding the maximum flow of a graph with vertex capacities, bipartite matching, disjoint path cover of acyclic-directed graphs, and minimizing the number of teaching staff required.

1 Edge Disjoint Paths

Suppose we have a directed graph $G = (V, E)$, and we want to find the maximum number of edge-disjoint paths from vertex s to vertex t .

Definition 1 (Edge disjoint paths). *Two paths $\mathcal{P}_1, \mathcal{P}_2$ are edge disjoint if the two paths do not contain a common edge, i.e., $\forall e \in \mathcal{P}_1, e \notin \mathcal{P}_2$ and vice versa, where $\mathcal{P}_i$ are paths containing edges $\{(v_1, v_2), (v_2, v_3), \dots, (v_{n-1}, v_n)\}$.*

Consider the reduction where we assign a capacity of 1 to each edge and compute the max-flow of this graph. We claim that the max-flow value represents the maximum number of edge-disjoint paths and that the edges with a flow value of 1 in the max-flow are part of these edge-disjoint paths. Furthermore, we can isolate each individual path by performing a depth-first search (DFS) to find a valid path from s to t , removing it, and then recursing on the remaining edges.

Proof. Suppose that a graph has k disjoint paths. Then we can send a flow of value 1 through each of the paths, with each edge not exceeding its capacity of 1, since the paths do not share edges. Thus, we can obtain a flow of k .

Now suppose that a graph has a flow with value k . Then by the flow decomposition theorem covered in the previous lecture, we can decompose the flow into paths and cycles. Note that each cycle contributes 0 flow value from vertices s to t , while each path can only contribute a flow of 1, due to the capacities of the edges. So, to obtain a flow value of k , there must be k paths. The paths must be edge-disjoint because if multiple paths share the same edge, to obtain a flow of 1 through each path, the shared edge must have a capacity of at least 2.

We can find valid paths by greedily allocating edges to paths because each edge is only part of one path. $\square$

2 Vertex Capacities

Instead of having the capacities on the edges, what if we have the capacities on the vertices? Suppose we have a directed graph $G = (V, E)$, and a capacity function $c : V \mapsto \mathbb{Z}$. Then, a valid flow $f : E \mapsto \mathbb{Z}$ is defined as $\sum_{u \in V : (u,v) \in E} f((u,v)) = \sum_{w \in V : (v,w) \in E} f((v,w))$, similar to the version with the capacities on the edges, and $\sum_{u : (u,v) \in E} f((u,v)) \leq c(v)$, for all vertices v . The intuition behind the second condition is that the total flow going into a vertex has to be less than or equal to the capacity of the vertex.

Consider the directed graph $G' = (V', E')$, where $\forall v \in V, x_v, y_v \in V'$, $\forall v \in V, (x_v, y_v) \in E'$, and $\forall (u, v) \in E, (y_u, x_v) \in E'$. We then have the edge capacities $c' : E' \mapsto \mathbb{Z}$ defined as follows: $\forall v \in V, c'((x_v, y_v)) = c(v)$, and $c'(e) = \max_v c(v)$ for all other edges.

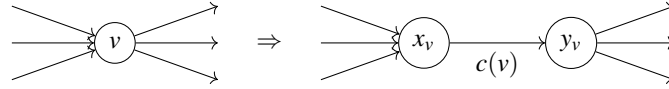


Figure 1: Transformation of the graph

With this construction, we can see that since all inflow = all outflow from a vertex, the inflow is limited by $c(v)$, and a max-flow on edge capacities guarantees that the inflow = outflow. Thus, we can solve it efficiently with the usual max-flow algorithm.

2.1 Vertex Disjoint Paths

Definition 2 (Vertex disjoint paths). Two paths $\mathcal{P}_1 = \{(s, u_1), (u_2, u_3), \dots, (u_n, t)\}$, $\mathcal{P}_2 = \{(s, v_1), (v_2, v_3), \dots, (v_m, t)\}$ are vertex disjoint if all vertices u_i and v_i are distinct and not s or t .

We are given a directed graph $G = (V, E)$ and we need to find the maximum number of vertex disjoint paths from vertices s and t . We can set a vertex capacity of 1 for each vertex in the graph and solve the max-flow problem with vertex capacities as described previously. The proof for this is very similar to the one for edge-disjoint paths.

3 Bipartite Matching

Definition 3 (Bipartite graphs). A bipartite graph $G = (L, R, E)$ has vertices in the sets L and R such that $L \cap R = \emptyset$ and $E \subseteq \{(u, v) : u \in L, v \in R\}$.

Definition 4 (Matching). A matching for a graph is a set of non-adjacent edges in the graph, i.e., none of the edges share a vertex.

We want to find the maximum matching of a bipartite graph $G = (L, R, E)$. The maximum matching contains the maximum number of edges while still remaining a valid matching.

We construct a new directed graph $G' = (V', E')$, where $V' = \{s, t\} \cup L \cup R$, $\forall e \in E, e \in E', \forall u \in L, (s, u) \in E$, and $\forall v \in R, (v, t) \in E$. Let the capacities of all edges in G' be 1. We then run the max-flow algorithm, and all edges with a flow value of 1 and is part of E are part of the matching. We claim that this set of edges is a maximum matching for the original graph G .

Proof. Suppose there exists a matching of size k , then we can construct a flow with value of k by sending 1 flow from s to t through the edges in the matching. This is a valid flow as all vertices are used at most once, so $\forall v$, (s, v) or (v, t) have a flow value of 1 at most.

Suppose there exists a flow $f : E \mapsto \{0, 1\}$ of value k . Let $(v_1, v'_1), (v_2, v'_2), \dots, (v_\ell, v'_\ell) \in E$ be the edges with flow value of 1. Then $\ell = k$ since all flow has to go through some edge in E and each edge in E can only allow 1 flow. Since the capacity of edges (s, u) and (v, t) are 1, each vertex would only be involved in 1 flow and are all distinct, creating a valid matching. $\square$

3.1 Ford-Fulkerson's Algorithm on Bipartite Matching

We now illustrate running the Ford-Fulkerson's algorithm on a bipartite graph to compute the maximum matching. Consider the following transformed graph.

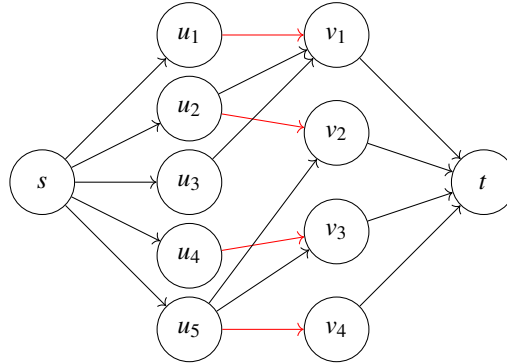


Figure 2: Transformed graph with a maximum matching highlighted

A possible maximum matching would be $\{(u_1, v_1), (u_2, v_2), (u_4, v_3), (u_5, v_4)\}$. Suppose Ford Fulkerson's algorithm chose a bad path of $\{(s, u_2), (u_2, v_1), (v_1, t)\}$ to create the residual graph below.

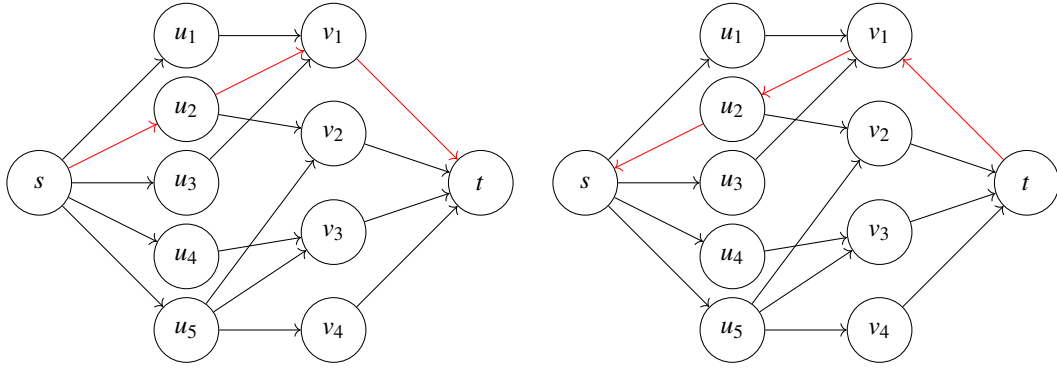


Figure 3: Chosen path and residual graph

Since (u_2, v_1) is not part of a maximum matching, eventually, a path will be chosen which will “undo” choosing that edge and include 2 edges in the maximum matching instead. Suppose later the algorithm chooses the path $\{(s, u_1), (u_1, v_1), (v_1, u_2), (u_2, v_2), (v_2, t)\}$.

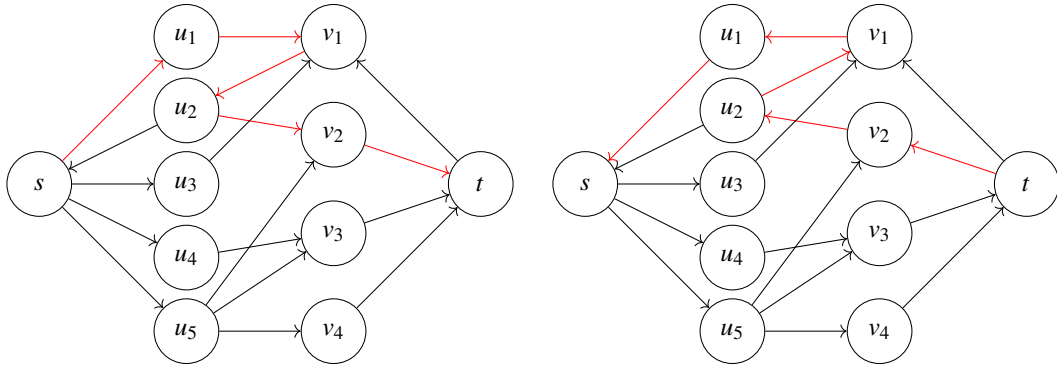


Figure 4: New chosen path and residual graph

Now (u_2, v_1) is removed from the matching and (u_1, v_1) and (u_2, v_2) are added to the matching. A similar process occurs if the next two paths chosen are $\{(s, u_5), (u_5, v_3), (v_3, t)\}$ and $\{(s, u_4), (u_4, v_3), (v_3, u_5), (u_5, v_4), (v_4, t)\}$.

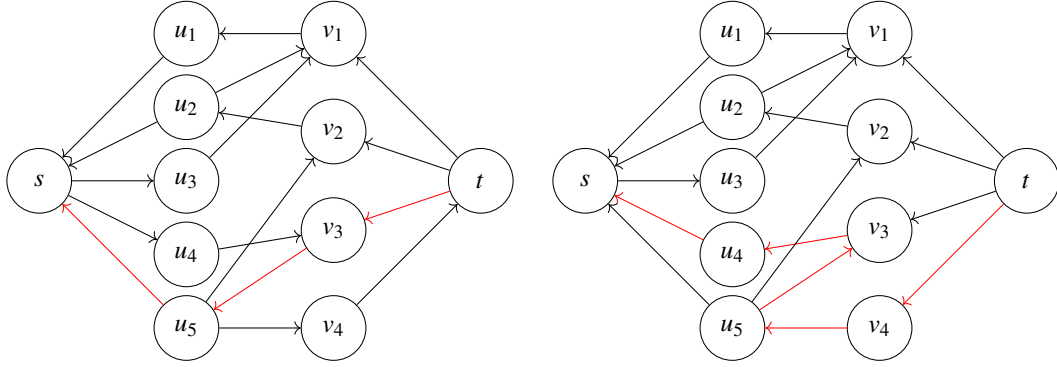


Figure 5: Residual graphs after choosing the two paths

We then arrive at our maximum matching as no other paths from s to t can be found, and the matching contains all the edges going from v_i to u_j , which are $\{(u_1, v_1), (u_2, v_2), (u_4, v_3), (u_5, v_4)\}$

4 Disjoint Path Cover of Acyclic Directed Graphs

Definition 5. A disjoint path cover is a set of paths in a graph such that all vertices in the graph appear in one and only one edge in the set of paths.

We are given a directed acyclic graph $G = (V, E)$ and we want to find the minimum disjoint path cover of the graph, i.e., the number of paths is minimized. Consider the following graph with the minimum disjoint path cover highlighted.

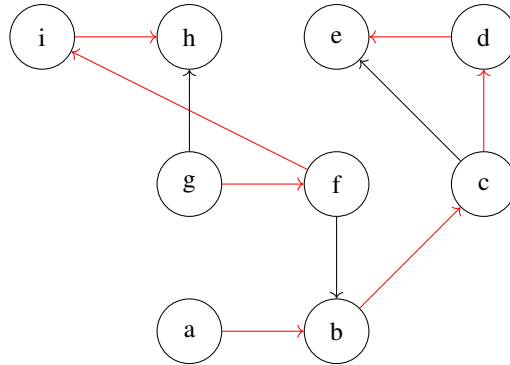


Figure 6: Sample graph for disjoint path covering

We first create a new bipartite graph $G' = (L, R, E')$ where $\forall v \in V, v_{\text{in}} \in L, v_{\text{out}} \in R$, and $\forall (u, v) \in E, (u_{\text{out}}, v_{\text{in}}) \in E'$. In the following diagram, the in-vertices are colored red, while the out-vertices are colored green.

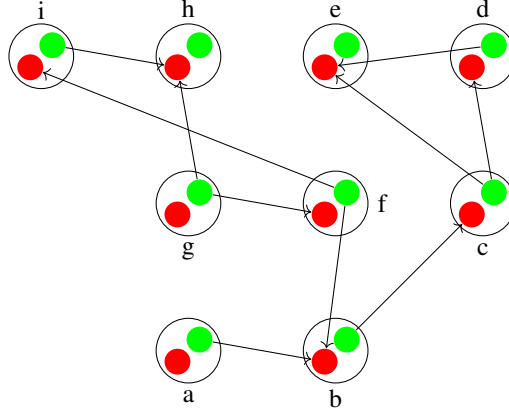


Figure 7: Transformed graph for disjoint path covering

Notice that this transformed graph has 2 pairs of v_{in}, u_{out} which are unmatched, which happens to be the minimum number of disjoint paths required to cover the original graph. We claim that the number of nodes left unmatched in the maximum bipartite matching of the transformed graph is twice the number of paths in the minimum disjoint path cover of the original graph. Furthermore, the edges in the matching belong to some path in the minimum disjoint path cover.

Proof. Suppose there exists a disjoint path cover in G of size k . This implies that there is a matching of size $(n - k)$ in G' , by choosing the edges u_{out}, v_{in} for the matching if (u, v) belongs to some path in the disjoint path cover. Since each vertex v is only used twice in the path cover (once as the destination and once as the source), then v_{in} and v_{out} would not be repeated in the matching. The matching is of size $(n - k)$ as each path leaves 2 vertices unmatched and each matching matches 2 out of $2n$ vertices.

Suppose there exists a matching of size k in G' , then there exists $2(n - k)$ unmatched vertices. Since there are an equal number of in-vertices and out-vertices, and each edge in the matching matches an in-vertex to an out-vertex, the unmatched vertices must contain the same number of in-vertices and out-vertices. Starting from an unmatched in-vertex, we can follow a path of matched vertices until we reach an unmatched out-vertex. This creates a total of $(n - k)$ paths which are disjoint as a matching does not repeat vertices which covers all the vertices in V . $\square$

5 Minimizing the Teaching Staff

Suppose we have a list $C[1, \dots, n]$ of classes, with each class having a start time, end time, and location, and a table $T[1, \dots, n][1, \dots, n]$ containing the time required to go from any location to any other location. Assuming each teaching staff can teach any class, we want to find the minimum number of teaching staff required to teach all the classes.

We can convert this into a graph problem by generating the directed graph $G = (V, E)$ where $V = \{1, 2, \dots, n\}$ corresponding to the classes, and $(i, j) \in E$ if $C[i].end + T[C[i].loc][C[j].loc] \leq C[j].start$, i.e., we add an edge if it is possible to rush from class i to class j in time. We claim that the minimum path cover is the number of teaching staff required, and each path in the cover is a teaching staff's schedule.

Proof. Suppose there exist k disjoint paths in the path cover, then each teaching staff can teach the “path” of classes in order, as we defined the edges such that it is always possible to make it to adjacent classes, resulting in k teachers required.

Now suppose that there exists a schedule involving k teachers. Then each teacher’s schedule is a disjoint path since each class is only taught by 1 teacher, and this is a path cover as all classes are taught by some teaching staff. So we have a disjoint path cover with k paths.

To solve this problem efficiently, we need to show that there are no cycles in the generated graph. Suppose there exists a cycle of vertices $(v_1, v_2), (v_2, v_3), \dots, (v_n, v_1)$. Then this implies that class v_1 ends before class v_2 which ends before $v_3, \dots$, which ends before v_n which ends before v_1 . This is clearly impossible unless backward time travel has been invented which would mean that a single teacher is enough to teach all the classes. $\square$

References

- [FF56] L. R. Ford and D. R. Fulkerson. “Maximal Flow Through a Network”. In: *Canadian Journal of Mathematics* 8 (1956), 399–404. DOI: 10.4153/CJM-1956-045-5.
- [Orl13] J. B. Orlin. “Max flows in $O(nm)$ time, or better”. In: *Proceedings of the Forty-Fifth Annual ACM Symposium on Theory of Computing*. STOC ’13. Palo Alto, California, USA: Association for Computing Machinery, 2013, 765–774. ISBN: 9781450320290. DOI: 10.1145/2488608.2488705. URL: <https://doi.org/10.1145/2488608.2488705>.

Semidefinite Programming

Lecturer: Divesh Aggarwal

October 24, 2024

Scribe: Tian Xiao, Šarūnas Griškus, Stanley Neoh Jia Jun, Lee Yi Hern, Guo Bokun

Abstract

Today, we introduce semidefinite programming (SDP), which extends linear programming to include constraints that cannot be expressed linearly. We begin by using the MaxCut problem as a motivating example to illustrate the need for semidefinite programming. We then formally define key terminologies associated with SDP. Lastly, we discuss the approximation guarantees offered when solving integer linear programming problems using semidefinite relaxation and rounding techniques.

1 MaxCut: A Motivating Example

Recall the MaxCut problem: Given an undirected, weighted graph $G = (V, E)$ with a weight function $w : E \rightarrow \mathbb{R}_{\geq 0}$ that assigns a non-negative weight to each edge, the goal is to partition the vertex set V into two disjoint subsets S and T such that the total weight of edges between S and T , denoted $\sum_{u \in S, v \in T} w(u, v)$, is maximized. For convenience, we denote this total weight as $w(S, T)$.

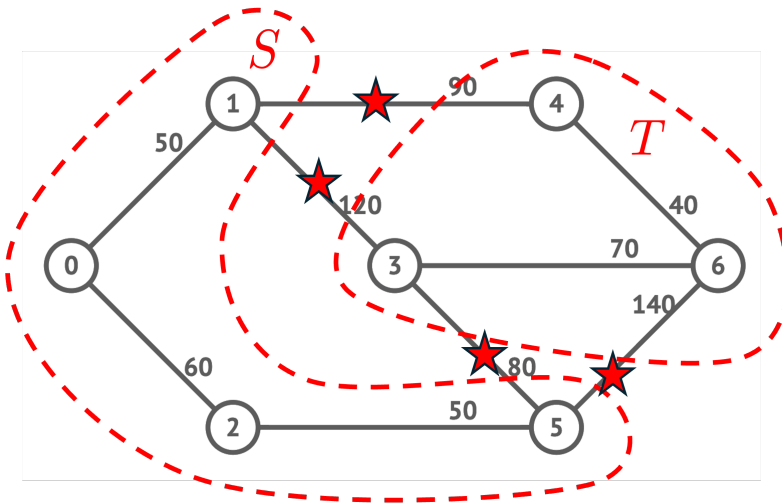


Figure 1: **Illustration of the MaxCut Problem:** The goal is to find a partition of the vertices into two subsets, S and T , such that the total weight of edges connecting nodes in S with nodes in T (marked with stars) is maximized.

In previous lectures, we demonstrated approaches to the MaxCut problem using a simple greedy algorithm and an integer linear programming (ILP) formulation, which we solved through linear programming (LP) relaxation and rounding. However, both methods provide an approximation factor of $\frac{1}{2}$ (i.e., $\frac{|w(S,T)|}{OPT} \geq \frac{1}{2}$). Our goal is to achieve a much better approximation. By using semidefinite programming (SDP) relaxation instead of LP, we can attain an approximation factor of 0.87856, which is the best-known approximation

factor and is believed to be the best possible under the Unique Games Conjecture (Khot, 2010), as improving upon it would also be NP-hard.

1.1 Semidefinite Programming Formulation for MaxCut

Before formally defining SDP, let's first explore the intuition behind using an SDP formulation for the MaxCut problem.

We start by formulating an integer linear programming (ILP) problem, by introducing two sets of variables:

- $\{x_v \in \{0, 1\} : v \in V\}$: $x_v = 0$ represents $v \in S$ and $x_v = 1$ represents $v \in T$;
- $\{y_{uv} \in \{0, 1\} : (u, v) \in E\}$: $y_{uv} = 1$ represents the edge (u, v) is cut.

Our **potential** ILP formulation is as follows (referred to as **potential** because some of the constraints are not truly linear):

$$\max \sum_{(u,v) \in E} y_{uv} w(u, v) \quad (1)$$

$$\text{s.t. } x_u \neq x_v \Rightarrow y_{uv} = 1; \quad (2)$$

$$x_u = x_v \Rightarrow y_{uv} = 0. \quad (3)$$

Constraint (2) specifies that edges connecting nodes in different partitions will be cut, while constraint (3) indicates that edges connecting nodes within the same partition will not be cut. We can combine (2) and (3) into a single constraint:

$$y_{uv} = |x_u - x_v|. \quad (4)$$

Furthermore, since we are maximizing our objective and the weights are non-negative, we want each y_{uv} to be as large as possible. Therefore, we can replace the equality sign above with a $\leq$ sign:

$$y_{uv} \leq |x_u - x_v|. \quad (5)$$

Unfortunately, we find that constraint (5) is not linear and cannot be expressed as a set of linear constraints either. On the other hand, if the problem were a MinCut problem, the constraint would become $y_{uv} \geq |x_u - x_v|$, which can be written as two linear constraints: $y_{uv} \geq x_u - x_v$ and $y_{uv} \geq x_v - x_u$.

To address this, Delorme and Poljak (Delorme and Poljak, 1993a; Delorme and Poljak, 1993b) propose an alternative approach. Instead of using values from $\{0, 1\}$, each x_v and y_{uv} is assigned a value from $\{+1, -1\}$. Specifically, $x_v = -1$ indicates $v \in S$, $x_v = 1$ indicates $v \in T$, and $y_{uv} = -1$ indicates that the edge (u, v) is cut. The problem then becomes:

$$\max \sum_{(u,v) \in V \times V} \left(\frac{1}{2} - \frac{1}{2} y_{uv} \right) w(u, v) \quad (6)$$

$$\text{s.t. } \forall u \in V, v \in V \quad [y_{uv} = x_u x_v]. \quad (*)$$

Note that we extend the feasible region of (u, v) from E to $V \times V$. For $(u, v) \notin E$, we can simply set $w(u, v) = 0$ to eliminate its impact. Constraint (*) has two implications:

$$\forall u \in V, v \in V \quad [y_{uv} = y_{vu}]; \quad (7)$$

$$\forall v \in V \quad [y_{vv} = 1]. \quad (8)$$

Eq. (7) holds because $y_{uv} = x_u x_v = x_v x_u = y_{vu}$, and equation (8) holds because $y_{vv} = x_v x_v = (\pm 1)^2 = 1$. We observe that the maximization objective (6), together with constraints (7) and (8), forms a valid linear programming (LP) problem, which can be solved. We refer to this LP problem as LP. Therefore, if we can find a solution to LP that satisfies constraint (*), our goal is achieved.

Suppose we have an LP solver $\mathcal{P}$ for the LP problem described above, which returns values of y_{uv} for all $(u, v) \in V \times V$. Clearly, these y_{uv} values will satisfy constraints (7) and (8), as they are part of the LP constraints. We are interested in determining what additional conditions y_{uv} must satisfy to fulfill constraint (*).

- $\sum_{(u,v) \in V \times V} y_{uv} \geq 0$: We observe that $\sum_{(u,v) \in V \times V} y_{uv} = \sum_{(u,v) \in V \times V} x_u x_v = \sum_{u \in V} x_u \sum_{v \in V} x_v = (\sum_{v \in V} x_v)^2 \geq 0$, as we can expand $\sum_{u \in V} x_u \sum_{v \in V} x_v$ using the distributive law. For example, if a graph has two vertices and the LP solver $\mathcal{P}$ provides $y_{11} = y_{22} = 1$ and $y_{12} = y_{21} = -2$, this would not be a valid solution fulfilling (*), as the sum $-2 < 0$.
- $\forall u \neq v \quad [-1 \leq y_{uv} - y_{vu} \leq 1]$: This follows because we have $y_{uu} + y_{uv} + y_{vu} + y_{vv} = x_u x_u + x_u x_v + x_v x_u + x_v x_v = (x_u + x_v)^2 \geq 0$ and $y_{uu} - y_{uv} - y_{vu} + y_{vv} = x_u x_u - x_u x_v - x_v x_u + x_v x_v = (x_u - x_v)^2 \geq 0$. Using the fact that $y_{uu} = y_{vv} = 1$, we conclude that $-1 \leq y_{uv} - y_{vu} \leq 1$. For example, if the LP solver $\mathcal{P}$ provides $y_{12} = y_{21} = 100$, this would not be a valid solution.
- Suppose we select an arbitrary real number c_v for each $v \in V$. Then we have

$$\sum_{(u,v) \in V \times V} c_u c_v y_{uv} \geq 0. \quad (**)$$

This is similar to the first point. We observe that $\sum_{(u,v) \in V \times V} c_u c_v y_{uv} = \sum_{(u,v) \in V \times V} (c_u x_u)(c_v x_v) = \sum_{u \in V} c_u x_u \sum_{v \in V} c_v x_v = (\sum_{v \in V} c_v x_v)^2 \geq 0$.

By combining our LP problem LP with constraint (**), we have, in fact, formulated an SDP problem.

Fact 1. *We can solve SDP into a very good additive approximation via LP solution.*

2 Semidefinite Programming

In this section, we will formally introduce SDP. Let's begin by restating the SDP problem formulated earlier:

$$\max \quad \sum_{(u,v) \in V \times V} \left(\frac{1}{2} - \frac{1}{2} y_{uv} \right) w(u, v) \quad (9)$$

$$\text{s.t.} \quad \forall u \in V, v \in V \quad [y_{uv} = y_{vu}]; \quad (7)$$

$$\forall v \in V \quad [y_{vv} = 1]; \quad (8)$$

$$\forall (c_v)_{v \in V} \in \mathbb{R}^{|V|} \quad \left[\sum_{(u,v) \in V \times V} c_u c_v y_{uv} \geq 0 \right]. \quad (**)$$

We can capture constraint (**) by a *positive semidefinite* (PSD) matrix:

Definition 2. [Positive semidefinite (PSD)] A matrix $\mathbf{Y} = (y_{uv})_{n \times n} \in \mathbb{R}^{n \times n}$ is *positive semidefinite* (PSD) if for all $\mathbf{c} = (c_v)_{v \in V} \in \mathbb{R}^n$ we have $\mathbf{c}^\top \mathbf{Y} \mathbf{c} = \sum_{u,v} c_u c_v y_{uv} \geq 0$. We denote this as $\mathbf{Y} \succeq 0$.

In other words, constraint (**) is satisfied when $\mathbf{Y} = (y_{uv})_{n \times n}$ is PSD. Note that PSD matrices must be symmetric. The following theorem states that we can determine whether a matrix is PSD in polynomial time:

Theorem 3. *There exists a polynomial-time algorithm that, when given a symmetric matrix $\mathbf{Y} \in \mathbb{R}^{n \times n}$, can either find a vector $\mathbf{c} \in \mathbb{R}^n$ such that $\mathbf{c}^\top \mathbf{Y} \mathbf{c} < 0$ or, if $\mathbf{Y}$ is PSD and no such $\mathbf{c}$ exists, it can correctly report that $\mathbf{Y}$ is PSD.*

Proof. We can find a Cholesky decomposition of $\mathbf{Y}$ in polynomial time¹:

$$\mathbf{Y} = \mathbf{L} \mathbf{D} \mathbf{L}^\top \quad (10)$$

$$= \begin{bmatrix} 1 & 0 & \cdots & 0 \\ * & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & 0 \\ * & * & \cdots & 1 \end{bmatrix} \begin{bmatrix} * & 0 & \cdots & 0 \\ 0 & * & \cdots & 0 \\ \vdots & \vdots & \ddots & 0 \\ 0 & 0 & \cdots & * \end{bmatrix} \begin{bmatrix} 1 & * & \cdots & * \\ 0 & 1 & \cdots & * \\ \vdots & \vdots & \ddots & * \\ 0 & 0 & \cdots & 1 \end{bmatrix}, \quad (11)$$

where $\mathbf{L}$ is a lower triangular matrix and $\mathbf{D}$ is a diagonal matrix.

We claim that $\mathbf{Y}$ is PSD if and only if all $\mathbf{D}$'s entries are non-negative:

($\Rightarrow$) Note that $\mathbf{L}$ and $\mathbf{L}^\top$ as triangular matrices whose diagonal entries are all 1 are invertible. Therefore, $\mathbf{D} = (\mathbf{L}^{-1} \mathbf{L}) \mathbf{D} (\mathbf{L}^\top (\mathbf{L}^\top)^{-1}) = \mathbf{L}^{-1} \mathbf{Y} (\mathbf{L}^\top)^{-1} = \mathbf{L}^{-1} \mathbf{Y} (\mathbf{L}^{-1})^\top$. Note that the v -th diagonal entry of $\mathbf{D}$ is equal to the v -th row of $\mathbf{L}^{-1}$ times $\mathbf{Y}$ times the v -th column of $(\mathbf{L}^{-1})^\top$. Since $\mathbf{Y}$ is PSD, by definition each diagonal entry is non-negative and thus all $\mathbf{D}$'s entries are non-negative.

($\Leftarrow$) We can write $\mathbf{D} = \sqrt{\mathbf{D}} \sqrt{\mathbf{D}}$, where $\sqrt{\mathbf{D}}$ denotes a diagonal matrix whose v -th diagonal entry is equal to the square root of the v -th diagonal entry of $\mathbf{D}$. Also note that $(\sqrt{\mathbf{D}})^\top = \sqrt{\mathbf{D}}$ because it is diagonal and thus symmetric. Then for any $\mathbf{c}$ we have $\mathbf{c}^\top \mathbf{Y} \mathbf{c} = \mathbf{c}^\top \mathbf{L} \mathbf{D} \mathbf{L}^\top \mathbf{c} = \mathbf{c}^\top \mathbf{L} \sqrt{\mathbf{D}}^\top \sqrt{\mathbf{D}} \mathbf{L}^\top \mathbf{c} = (\sqrt{\mathbf{D}} \mathbf{L}^\top \mathbf{c})^\top (\sqrt{\mathbf{D}} \mathbf{L}^\top \mathbf{c}) \geq 0$ (because for any vector its transpose times itself is non-negative). Therefore, $\mathbf{Y}$ is PSD.

Therefore, our claim is indeed true. If $\mathbf{D}$ is non-negative, we can say $\mathbf{Y}$ is PSD. Otherwise, if the v -th diagonal entry of $\mathbf{D}$ is negative, we can find a certificate such that $\mathbf{c}^\top \mathbf{Y} \mathbf{c} < 0$ by setting the v -th column of $(\mathbf{L}^{-1})^\top$ as $\mathbf{c}$. This can all be done in polynomial time. $\square$

Proposition 4. *$\mathbf{Y}$ is PSD if and only if $\mathbf{Y} = \mathbf{Z}^\top \mathbf{Z}$ for some matrix $\mathbf{Z}$.*

Proof. ($\Rightarrow$) Just set $\mathbf{Z} = \sqrt{\mathbf{D}} \mathbf{L}^\top$.

($\Leftarrow$) Just set $\mathbf{D}$ to be the identity matrix and $\mathbf{L} = \mathbf{Z}^\top$. $\square$

We write the result of Prop. 4 in its expanded form (where $\mathbf{z}_v$ denotes the v -th row of $\mathbf{Z}$):

$$\begin{bmatrix} y_{11} & y_{12} & \cdots & y_{1n} \\ y_{21} & y_{22} & \cdots & y_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ y_{n1} & y_{n2} & \cdots & y_{nn} \end{bmatrix} = \begin{bmatrix} \mathbf{z}_1 \\ \mathbf{z}_2 \\ \vdots \\ \mathbf{z}_n \end{bmatrix} [\mathbf{z}_1^\top \quad \mathbf{z}_2^\top \quad \cdots \quad \mathbf{z}_n^\top]. \quad (12)$$

Consider constraints (7) and (8) again. Recall that $\cdot$ denotes dot product. For each $u \in V, v \in V$ we have

$$y_{uv} = \mathbf{z}_u \cdot \mathbf{z}_v; \quad (13)$$

$$y_{vv} = \mathbf{z}_v \cdot \mathbf{z}_v = 1 \Rightarrow \|\mathbf{z}_v\|_2 = 1. \quad (14)$$

Therefore, we can denote all the y_{uv} using a set of n **unit vectors** $(\mathbf{z}_v)_{v \in V}$.

¹You just need to know that every symmetric matrix can be decomposed in this way and do not need to know how the decomposition is done.

3 Semidefinite Relaxation for Integer Programming

In this section, we will discuss if we can solve an ILP problem (e.g., MaxCut) using its SDP relaxation and rounding, and what kind of approximation guarantee we can have.

Let us still consider the MaxCut problem where we now have an SDP that maximizes $\sum_{(u,v) \in V \times V} (\frac{1}{2} - \frac{1}{2} \mathbf{z}_u \cdot \mathbf{z}_v) w(u,v)$. We need to map each of the SDP solution $\mathbf{z}_v$ to one $x_v \in \{+1, -1\}$ such that it tells us which partition vertex v belongs to. Intuitively, if the angle between two vectors $\mathbf{z}_u$ and $\mathbf{z}_v$ is small, the two vectors are likely to be in the same partition; otherwise, the two vectors are likely to be in different partitions. For example, if $\mathbf{z}_u = \mathbf{z}_v = (1, 0, \dots, 0)$ (i.e., they point to the same direction), we have $y_{uv} = \mathbf{z}_u \cdot \mathbf{z}_v = 1$ and they belong to the same partition; if $\mathbf{z}_u = (1, 0, \dots, 0)$ and $\mathbf{z}_v = (-1, 0, \dots, 0)$ (i.e., they point to opposite direction), we have $y_{uv} = \mathbf{z}_u \cdot \mathbf{z}_v = -1$ and they belong to different partitions.

Therefore, if we round our solution in a way that vectors $\mathbf{z}_u$ and $\mathbf{z}_v$ with a small angle between them are likely to be put in the same partition, whereas vectors $\mathbf{z}_u$ and $\mathbf{z}_v$ with a large angle between them are likely to be put in different partitions, then our rounding will be a good approximation. See Fig. 2 below. Let θ_{uv} be the angle between two unit vectors $\mathbf{z}_u$ and $\mathbf{z}_v$. Suppose we randomly choose a hyperplane that passes through the origin (the red triple line). The probability that the two vectors fall into different sides of the hyperplane is clearly $\frac{\theta_{uv}}{\pi}$ (we can look at only the right semi-circle: only if the hyperplane in the right semi-circle falls between the two vectors, the two vectors will fall into different sides of the hyperplane). Based on this, we can round our solution based on which side of a randomly chosen hyperplane each vector $\mathbf{z}_v$ falls into. When two vectors are similar to each other and the angle between them θ_{uv} is small, $\frac{\theta_{uv}}{\pi}$ is small and they are likely to be put into the same partition; when two vectors are different from each other and the angle between them θ_{uv} is large, $\frac{\theta_{uv}}{\pi}$ is large and they are likely to be put into different partitions. This adheres to our intuition earlier.

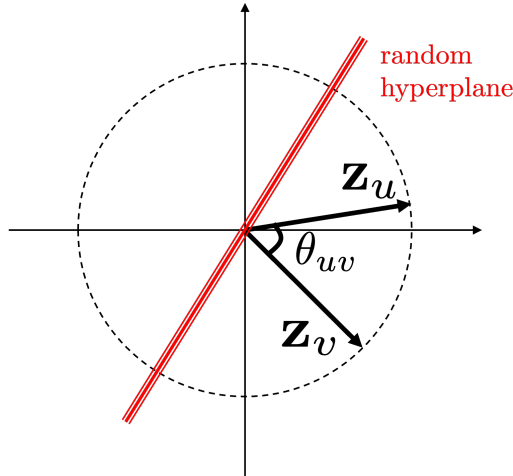


Figure 2: Illustration of rounding using a random hyperplane. The dashed circle represents the ball centered at the origin with radius 1 because all $\mathbf{z}_v$ are unit vectors.

Following this idea, we have the following randomized rounding algorithm:

1. Pick a random hyperplane through the origin. One can do this by picking n random numbers $g_1, g_2, \dots, g_n$ from the standard normal distribution $\mathcal{N}(0, 1)$ and the hyperplane is the plane orthogonal to the vector

$$\mathbf{g} = (g_1, g_2, \dots, g_n)^2$$

2. Partition the vertices based on which side of the plane the vectors are (i.e., $\mathbf{g}_v \cdot \mathbf{z}_v \geq 0$ or $\mathbf{g}_v \cdot \mathbf{z}_v < 0$).

Theorem 5. *The above algorithm has an approximation factor of 0.87856.*

Proof. Recall that the SDP optimum is

$$\begin{aligned} & \sum_{(u,v) \in V \times V} \left(\frac{1}{2} - \frac{1}{2} \mathbf{z}_u \cdot \mathbf{z}_v \right) w(u,v) \\ &= \sum_{(u,v) \in V \times V} \left(\frac{1}{2} - \frac{1}{2} \|\mathbf{z}_u\|_2 \|\mathbf{z}_v\|_2 \cos \theta_{uv} \right) w(u,v) \\ &= \sum_{(u,v) \in V \times V} \left(\frac{1}{2} - \frac{1}{2} \cos \theta_{uv} \right) w(u,v). \end{aligned} \quad (15)$$

We have explained earlier that the probability of two vertices u and v to be put in different partitions (and thus edge (u, v) is cut) is equal to $\frac{\theta_{uv}}{\pi}$. Thus, after rounding, the expectation of the total weight of the edges cut is

$$\begin{aligned} & \mathbb{E}[\text{total weight of edges cut}] \\ &= \sum_{(u,v) \in V \times V} w(u,v) \frac{\theta_{uv}}{\pi}. \end{aligned} \quad (16)$$

We want to bound the ratio between (16) and (15) such that the rounding is good. We notice (see the figure below) that for any $(u, v) \in V \times V$ (and thus any θ_{uv}),

$$\frac{\theta_{uv}/\pi}{\frac{1}{2} - \frac{1}{2} \cos \theta_{uv}} \geq 0.87856.$$

Therefore, the rounded solution achieves at least 0.87856 of the optimum of SDP relaxation. Since the optimum of SDP relaxation is larger than or equal to the optimum of the ILP, we have proven that the approximation factor is 0.87856. $\square$

4 Conclusion and Next Lecture

In this lecture, we have introduced SDP and demonstrated how we can solve the MaxCut ILP problem by using its SDP relaxation and how to round the SDP solution to get a good approximated solution to the ILP problem. Till now, we have been focusing on how to formulate ILP, LP and SDP and how to analyze the goodness of approximation. In the next lecture, we will introduce how to actually solve these problems.

References

- [DP93a] C. Delorme and S. Poljak. “Combinatorial properties and the complexity of a max-cut approximation”. In: *European Journal of Combinatorics* 14.4 (1993), pp. 313–333.
- [DP93b] C. Delorme and S. Poljak. “Laplacian eigenvalues and the maximum cut problem”. In: *Mathematical Programming* 62 (1993), pp. 557–574.
- [Kho10] S. Khot. “On the unique games conjecture”. In: *2010 IEEE 25th annual conference on computational complexity*. IEEE Computer Society. 2010, pp. 99–121.

²It can be easily seen by symmetry that the probability that the vector points to every direction is equal. In fact, this is equivalent to picking a multivariate Gaussian variable with mean $\mathbf{0}$ and covariance matrix $\mathbf{I}_{n \times n}$.

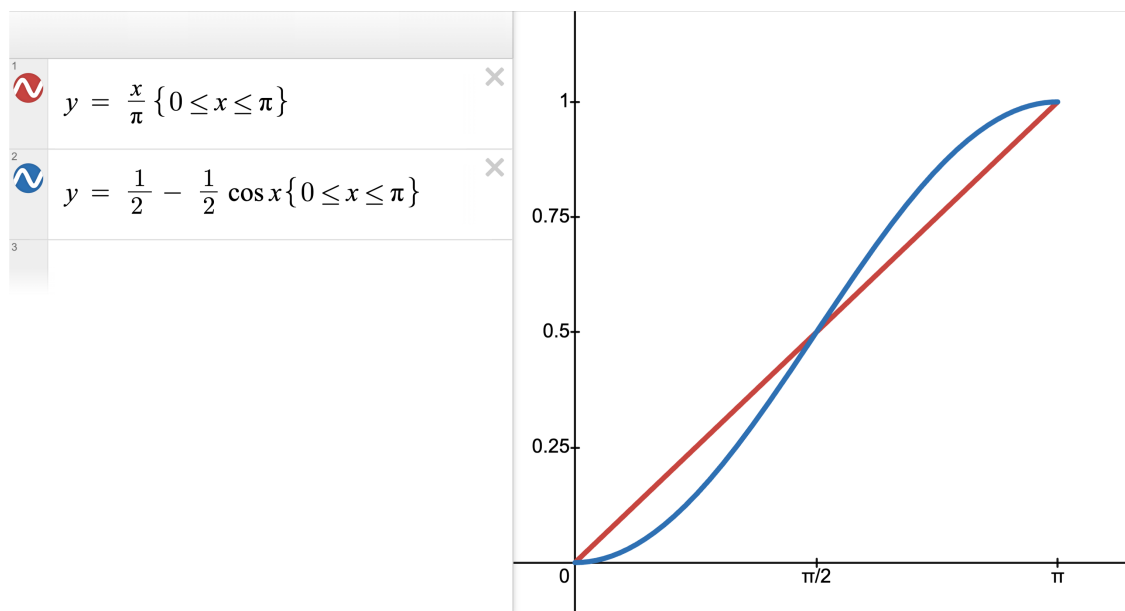


Figure 3: The two functions $\frac{\theta}{\pi}$ and $\frac{1}{2} - \frac{1}{2} \cos \theta$ are always close to each other and the largest ratio is 0.87856.

Linear Programming

Lecturer: Divesh Aggarwal

November 7, 2024

*Scribe: Sindhura Rajiv Jain, Zhao Yao, Yashwit Polapragada, Bibin Muttappillil, Tan Kwang Thiag,
Shen Licong, Nathan Xu*

Abstract

Linear programming (LP) is a technique used to solve optimization problems where the objective function and the constraints are linear. The central goal of LP is to maximize or minimize a linear function subject to a set of linear inequalities or equalities.

1 Introduction to Linear Programming

Linear Programming (LP) represents a fundamental optimization framework in mathematical programming.

Two significant developments shaped our understanding of LP:

- In 1979, Khachiyan proved that LP can be solved in polynomial time using the Ellipsoid Method [2]. This method provides a theoretical solution, showing that LP is tractable in a polynomial sense.
- In 1947, the Simplex Method [1] was developed by Dantzig. While the Simplex Method is heuristic and does not guarantee polynomial-time performance in all cases, it performs well on most real-world LP problems.

2 Problem Formulation in Linear Programming

2.1 Types of Problems in Linear Programming

Consider the system of inequalities K :

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &\geq b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &\geq b_2, \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &\geq b_m, \end{aligned}$$

or, in matrix form, $A\mathbf{x} \geq \mathbf{b}$.

Linear programming problems can generally be approached through two types of questions:

- **Feasibility Problem:** Find a $\mathbf{x} = (x_1, x_2, \dots, x_n)$ that satisfies all these inequalities, or to provide a proof that no such solution exists.

- **Optimization Problem:** In the optimization variant, we aim to maximize or minimize a linear objective function while adhering to a set of constraints. Solving this can often be reduced to a feasibility question by reformulating the problem, allowing us to focus on finding feasible solutions within a given search space.

2.2 Feasibility

To determine the feasibility of a system of inequalities, we need to check if there exists a solution. If no solution exists, the proof of infeasibility can take one of two forms:

1. **Incompatible Constraints:** There may be pairs of constraints that are inherently incompatible, meaning that they cannot be satisfied simultaneously. This approach may be overly simplistic and works only in special cases.
2. **Contradictory Inequality:** A more general approach is to derive a contradiction, such as an inequality of the form $0 \geq 1$, by forming a combination of the inequalities that leads to an impossible statement.

Example of Contradictory Inequality

Consider the following system of inequalities:

$$\begin{aligned} 2x - 3y &\geq 9, \\ z + 4y - 3x &\geq -8, \\ x - y - z &\geq 2. \end{aligned}$$

Adding these inequalities yields:

$$0 \geq 3,$$

which is a contradiction, meaning that no values of x , y , and z can simultaneously satisfy all three inequalities.

Generalizing the Idea: Positive Linear Combination

To formalize this approach, we aim to construct a *positive linear combination* of the inequalities, where we introduce multipliers $\lambda_1, \lambda_2, \dots, \lambda_m \geq 0$ for each inequality. This is done as follows:

$$\begin{aligned} \lambda_1 (a_{11}x_1 + \dots + a_{1n}x_n) + \lambda_2 (a_{21}x_1 + \dots + a_{2n}x_n) + \dots + \lambda_m (a_{m1}x_1 + \dots + a_{mn}x_n) &= 0, \\ \lambda_1 b_1 + \lambda_2 b_2 + \dots + \lambda_m b_m &= 1. \end{aligned}$$

The goal is to set the left-hand side (LHS) of the combined inequalities to zero while ensuring that the right-hand side (RHS) sums to a positive constant, such as 1. Since 0 cannot be greater than a positive number, this would confirm infeasibility by contradiction.

2.3 Theorem: Farkas' Lemma / LP Duality

We now introduce a key result in linear programming called Farkas' Lemma, also known as LP Duality. This result provides a characterization of infeasibility for a system of linear inequalities.

Theorem 1: (Farkas' Lemma / LP Duality) If the linear system $A\mathbf{x} \geq \mathbf{b}$ has no solution, then there exists a $\lambda = (\lambda_1, \lambda_2, \dots, \lambda_m)$, where each $\lambda_i \geq 0$, such that:

$$\lambda^T A\mathbf{x} = 0, \quad \text{and} \quad \lambda^T \mathbf{b} = 1.$$

The first equation $\lambda^T A\mathbf{x} = 0$ indicates that the linear combination of the inequalities (weighted by λ) results in no net effect.

The second equation $\lambda^T \mathbf{b} = 1$ ensures that the weighted sum of the constants b_i is equal to 1.

The proof of Farkas' Lemma is an exponential-time algorithm.

Proof of Farkas' Lemma:

Step 1: The System of Inequalities

Consider the following system of inequalities:

$$x - 2y + z \geq 3 \quad (1)$$

$$3x + y - z \geq -2 \quad (2)$$

$$y - 2z \geq 1 \quad (3)$$

$$-x + 2y - 3z \geq 4 \quad (4)$$

$$x - y - 4z \leq -5 \quad (5)$$

Step 2: Expressing x in Terms of y and z

To analyze this system, we can simplify it by expressing each inequality in terms of x . Solving each inequality for x in terms of y and z allows us to understand the bounds on x imposed by these inequalities.

From inequality (1), we get:

$$x \geq 3 + 2y - z. \quad (6)$$

From inequality (2), we get:

$$x \geq -\frac{2}{3} - \frac{y}{3} + \frac{z}{3}. \quad (7)$$

From inequality (3), we get:

$$y - 2z \geq 1. \quad (8)$$

From inequality (4), we get:

$$x \leq 2y - 3z - 4 \quad (9)$$

From inequality (5), we get:

$$x \leq y + 4z - 5 \quad (10)$$

Now we have expressions for x . By substituting the expressions for x into each other, we can eliminate x .

Step 3: Eliminate x :

Restating (8) as it does not contain x :

$$y - 2z \geq 1. \quad (11)$$

Combining (6) and (9), we get:

$$3 + 2y - z \leq 2y - 3z - 4 \quad (12)$$

Combining (7) and (9), we get:

$$-\frac{2}{3} - \frac{y}{3} + \frac{z}{3} \leq 2y - 3z - 4 \quad (13)$$

Combining (6) and (10), we get:

$$3 + 2y - z \leq y + 4z - 5 \quad (14)$$

Combining (7) and (10), we get:

$$-\frac{2}{3} - \frac{y}{3} + \frac{z}{3} \leq y + 4z - 5. \quad (15)$$

These equations represent a system of inequalities that must hold true for a valid λ to exist. The system of inequalities in step 3 has solution if and only the system of inequalities in step 2 has a solution as the same solution is used in both steps. Do this again by eliminating y and then eliminating z . Combining these, we either get a contradiction or a solution that satisfies the inequalities.

Step 4: Relating to Farkas' Lemma

In each step, we are multiplying the inequality by a non-negative number then adding the inequalities to eliminate variables. Eventually, we either get a contradiction or a solution that satisfies the inequalities.

We now analyze the relationships between the inequalities above. Specifically, if the system $A\mathbf{x} \geq \mathbf{b}$ has no solution, then the system must have a solution. The system involves finding a non-negative $\lambda = (\lambda_1, \lambda_2, \dots, \lambda_m)$, where each $\lambda_i \geq 0$, such that:

1. $\lambda^T A\mathbf{x} = 0$, which ensures that the inequalities hold in a balanced way.
2. $\lambda^T \mathbf{b} = 1$, which guarantees that the system of inequalities is consistent.

This is essentially the statement of Farkas' Lemma. In other words, for the given system of inequalities, if no solution exists for $\mathbf{x}$, there must exist a corresponding λ that satisfies the conditions.

Step 5: Conclusion

We have shown that if the system $A\mathbf{x} \geq \mathbf{b}$ has no solution, then the corresponding system involving λ must have a solution. λ satisfies the conditions:

$$\lambda^T A\mathbf{x} = 0, \quad \text{and} \quad \lambda^T \mathbf{b} = 1,$$

with each $\lambda_i \geq 0$, as required by Farkas' Lemma. This completes the proof.

2.4 Remarks

To prove that there is no solution for the system of inequalities $A\mathbf{x} \geq \mathbf{b}$, we aim to find a $\lambda = (\lambda_1, \dots, \lambda_m)$ with $\lambda_i \geq 0$ for each i , such that:

$$\lambda^T A\mathbf{x} = 0$$

and

$$\lambda^T \mathbf{b} = 1.$$

This proof can be formulated as a linear program (LP) because it is a system involving linear equalities and inequalities. To convert the system into a standard linear program, we handle the equalities by rewriting them as two separate inequalities:

$$\lambda^T A\mathbf{x} \geq 0 \quad \text{and} \quad \lambda^T A\mathbf{x} \leq 0.$$

Solving a linear program with $\geq$ and $=$ constraints can be reduced to standard form. Note, however, that there is no efficient algorithm for linear programs involving strict inequalities (e.g., $>$ or $<$), so making the use of $\geq$ or $\leq$ constraints crucial in our formulation.

3 Complexity Class Implications

This problem is in $\text{NP} \cap \text{co-NP}$. To show this, we need to demonstrate that there exists a solution of polynomial size in terms of the bitlength L of the input data.

Theorem 2: Given a system of inequalities $A\mathbf{x} \geq \mathbf{b}$, where $A \in \mathbb{Z}^{m \times n}$ and $\mathbf{b} \in \mathbb{Z}^m$, with each entry of A and $\mathbf{b}$ having bitlength at most L , then:

1. No solution $\Rightarrow \exists$ a rational proof $\lambda_1, \dots, \lambda_m$, of bitlength that is polynomial in L .
2. A solution exists $\Rightarrow \exists$ a rational solution of bitlength polynomial in L .

These two statements imply that the problem is in $\text{NP} \cap \text{co-NP}$, and to show this, it is sufficient to prove only the first statement (non-existence of a solution) as finding a proof is also a linear program that is polynomial in the size of the input.

Remark: BitLength of Rational Numbers

For a rational number $\frac{p}{q} \in \mathbb{Q}$, the bitlength of $\frac{p}{q}$ is given by:

$$\text{bitlength}\left(\frac{p}{q}\right) = \lceil \log p \rceil + \lceil \log q \rceil.$$

This means that the number of bits needed to express a rational solution is polynomially bounded in L .

4 Proving Theorem 2

4.1 Case 1: The Feasible Region Is Bounded.

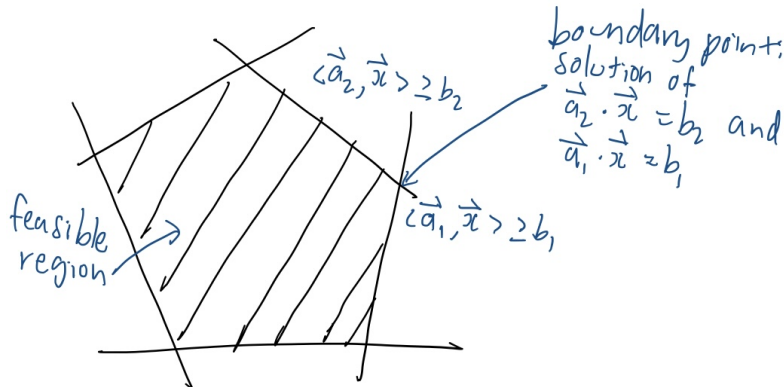


Figure 1: Bounded Feasible Region.

The shaded area in Figure 1 represents the feasible region defined by the set of constraints.

The feasible region defined by a set of constraints forms a *convex space*. Any vertex of the feasible region, if we had n variables, corresponds to a solution of n linear equalities.

A system of n equalities is solved using Gaussian elimination. The bitlength of the solution is bounded because any vertex of the feasible region is given by

$$x_i = \frac{\det(\text{entries from } \mathbf{A}, \mathbf{b})}{\det(\text{entries from } \mathbf{A}, \mathbf{b})}$$

The largest possible determinant is obtained by multiplying n entries. The bitlength of the determinant is the sum of the bitlengths of the individual entries, so the bitlength of the determinant is at most n times the bitlength of the original entries.

Hence, the bitlength of any vertex of the feasible region is *polynomial in the bitlength L* of the input data, and a vertex of a feasible region is also feasible, hence proving Theorem 2.

4.2 Case 2: The Feasible Region Is Not Bounded

However, in some cases, the feasible region may not be bounded (i.e. there is no vertex), as shown in Figure 2.

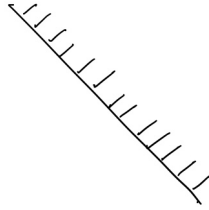


Figure 2: Unbounded Feasible Region.

In these cases where the LP does not have a vertex (i.e., the feasible region is not bounded), we can introduce box constraints of the form:

$$-B \leq x_i \leq B$$

where $B = 2^{\text{poly}(L)}$ (i.e. B is very large), with L being the bitlength of the input data. Here, B is chosen to be very large so that there definitely is a non-trivial intersection but is not so large so B is still $2^{\text{poly}(L)}$.

By introducing box constraints, we will always have a vertex so the argument for case 1 applies, proving Theorem 2.

Question: Why can we always introduce box constraints and still have a valid solution?

To answer this, modify the form of LP.

Claim: Starting from LP in this form:

$$\mathbf{Ax} \geq \mathbf{b}$$

we can always find an equivalent linear program that is of this form:

$$\mathbf{Cy} = \mathbf{d}, \quad \mathbf{y} \geq 0$$

where the only inequality constraints are that all variables are non-negative and every other constraint is an equality constraint.

We can do so as follows using slack variables:

Represent

$$Ax \geq b$$

as

$$Ax - z = b, \quad z \geq 0$$

Replace each variable x_i with $x_i = x_i^+ - x_i^-$ where $x_i^+ \geq 0$ and $x_i^- \geq 0$ as x_i can be positive or negative. For example, if $x_i = 2$, let $x_i^+ = 2$ and $x_i^- = 0$. If $x_i = -2$, let $x_i^+ = 0$ and $x_i^- = 2$.

Now, y can be redefined in terms of the variables $x_1^+, \dots, x_n^+, x_1^-, \dots, x_n^-, z_1, \dots, z_m$, with a total of $2n + m$ variables.

Hence, the LP can be represented as

$$Cy = d, \quad y \geq 0$$

where C and d depend on A and b respectively.

Without loss of generality, assume full rank: $m \leq n$ so the number of variables is at least the number of constraints and assume all of them are independent.

Recall that the system now of this form:

$$Cy = d, \quad y \geq 0, \quad C \in \mathbb{Z}^{m \times n}, \quad d \in \mathbb{Z}^m$$

Case 1: $m = n$.

Solve the linear system $Cy = d$ and check where that solution is at least 0. A unique solution is obtained, which represents a point.

Case 2: $m = n - 1$.

Now, there is one less constraint than the number of variables. The feasible solution corresponds to a line after it crosses the boundary of the first quadrant, so it intersects either $y_1 = 0$ or $y_2 = 0$ or $\dots$ or $y_n = 0$ at some point. Some y_i must be equal to 0. With this additional constraint, we get a unique solution with bitlength polynomial in L , and hence the solution lies inside the bounding box.

Case 2: $m = n - 2$.

Instead of a line, the solution is a 2-dimensional plane. Two additional constraints can be introduced: $y_i = 0$ and $y_j = 0$ as they will cross at some point. Similar to previous cases, we get a system of n linear equations and obtain a solution with bitlength polynomial in L , so the solution lies inside the bounding box.

General Case:

When $m = n - k$, we can introduce k constraints and still get a feasible solution that is bounded by the size of the input.

Given $\binom{n}{k}$ possible choices of constraints, we obtain an algorithm with complexity $O(2^n)$.

Hence, we have shown that there exists a solution is bounded by bitlength polynomial in L and we can introduce constraints to get this solution at a vertex.

4.3 Proof Summary: LP is in $\text{NP} \cap \text{co-NP}$

This procedure shows that any system of inequalities with bitlength L remains feasible if we introduce bounding constraints of the form $-B \leq x_i \leq B$, where $B = 2^{\text{poly}(L)}$. This means that we can introduce these box constraints and still get a feasible solution. This feasible solution is now one of the vertices of the new linear program. By introducing box constraints, we will always have a vertex so the argument for case 1 applies, proving Theorem 2.

Therefore, LP belongs to $\text{NP} \cap \text{co-NP}$.

5 Search vs Decision

5.1 Search Problem

We are given an LP in **equational form**:

$$K = \{A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq 0\}$$

where A is a matrix, $\mathbf{x}$ is a vector of variables, and $\mathbf{b}$ is a vector of constraints.

Goal: We aim to find a solution $\mathbf{x}$ using a decision oracle, which allows us to query whether the linear program (LP) has a feasible solution or not. In other words, this oracle decides whether a solution exists.

1. Suppose $m = n - k$, where m is the number of constraints, n is the number of variables, and k is the rank deficiency of the system. We have previously proven that we can introduce k constraints of the form $x_i = 0$, $x_j = 0$, ..., $x_k = 0$ and still get a feasible solution that is bounded in the size of the input (see 4.2).
2. Therefore, we can set $x_1 = 0$ and use the decision oracle to check if the system still has a solution. If the system remains feasible, we proceed by setting $x_2 = 0$ and repeating the process. If the system no longer has a solution, we disregard x_1 and continue the process by setting $x_2 = 0$.
3. By repeating this process until x_k is set and the decision oracle is queried to check if the system still has a solution, we can identify specific variables $x_{i_1}, \dots, x_{i_k}$ whose values contribute to a feasible solution.
4. This approach requires at most n^k calls to the decision oracle to find a solution.

This shows how you can solve the search problem using a decision oracle.

5.2 Optimization Problem

Objective:

$$\text{Maximize } \mathbf{C}^T \cdot \mathbf{x} \quad \text{subject to } A \cdot \mathbf{x} \geq \mathbf{b}$$

There are two possible outcomes for the maximization problem:

1. Unbounded Maximizer: The maximizer goes to infinity (i.e., $\max(\mathbf{C}^T \cdot \mathbf{x}) \rightarrow \infty$).
2. Finite Maximizer: $\max(\mathbf{C}^T \cdot \mathbf{x})$ is a finite value.

We focus on the second case, where the feasible region is bounded, ensuring a finite maximum.

Claim: In the finite case, there always exists a **vertex** (intersection of boundary constraints) in the feasible region that maximizes $\mathbf{C}^T \cdot \mathbf{x}$.

Reason: This is because any maximum of a linear function over a convex polytope occurs at one of its vertices.

Hence, it suffices to find the vertex of the feasible region that maximizes $\mathbf{C}^T \cdot \mathbf{x}$.

To find this vertex, we introduce a parameter t and add a new constraint:

$$\mathbf{C}^T \cdot \mathbf{x} \geq t$$

To ensure that we maximize $\mathbf{C}^T \cdot \mathbf{x}$, our goal is to find the **largest** value of t such that there exists a solution.

The vertices are bounded by a polynomial in the bitlength so all vertices are within the region:

$$-2^{L^2} \leq t \leq 2^{L^2}$$

This means that the maximal is also within this region

1. Use **binary search** on t within the range:

$$-2^{L^2} \leq t \leq 2^{L^2}$$

where L is the bitlength of the input.

2. With each step of binary search, we adjust t and check feasibility until finding the optimal t value that satisfies the constraint. This optimal t value is the largest value of t such that the constraint is fulfilled.

3. This approach yields a $(1 + \varepsilon)$ -approximation of the maximum in time $\text{poly}(L, \log(1/\varepsilon))$.

This gives a good approximation. Alternatively, the maximum can be computed exactly using continued fractions for improved precision.

References

- [1] George B Dantzig, Alex Orden, Philip Wolfe, et al. “The generalized simplex method for minimizing a linear form under linear inequality restraints”. In: *Pacific Journal of Mathematics* 5.2 (1955), pp. 183–195.
- [2] Leonid Genrikhovich Khachiyan. “A polynomial algorithm in linear programming”. In: *Doklady Akademii Nauk*. Vol. 244. 5. Russian Academy of Sciences. 1979, pp. 1093–1096.

Ellipsoid Method

Lecturer: Divesh Aggarwal

November 14, 2024

Scribe: Aprup Kale, Andrew Yapp Wei Rong, Ong Wei Hong, Xin Dongyue, Chong Chin Herng

Abstract

In this lecture, we continue from the previous discussion on the feasibility of Linear Programming (LP) problems and their placement in $NP \cap co-NP$. We focus on the Ellipsoid Method, a polynomial-time algorithm for solving the feasibility decision problem in LP. We explore its theoretical foundation, including the iterative reduction of ellipsoids, the role of separating hyperplanes, and the conditions for convergence. We also discuss the implications of LP duality, highlighting the Strong Duality Theorem and its applications to optimization problems like maximum bipartite matching.

1 Introduction

In this lecture, we build upon the foundational understanding of Linear Programming (LP) established in the previous session. We revisit the feasibility question, which involves determining whether a given system of linear inequalities has a solution. The discussion centers on the placement of LP feasibility problems within $NP \cap co-NP$ and the theoretical guarantees associated with such classifications.

The motivation for this lecture stems from the need for efficient algorithms to solve LP problems. We introduce the Ellipsoid Method, a theoretically robust algorithm that provides a polynomial-time solution to the feasibility decision problem.

Through this lecture, we aim to understand the core ideas of the Ellipsoid Method, explore its algorithmic structure, and examine its convergence properties. Additionally, we discuss the implications of LP duality, with a focus on the Strong Duality Theorem, and highlight practical applications, such as maximum flow problems and bipartite matching.

2 The Ellipsoid Method

The Ellipsoid Method is a polynomial-time algorithm for solving the feasibility decision problem in Linear Programming (LP) ([1]). The LP optimization problem is polynomial-time reducible to the LP feasibility search, and the LP feasibility search is polynomial-time reducible to the LP feasibility decision. Therefore, it is sufficient to solve the feasibility decision problem to address broader LP challenges. Below, we describe the method in detail, based on the insights provided in the lecture.

We are tasked with determining whether a given system of linear equations (we saw in the last lecture that we can always reduce any standard LP feasibility problem to this form):

$$Ax = b$$

$$x \geq 0$$

has a feasible solution. Here, A is a matrix of coefficients, x is the vector of variables, and b is a vector of constants.

In a “two dimensional” view, the YES and NO cases look as in Figure 1.

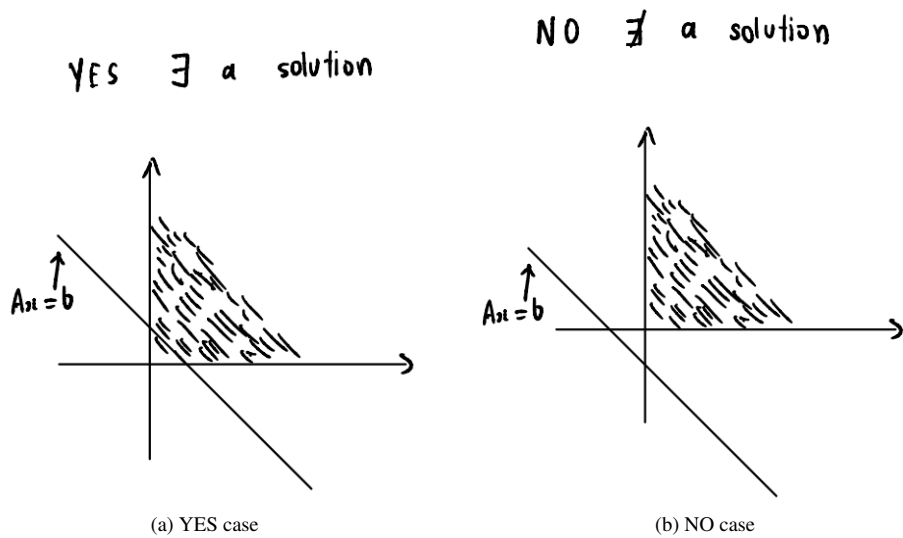


Figure 1: YES and NO cases in a “two dimensional” view

Let κ be the bit size of the input.

If no solution exists, proof of infeasibility of polynomial size exists in the input’s bit size κ . When we are in the NO case, the gap between the “line” $Ax = b$ and the feasible region is at least $\frac{1}{2^{\text{poly}(\kappa)}}$ (see Figure 2). This comes from the fact that there is a proof of infeasibility that is of size $\text{poly}(\kappa)$.

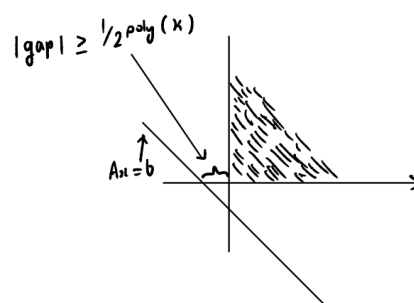


Figure 2: Gap between the “line” and the feasible region

If we replace $Ax = b$ by:

$$b - \varepsilon \leq Ax \leq b + \varepsilon$$

where $\varepsilon = \frac{1}{2^{\text{poly}(\kappa)}}$, then infeasible inputs would remain infeasible, but the feasible inputs will contain a small full-dimensional cube. We can essentially now fit a small “cube” of dimension $\frac{1}{2^{\text{poly}(\kappa)}}$ in the feasible region.

Now we try to find whether:

- there is no solution, OR

- there is a region of feasible points containing the cube

Note that we have chosen the ε small enough so that if we were originally in the NO case, we would still have no solution.

The feasible region K is contained in a box of dimension $B = 2^{\text{poly}(\kappa)}$. So essentially what we have is that we have a small cube (of feasible points), which is contained in the feasible region K , and the feasible region K is in turn contained in the larger cube.

If we have an algorithm that finds the feasible solution, if it exists, then we also have an algorithm that tells us whether there exists a feasible solution or not. Hence, it suffices to show that if a feasible solution exists, we will find it. We will assume there is a feasible point and try to find it.

K is contained in a ball centered at the origin and of radius $B\sqrt{n}$ (see Figure 3).

$$K \subseteq \text{Ball}(\vec{0}, B\sqrt{n})$$

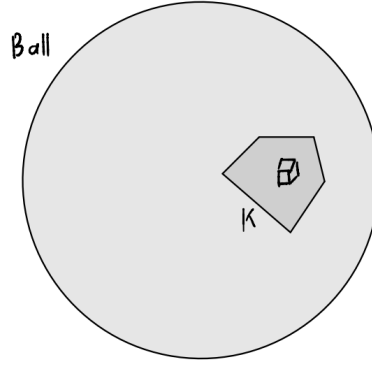


Figure 3: The ball and the region K

Let the initial ellipsoid E_0 be the ball $\text{Ball}(\vec{0}, B\sqrt{n})$. At each step, the algorithm evaluates whether the center $\vec{c}$ of the current ellipsoid E_k lies in the feasible region K . There are two possibilities:

- **Feasible Center:** If $\vec{c} \in K$, the algorithm terminates, and $\vec{c}$ is reported as a feasible point.
- **Infeasible Center:** If $\vec{c} \notin K$, a separating hyperplane is determined. This hyperplane divides the ellipsoid into two parts, ensuring that K lies entirely within one half.

The algorithm then constructs a new ellipsoid E_{k+1} of minimum volume that contains the half-ellipsoid intersecting K (see Figure 4). The volume reduction at each step is given by:

$$\frac{\text{Vol}(E_{k+1})}{\text{Vol}(E_k)} < 1 - \frac{1}{n},$$

The initial volume of the ball (also an ellipsoid) E_0 is at most:

$$\text{Vol}(E_0) \leq \pi^n \cdot (B\sqrt{n})^n \cdot n^n < \pi^n \cdot 2^{n \cdot \text{poly}(\kappa)} \cdot n^{2n}$$

The volume of the final ellipsoid, say E_f , is at least:

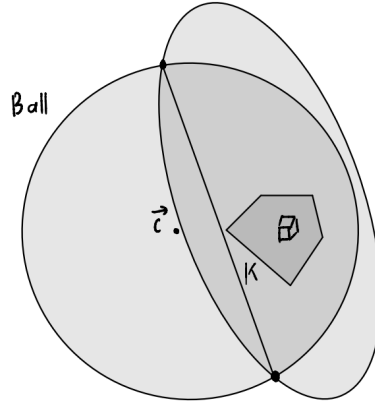


Figure 4: Ellipsoid E_{k+1} is the ellipsoid that passes through the points of intersection of the ellipsoid E_k and the separating hyperplane

$$\text{Vol}(E_f) \geq \left(\frac{1}{2^{\text{poly}(\kappa)}}\right)^n$$

as the final ellipsoid must contain the small cube of the feasible solutions. Hence, the ratio of the initial volume of the ellipsoid to the final volume of the ellipsoid is at most:

$$\frac{\text{Initial volume of ellipsoid}}{\text{Final volume of ellipsoid}} \leq 2^{\text{poly}(\kappa)} \cdot n^{2n} \cdot \pi^n$$

The algorithm thus terminates after $\log_{\left(\frac{1}{1-\frac{1}{n}}\right)}(2^{\text{poly}(\kappa)} \cdot n^{2n} \cdot \pi^n)$ steps (as the volume decreases by a factor of $1 - \frac{1}{n}$ at each step). The number of iterations for the algorithm is thus bounded by:

$$\log_{\left(\frac{1}{1-\frac{1}{n}}\right)}(2^{\text{poly}(\kappa)} \cdot n^{2n} \cdot \pi^n) = O(n \cdot (2 \cdot \text{poly}(\kappa) \cdot n + 2n \cdot \log(n) + n \cdot \log(\pi)))$$

which is obviously polynomial in n and κ .

3 LP Duality

The concept of LP duality is a cornerstone of Linear Programming, providing a framework to relate two optimization problems: the primal problem and the dual problem. The duality principle not only allows for efficient verification of solutions but also plays a crucial role in the theoretical understanding of optimization problems. In this section, we explore the key ideas of LP duality as covered in the lecture.

Given a primal LP problem of the form:

$$\text{maximize } c^T x, \quad \text{subject to } Ax \leq b, x \geq 0,$$

its corresponding dual LP problem is:

$$\text{minimize } b^T y, \quad \text{subject to } A^T y \geq c, y \geq 0.$$

Here:

- x represents the primal variables.

- y represents the dual variables.
- The primal constraints $Ax \leq b$ correspond to the dual variables y , and the primal objective coefficients c correspond to the dual constraints $A^T y \geq c$.

To see how we obtain the dual LP problem, consider some $\vec{y} = (y_1, y_2, \dots, y_m)$. We can then derive the inequality

$$(y_1 \vec{a}_1 + y_2 \vec{a}_2 + \dots + y_m \vec{a}_m) \cdot \vec{x} \leq y_1 b_1 + y_2 b_2 + \dots + y_m b_m$$

from the following set of inequalities

$$\begin{aligned} y_1(\vec{a}_1 \cdot \vec{x} &\leq b_1) \\ y_2(\vec{a}_2 \cdot \vec{x} &\leq b_2) \\ &\vdots \\ y_m(\vec{a}_m \cdot \vec{x} &\leq b_m) \end{aligned}$$

If $\sum_{1 \leq i \leq m} y_i \vec{x} = A^T y = \vec{c}$, then we get an upper bound which is $\vec{y} \cdot \vec{b}$.

If we get another $\vec{y}'$ such that $A^T y' = c$, then the upper bound of Ax changes to the minimum of $\vec{y} \cdot \vec{b}$ and $\vec{y}' \cdot \vec{b}$. Thus, we want to choose a $\vec{y}$ such that the linear function of $\vec{x}$, Ax , for which we get an upper bound is, in turn, an upper bound for $c^T x$.

If we choose $\vec{y}$ such that:

$$\begin{aligned} c_1 &\leq \vec{a}_1 \cdot \vec{y} \\ c_2 &\leq \vec{a}_2 \cdot \vec{y} \\ &\vdots \\ c_m &\leq \vec{a}_m \cdot \vec{y} \end{aligned}$$

Now for every $\vec{y}$ that satisfies the above inequalities and for every $\vec{x}$ that is feasible, we have:

$$c^T x \leq (y_1 \vec{a}_1 + y_2 \vec{a}_2 + \dots + y_m \vec{a}_m) \cdot \vec{x} \leq y_1 b_1 + y_2 b_2 + \dots + y_m b_m$$

Clearly we want to find the non-negative values $y_1, y_2, \dots, y_m$ such that the above upper bound is as strong as possible. That is, we want to:

$$\text{minimize } b^T y, \quad \text{subject to } A^T y \geq c, y \geq 0.$$

Thus, we have obtained exactly the corresponding dual LP problem.

The duality relation ensures that if both the primal and dual problems have feasible solutions, the optimal value of the primal objective is equal to the optimal value of the dual objective.

Theorem 1 (Strong Duality Theorem). *If the primal and dual Linear Programs have finite optimal solutions, then the optimal values of their objectives are equal.*

The proof of the Strong Duality Theorem hinges on the separation of feasible and infeasible regions. Let x^* be the optimal solution to the primal problem, and y^* be the optimal solution to the dual problem. We aim to show:

$$c^T x^* = b^T y^*.$$

Then clearly, $c^T x^* \leq b^T y^*$.

Assume, for contradiction, that:

$$\begin{aligned}
& b^T y^* > c^T x^* \\
\implies & \forall x, y : b^T y > c^T x \\
\implies & \forall x, y : b^T y \not\leq c^T x
\end{aligned}$$

Thus, the system of equations

$$\begin{aligned}
Ax &\leq b \\
A^T y &= c \\
y &\geq 0 \\
b^T y &\leq c^T x
\end{aligned}$$

must be infeasible. This means, by Farka's lemma, that there must exist a proof of infeasibility. That is, we must be able to multiply these inequalities by some non-negative real numbers to arrive at a contradiction.

By Farka's lemma, there must exist vectors $\vec{z}_1, \vec{z}_2$ and a real number z_3 such that:

$$\begin{aligned}
z_1^T Ax + z_2^T y + z_3(b^T y - c^T x) &= 0 \\
\text{and} \\
z_1^T b + z_2^T c &> 0
\end{aligned}$$

From the first equation we get:

$$\begin{aligned}
z_1^T A = z_3 c^T &\implies z_1^T A z_2 = z_3 c^T z_2 \\
\text{and} \\
z_2^T A^T + z_3 b = 0 &\implies z_2^T A^T z_1 = -z_3 b^T z_1
\end{aligned}$$

However, these two equations, $z_1^T A z_2 = z_3 c^T z_2$ and $z_2^T A^T z_1 = -z_3 b^T z_1$, lead to a contradiction:

$$z_1^T b + z_2^T c = 0$$

Given a graph $G = (V, E)$, the maximum flow problem can be formulated as the following primal linear program (LP):

$$\begin{aligned}
& \text{maximize} \quad \sum_{(s,v) \in E} f_{sv} - \sum_{(v,s) \in E} f_{vs}, \\
& \text{subject to:} \\
& \quad \sum_{(u,v) \in E} f_{uv} - \sum_{(v,u) \in E} f_{vu} = 0, \quad \forall v \in V \setminus \{s, t\}, \\
& \quad 0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E.
\end{aligned}$$

where:

- f_{uv} is the flow along edge (u, v) .
- c_{uv} is the capacity of edge (u, v) .
- s and t are the source and sink vertices, respectively.

The dual linear program to the maximum flow linear program is exactly the linear program for the minimum cut problem. Thus, by the strong duality theorem, the value of the maximum (s, t)-flow in graph G is exactly the same as the capacity of the minimum (s, t)-cut in graph G .

LP duality is particularly useful for understanding problems by examining their duals, which often provide new perspectives or simplifications. For instance, while the primal formulation of a problem might be complex, the dual can highlight structural properties that simplify analysis or lead to efficient algorithms. By looking at the dual, we can sometimes identify more intuitive or computationally efficient ways to solve the original problem.

4 Maximum Bipartite Matching

Throughout the course, we saw multiple approximation algorithms for NP-hard problems that could be written as Integer Linear Programs (ILPs) and then relaxed to LPs. We then solved the LPs and rounded the LP optimal solutions to obtain approximate solutions to the original ILP.

There are some problems, however, where the LP solution is the same as the ILP solution. One such problem is Maximum Bipartite Matching.

Given a bipartite graph $G = (V, E)$ and a weight function $w : E \rightarrow \mathbb{R}_{\geq 0}$, we can formulate the maximum bipartite matching problem as the following LP:

$$\begin{aligned} & \text{maximize} && \sum_{(u,v) \in E} x_{uv} w(u,v), \\ & \text{subject to:} && \\ & && \sum_{u: (u,v) \in E} x_{uv} = 1, && \forall v \in V, \\ & && 0 \leq x_{uv} \leq 1, && \forall (u,v) \in E. \end{aligned}$$

If the LP returns an integral solution, that is, $\forall (u,v) \in E : x_{uv} \in \{0, 1\}$, then we are done.

Suppose the LP does not return an integral solution. Thus, $\exists (u,v) \in E : x_{uv} \notin \{0, 1\}$. Since the solution is a perfect matching, $\exists (v,w) \in E$ such that $0 < x_{vw} < 1$. Similarly, $\exists (w,p) \in E$ such that $0 < x_{wp} < 1$ (see Figure 5).

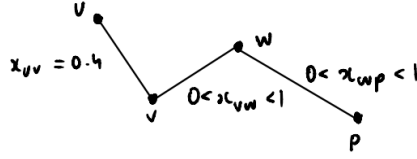


Figure 5: Edges adjacent to any edge with non-integral value must also have non-integral values

Eventually, this walk will lead to a cycle (or a path) such that all for all the edges (a,b) in the cycle (or path), we will have $0 < x_{ab} < 1$ (see Figure 6). Also note, that this cycle must have an even number of edges as a bipartite graph cannot have a cycle with an odd number of edges.

There is a better solution by changing the values for x_{ab} for all edges (a,b) on this cycle.

If in the graph as shown in Figure 6, we change the values as follows:

$$\begin{aligned} x_{u_1 u_2} &\leftarrow x_{u_1 u_2} + \epsilon \\ x_{u_2 u_3} &\leftarrow x_{u_2 u_3} - \epsilon \\ &\vdots \\ x_{u_7 u_8} &\leftarrow x_{u_7 u_8} + \epsilon \\ x_{u_8 u_1} &\leftarrow x_{u_8 u_1} - \epsilon \end{aligned}$$

then $\sum_{u: (u,v) \in E} x_{uv}$ changes by $\epsilon(w_1 + w_3 + w_5 + w_7) - \epsilon(w_2 + w_4 + w_6 + w_8) = x^+$ (say)

If instead, we change the values as follows:

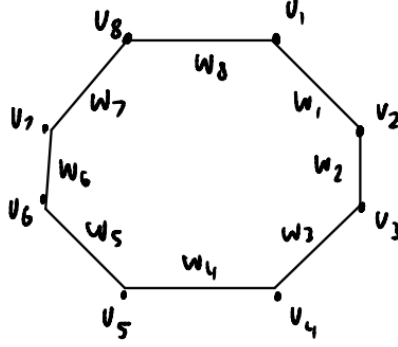


Figure 6: Cycle with edges that have non-integral values

$$\begin{aligned}
 x_{u_1 u_2} &\leftarrow x_{u_1 u_2} - \varepsilon \\
 x_{u_2 u_3} &\leftarrow x_{u_2 u_3} + \varepsilon \\
 &\vdots \\
 x_{u_7 u_8} &\leftarrow x_{u_7 u_8} - \varepsilon \\
 x_{u_8 u_1} &\leftarrow x_{u_8 u_1} + \varepsilon
 \end{aligned}$$

then $\sum_{u:(u,v) \in E} x_{uv}$ changes by $\varepsilon(w_2 + w_4 + w_6 + w_8) - \varepsilon(w_1 + w_3 + w_5 + w_7) = x^-$ (say)

Both x^+ and x^- must be equal to 0, otherwise the optimality of the LP solution is contradicted. Thus, now we can choose $\varepsilon = \min_{i \in \{0 \dots 8\}} (\min(x_{u_i u_{i+1}}, 1 - x_{u_i u_{i+1}}))$. Then we update the x_{ab} 's as above. Each such update makes at least one edge in the cycle have an integral x_{ab} .

In case we have a path and not a cycle, we can update the edge values in a similar way, considering that for the vertices at the start and end of the path, all edges connected to these vertices will have value 0 as they are integral (if they are not, we can simply add them to the path) and the value cannot be 1 as that would break the $x_{uv} \leq 1, \forall (u, v) \in E$ constraint. Thus in either case, in a finite number of steps, we will have an equally optimal integer solution.

In general, the value of the LP solution is the same as the value of the ILP solution when the matrix of constraints is totally unimodular.

5 Conclusion

In this lecture, we explored the theoretical foundations and practical implications of the Ellipsoid Method and LP duality, two cornerstones of optimization theory. The Ellipsoid Method was introduced as a polynomial-time algorithm for solving LP feasibility problems.

We delved into LP duality, discussing its role in bridging the primal and dual formulations of optimization problems. The Strong Duality Theorem and its applications, including the Max-Flow Min-Cut Theorem, demonstrated how duality can simplify problem analysis and solution derivation. Moreover, duality provides powerful insights into the structure of optimization problems, often making complex problems more tractable.

Finally, we looked at a special problem where the ILP optimal solution had a value equal to the LP optimal solution.

References

- [1] Leonid G. Khachiyan. “A Polynomial Algorithm in Linear Programming”. In: *Doklady Akademii Nauk SSSR* 244 (1979). Translated in Soviet Mathematics Doklady, 20(191–194), 1979, pp. 1093–1096.